

Development of a Spectral Element Method-based Incompressible Flow Solver

*A Thesis Submitted in Partial Fulfilment of
the Requirements for the Degree of*
Master of Technology

by

Nishant Nanasaheb Jagtap

Roll No. 234103329

Under supervision of

Dr. Atul Kumar Soti



**DEPARTMENT OF MECHANICAL ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI**

June, 2025

Certificate

This is to certify that the project report entitled “Development of a Spectral Element Method-based Incompressible Flow Solver” being submitted to the Department of Mechanical Engineering, Indian Institute of Technology (IIT) Guwahati by Mr. Nishant Nanasaheb Jagtap (Roll No. 234103329) for the partial fulfilment of the requirement for the degree of Master of Technology is a record of Bonafide research work carried out by him under my supervision and he fulfils the requirement of the regulations of the same. The results embodied in this report have not been submitted to any other University or Institute for the award of any degree.

June 13, 2025

Dr. Atul Kumar Soti
Assistant Professor
Department of Mechanical Engineering
Indian Institute of Technology Guwahati

Declaration

I certify that this written submission on “Development of a Spectral Element Method-based Incompressible Flow Solver” is written entirely by me, including citations to other authors' ideas or words. I further affirm that I have followed all rules governing academic honesty and integrity and that I have not created or faked any notion, data, fact, or source in my work. I am aware that any breach of the rules will subject the Institute to disciplinary action and may result in legal repercussions for the sources that were improperly cited or from whose sufficient permission was not obtained when required.

June 13, 2025

Nishant Nanasaheb Jagtap

Roll No. 234103329

Department of Mechanical Engineering

Indian Institute of Technology Guwahati

Guwahati, Assam, India

Acknowledgement

I owe my heartfelt gratitude and deepest regard to my supervisor, Dr. Atul Kumar Soti, Department of Mechanical Engineering, IIT Guwahati, India, for his relentless guidance, constant support, and full cooperation throughout the project. His methods, vision and enthusiasm has always been a constant motivation and inspiration to me. Without his supervision and direction, this project could not have been a success.

I am extremely grateful to Professor Shyamanta M. Hazarika (HOD, Department of Mechanical Engineering) for allowing me to use the research facilities available at the Department of Mechanical Engineering. I am also thankful to all the faculty members, technical/non-technical staffs, and research scholars of Department of Mechanical Engineering, IIT Guwahati, for rendering their whole-hearted cooperation and support in the entire course of work.

My deepest acknowledgment goes towards my family members who have been supportive right from the beginning of my master's journey. I would fail in my duty if I do not highlight here, Mr. Suman Swaraj Mishra (Ph.D. student, IIT Guwahati) for his great support without which this outcome would not have been possible. Mr. Suman's experience, research and his insights proved vital for this study.

Further, the Ministry of Human Resource Development (MHRD), Government of India through their scholarship scheme to the master's student has helped me with the basic financial requirements for which I am thankful.

Abstract

This study explores the application of the Spectral Element Method (SEM) for solving fluid flow problems governed by the Navier-Stokes equations. SEM, a hybrid of the Finite Element Method (FEM) and spectral methods, combines the geometric flexibility of FEM with the high-order accuracy of spectral techniques, making it particularly effective for solving problems on complex domains with coarse meshes. The study also incorporates the Fractional Step Method to decouple velocity and pressure computations, using unequal-order and equal order interpolation for velocity and pressure. Taylor-Hood element is considered to analyse decoupling through lower order FEM with a 2-1 order expansion function for velocity and pressure. Equal order interpolation for SEM gives accurate results with coarse mesh. Key benchmark problems, such as lid-driven cavity flow problem, flow around stationary square, circular and elliptical obstacles are simulated with validating against experimental and literature data. The Navier–Stokes equations are solved using both element-by-element runtime assembly and the global assembly approach. The global assembly approach uses PETSC Solver as well as inhouse Conjugate Gradient solver to iterate solution. Use of curvilinear mesh and linear mesh along with validation of lift-drag forces have been discussed. The findings highlight the advantages of SEM in achieving high precision and efficiency with ease in handling Coarse grids. This work underscores the potential of SEM for advanced Computational Fluid Dynamics (CFD) applications and lays the groundwork for future enhancements in solving coupled fluid-structure interaction problems.

Keywords: Spectral Element Method (SEM), Sparse Storage, Fractional Step Method (FSM), Element-by-Element (EBE), Conjugate Gradient (CG), PETSC

Table of Contents

Acknowledgement	iii
Abstract	iv
Table of Contents	v
Nomenclature	vii
List of Figures	ix
List of Tables	x
1. Introduction	1
1.1 Background	1
1.2 Organisation of the Report	2
2. Literature Review	3
2.1 Overview of the Spectral Element Method	3
2.2 Review of the Fractional Step Method	6
2.3 Analysis of Linear Solver and Sparse Storage	6
2.4 Review of Taylor-Hood elements and unequal order interpolation velocity & pressure	7
2.5 Review on Unstructured Grid	8
2.6 Objective of the Study	8
3. Methodology	9
3.1. Introduction	9
3.2. Governing Equations	9
3.3 Discretization of the Governing Equation	9
3.3.1. Method of Weighted Residual (MWR) and Nodal Expansion	9
3.3.2. Interpolating Polynomials and Their Roots	13
3.3.3. Numerical Integration	16
3.3.4. Element Level Assembly	17
3.3.5. Direct Summation Stiffness or Global Assembly	19
4. Unsteady Navier-Stokes Equation using the Fractional Step Method	23
4.1 Introduction	23
4.2 Unsteady Incompressible Navier-Stokes Equation	23
4.3 Fractional Step Method	24
4.4 Discretization of Governing Equation & Taylor Hood Element	25
4.5 Taylor Hood Element	36
4.6 Element by Element (EBE) Conjugate Gradient Method	37
4.5. Unstructured Grid	40

4.6 Linear Mesh / Straight Edge Mesh Elements	43
5. Code Restructuring and Solver Analysis	44
5.1 CPP Code Parallelization	44
5.2 Code Restructuring and Use of Other Solvers	45
5.3 PETSC Solver	46
5.4 CG Solver	47
6. Hydrodynamic Forces and Mesh Generation	51
6.1 Hydrodynamic Forces	51
6.1.1 Lift and Drag Forces	51
6.1.2 Normal Calculation	54
6.2 Spectral Element Mesh	55
6.3 Curvilinear Mesh / Curved Mesh Elements	57
6.3 Program Structure / Flowchart	58
7. Results and Discussion	59
7.1 Lid-Driven Cavity flow problem	59
7.2 Plane Poiseuille flow problem	65
7.3 Flow around Circular cylinder:	66
7.4 Flow around square cylinder:	73
7.5 Flow around Elliptical cylinder:	78
8. Summary and Future Work	86
8.1. Summary	86
8.2. Future Work	87
References	88

Nomenclature

Notations

L_c	Characteristic length
C	Connectivity Matrix
$ \cdot $	Determinant of matrix
D	Differentiation Matrix
L	Laplacian Matrix
M	Mass Matrix
D	Diameter of cylinder
U	Fluid velocity vector
U_∞	Free stream velocity
w	Integration weights
J	Jacobian Matrix
$\otimes$	Kronecker delta
∇^2	Laplacian operator
Δx	Node spacing in the x direction
Δy	Node spacing in y direction
$\bar{C}_l$	Mean lift coefficient
$\bar{C}_d$	Mean drag coefficient
N_v	Polynomial order for interpolating velocity
N_p	Polynomial order for interpolating Pressure
Q	Number of integration points or quadrature points
p	Pressure
Re	Reynolds number
f_s	Shedding frequency of the vortices
S_t	Strouhal number
Δt	Time step
F_D	Total drag force

Greek Symbols

Γ	Boundary
ϵ	Residue
μ	Dynamic viscosity
ν	Kinematic viscosity
ρ	Density of fluid
τ_w	Shear force

Ω	Domain
Ω_e	Elemental/subdomain
ψ_i	Test function
ψ_j	Basis function
Ψ	Interpolation function in 2D

Abbreviations

CFD	Computational Fluid Dynamics
FDM	Finite Difference Method
FEM	Finite Element Method
SEM	Spectral Element Method
FSI	Fluid-Structure Interaction
FSM	Fractional Step Method
EBE	Element By Element Conjugate Gradient
CV	Control Volume
CG	Conjugate Gradient
CFL	Courant Friedrich Lewy

Subscripts

I	Global x Index
i	Local x index or row number for matrices
J	Global y Index
j	Local y index or column number for matrices

Superscripts

e	Element number
n	n^{th} time step
$*$	Intermediate value
Leg	Legendre Polynomial
Lob	Lobatto Polynomial
RMS	Root Mean Square

List of Figures

Figure 3.1: Element of order 1	13
Figure 3.2: Distribution of Lobatto points in in 1D and 2D domain [2]	15
Figure 3.3 Domain transformation in 1D	16
Figure 3.4 Domain transformation in 2D	17
Figure 3.5 Global Assembly	19
Figure 3.6 Global Assembly in 1D and 2D domain	21
Figure 4.1 Pressure element in 1D domain	28
Figure 4.2 Taylor-Hood Element	37
Figure 4.3 1D Element-by-Element Assembly	38
Figure 4.4: Transformation of straight edge quadrilaterals (constant ξ and η are mapped into straight lines).....	43
<i>Figure 5.1: Workflow of PETSC Solver</i>	46
Figure 5.2: Workflow of in-house CG Solver	48
Figure 5.3: Structure of transformation of Matrix to Compressed Matrix & CSR Matrix	48
Figure 5.4: Global Assembly to form CSR Matrix	49
Figure 5.5: Comparison of lift-drag coefficient of in-house CG Solver with PETSC solver	50
Figure 6.1: Lift and Drag forces on bluff body	51
Figure 6.2: Lift and Drag forces components	53
Figure 6.3: Lift-Drag Parameters	55
Figure 6.4: Lift and Drag forces Calculation Flowchart	55
Figure 6.5: SEM Mesh Generation in Fortran	56
Figure 6.6: Curved Mesh Elements.....	57
Figure 6.7: Code Flowchart	58
Figure 7.1: Lid-Driven Cavity Flow	60
Figure 7.2: Velocity variation at mid-lines at Re=100	61
Figure 7.3: Velocity variation at mid-lines at Re=100, 400, & 1000	61
Figure 7.4: Velocity contours, stream function and vorticity contours at Re=100.....	62
Figure 7.5: Velocity contours, stream function and vorticity contours at Re=400.....	63
Figure 7.6: Velocity contours, stream function and vorticity contours at Re=1000.....	64
Figure 7.7: Velocity profile for Poiseuille's flow at Re=100	65
Figure 7.8 Schematic Diagram for circular cylinder.....	66
Figure 7.9 Taylor-Hood FEM Mesh for circular cylinder.....	67
Figure 7.10 Taylor-Hood FEM plots for circular cylinder.....	69
Figure 7.11 Initial GMSH coarse mesh and 4 th order SEM mesh for circular cylinder	71
Figure 7.12 Total Lift Coefficient plots for FEM & SEM Mesh for circular cylinder.....	71
Figure 7.13 Schematic Diagram for square cylinder	74
Figure 7.14 Initial GMSH coarse mesh and 4 th order SEM mesh for square cylinder.....	74
Figure 7.15 4 th order SEM plots for square cylinder.....	77
Figure 7.16 Schematic Diagram for elliptical cylinder.....	79
Figure 7.17 Initial GMSH coarse mesh and 4 th order SEM mesh for vertical elliptical cylinder	79
Figure 7.18: 4 th order SEM plots for elliptical cylinder	82
Figure 7.19: 45° backward inclined elliptical cylinder at Re = 100	84
Figure 7.20: 4 th order SEM plots for 45° backward inclined elliptical cylinder at Re = 100	85

List of Tables

Table 3.1: Different Sturm Liouville Eigenfunctions/polynomials.....	14
Table 3.2: Roots and weights of Lobatto polynomials.....	15
Table 3.3: Determinant of Jacobian for structured grid	17
Table 3.4: 1D Mass and Laplacian element matrix entries	19
Table 3.5: Local and Global Node numbering and connectivity matrix	20
Table 4.1: Mass, Laplacian and Differentiation matrix.....	27
Table 4.2: Basis functions in 2D	31
Table 4.3: Element-by-Element Assembly.....	39
Table 4.4: Shape functions for linear elements	40
Table 4.5: Bilinear Transformation	41
Table 4.6: Notations for transformed test/basis functions.....	42
Table 5.1: Parallelisation for EBE PCG Solver	45
Table 5.2: Results for CG with Jacobi Preconditioner	50
Table 5.3: Results for CG with No Preconditioner	50
Table 6.1: Equations of Lift and Drag Forces	51
Table 6.2: 1D shape functions and derivatives for lift calculation.....	54
Table 6.3: 2D shape functions for curved elements	57
Table 7.1: Results for various preconditioners for FEM mesh for circular cylinder at Re=100	69
Table 7.2: Lift and Drag coefficients for FEM Mesh for circular cylinder at Re = 100	70
Table 7.3: Validation of Lift & drag coefficients and Strouhal number with literature for circular cylinder at Re = 100	70
Table 7.4: Results for various SEM orders for circular cylinder at Re = 100	72
Table 7.5: Mesh/Order Independence for circular cylinder at Re = 100	73
Table 7.6: Results for various SEM orders for square cylinder at Re = 100	75
Table 7.7: Lift and Drag coefficients for 4 th order SEM Mesh for circular cylinder at Re=100	77
Table 7.8: Validation of Lift & drag coefficients and Strouhal number with literature for square cylinder at Re = 100	77
Table 7.9: Mesh/Order Independence for square cylinder at Re = 100	78
Table 7.10: Results for various SEM orders for vertical elliptical cylinder at Re = 150	80
Table 7.11: Results for various SEM orders for horizontal elliptical cylinder at Re = 150	80
Table 7.12: Lift & Drag coefficients for 4 th order SEM Mesh for elliptical cylinder at Re=150	82
Table 7.13: Validation of Lift & drag coefficients and Strouhal number with literature for elliptical cylinder at Re = 150	83
Table 7.14: Mesh/Order Independence for elliptical cylinder at Re = 150	83
Table 7.15: Lift-Drag calculation for 45° inclined elliptical cylinder at Re = 100	85

Introduction

1.1 Background

Computational Fluid Dynamics (CFD) is widely used across industries and academics for research, innovation, testing, simulations, etc, to deal with fluid flow governing equations. There are various approaches to dealing with CFD. Some of the significant approaches include FDM, FVM, and FEM. The finite Difference Method (FDM) is the oldest method for solving Partial Differential Equations (PDEs) by approximating the derivatives locally at each grid point. FDM works best with uniform meshes but has limitations in solving complex geometries and requires boundary-fitted nodes with distinct linear interpolation from surrounding nodes for each node. The finite Volume Method (FVM) partitions the domain into small sub-volumes and solves for the PDEs in their integral form in each volume. It is a widely used method in Fluid Mechanics (FM), and many current industry solvers, like Ansys Fluent, Open FOAM, etc., are based on this method. It uses the immediate surrounding neighbour data to interpolate and update the variables. FDM & FEM require updation of all equations and dependencies at each node if we wish to increase the accuracy by using higher order solution. Many other methods, like meshless methods, multigrid methods, domain decomposition, Lattice Boltzmann, etc., also deal with various CFD-related problems.

FEM or Finite Element Method also solves the PDEs in their integral form. It deals with equations in weak form with relaxed constraints and less sensitivity to discontinuities or singularities in the solution. The use of weighted residuals and test functions reduces the continuity requirement to satisfy the equations in an average sense rather than at every point. The domain is discretized in several non-overlapping elements, and the solution is approximated inside each element using polynomials or functions. Each element is transformed from a physical to a computational domain to apply the generalized solution of the master domain over each element. The interior nodes of an element are independent of the surrounding elements, which are taken care of by the common boundary element nodes. Flexibility regarding the order of interpolating polynomials, size, and shape of elements with Numerical Integration makes it easier to deal with complex geometries and control error and accuracy. FEM is widely used in Solid Structural Mechanics (SM), and it is used in some industry solvers for Fluid Mechanics like COMSOL, Nektar++, etc. Thus, FEM can be used in both solid and

fluid Mechanics and can be used for dealing with Fluid-Structure Interaction (FSI) problems which include moving body/mesh within the fluid.

Spectral Element Method (SEM) is a combination of spectral method and FEM to have high-order accuracy of spectral method with geometrical flexibility of FEM. The higher-order piecewise polynomials results in exponential convergence. This method involves partitioning the domain into multiple subdomains or components, with each subdomain's solution represented by higher-order basis functions. Within each element, a higher number of interior nodes automatically converts the coarse mesh into a fine mesh for generating a fine grid. Sparsity is still preserved like any other method, but the accuracy and precision of the solution increase. Patera first put forward this method [1] in 1984, and it has been successfully explored and implemented for various algorithms and problems, as explained later.

Lower unequal order interpolation and higher equal order interpolation for velocity and pressure have been investigated in this thesis. Use of element-by-element level assembly approach as well as global assembly approach with existing linear solver library and in-house solver has been explored. The use of linear and curvilinear meshes, along with calculating forces on any bluff body, is the main highlight of this work.

1.2 Organisation of the Report

Chapter 1 introduces various significant methods used in CFD and briefs about FEM and SEM. Chapter 2 covers the Literature review for SEM, FEM, and Fractional Step Method. It also discusses various techniques that can be used to solve systems of linear equations effectively. The chapter concludes by outlining the objectives of the present study. Chapter 3 provides an in-depth analysis and formulation of FEM and SEM equations for 1D and 2D Heat conduction equations. Chapter 4 covers the methodology and implementation of FEM/SEM for the general transient Navier-Stokes Equation, highlighting the critical aspects of the process. Chapter 5 discusses the approaches used for solving the system of equations arising from Navier-stokes formulation. Chapter 6 sheds light on the Lift-Drag Calculation, SEM Mesh generation and use of curved mesh. Chapter 7 presents the results for benchmark problems like the lid-driven cavity flow problem, Poiseuille flow problem, flow around square, circular, and elliptical cylinders, etc. Chapter 7 summarizes work undertaken so far and discusses the findings and shortcomings underlying the future direction of work.

Chapter 2

Literature Review

2.1 Overview of the Spectral Element Method

The Spectral Element Method (SEM) combines the advantages of lower-order method (FEM) with the accuracy of higher order method (Spectral Methods). Spectral series is fitted within each element. It uses Gaussian Quadrature to integrate the orthogonal basis functions with the orthogonal test functions to make the net residual zero to arrive at an approximate solution. The accuracy of this solution depends on the grid used and the order of interpolating basis and test functions (h-type and p-type methods) [2]. Exact or Inexact integration can be used with equal or unequal order interpolating polynomials for various variables within an element. A staggered grid can be automatically generated using polynomials of different orders for multiple variables, such as pressure and velocity, in the Navier-Stokes equations. The elemental-level basic matrix formulation can be easily applied and transformed to deal with unstructured grids. This approach offers numerous advantages, including high accuracy and reliable solution procedures with a limited number of elements across various conditions.

Schumack et al. [3] have tried to explain various ways to avoid the singularity in the solution due to spurious pressure nodes using the pseudospectral method and a method that solves the weak form of the governing equations. Staggered grids are not necessary to eliminate spurious pressure modes in SEM, and non-staggered grids can be used for decoupling velocity and pressure using different orders for velocity and pressure. These results give better accuracy than the staggered grid.

Hong Ma [4] has studied frontal phenomena in oceanic problems by solving Rossby modon's reflection problem using SEM with the explicit scheme. Shallow Water equations analyze the behavior of thin fluid layers on the Earth's surface. Since the grid resolution is not uniform, it imposes restrictions on the timestep for the initial value problem. Therefore, explicit methods must be handled more carefully to avoid divergence compared to implicit schemes.

Established by Navier in 1825, the spectral method became a standard approach for solving partial differential equations related to elastic sheets, approximating solutions within a single broad domain. In 1984, Patera [1] introduced a spectral method for multiple subdomains by combining the finite element method (FEM) with the spectral method, resulting in the

spectral element method (SEM). He used spectral element discretization to solve the one-dimensional inflow-outflow advection-diffusion equation and laminar two-dimensional (separated) flow in a channel expansion using high-order Lagrangian interpolants through Chebyshev collocation points.

The SEM is 10 times more efficient than FDM in the storage aspect for 1D cases [1]. This method offers numerous advantages, including high precision and dependable solution procedures, even with limited elements across various settings.

Shallal [5] used the cubic Hermite element to solve one-dimensional Burger's equation, widely used in mathematical models for turbulence, gas dynamics, heat conduction, elasticity, and shock wave theory. They tried to verify the unconditional stability of this method. At lower viscosity levels, mesh adaptivity becomes essential to resolve the exponential layer that develops near boundaries with weak viscosity. The spectral element method has the potential to overcome this challenge.

In the FSI problem of vortex-induced vibration of a cylinder, which is used to generate electricity, the spectral element method (SEM) is employed to solve the fluid and cylinder equations simultaneously, using Lagrangian basis functions based on Gauss-Lobatto-Legendre nodes [6].

The SEM method also generates a sparse matrix, unlike FEM, but has more nonzero entries in each row of the system of linear equations due to higher order polynomials approximation leading to more significant and more number of nodes in a single element. Solving the Linear system of equations using methods like LU factorization makes the matrix denser and destroys sparsity. It also increases memory storage and CPU computation time. The sparse linear finite element matrix built on the Gauss-Lobatto-Legendre nodes can be effectively solved using the preconditioned Conjugate Gradient method, Eigen solvers, Maps, Coordinate storage format, or Compressed matrix storage. The sparse linear finite element matrix, constructed on the Gauss-Lobatto-Legendre nodes of the spectral element mesh, can precondition the more comprehensive spectral element matrix, and the finite element preconditioned Conjugate Gradient Method helps to achieve rapid convergence for a symmetric system. Deville and Mund [7] [7] have used finite elements as preconditioners of pseudospectral algorithms to solve second-order elliptic partial differential equations.

To capture discontinuities in the solution field and deal with no-slip boundary conditions for compressible flows, FEM with streamlined upwind/Petrov-Galerkin (SUPG)

formulation was utilized by G. J. Le Beau & T. E. Tezduyar [8]. Euler's Equation considered no-penetration boundary condition implicitly with SUPG stabilization term and discontinuities in solution through shock capturing term along with Galerkin FEM term for stability. It was concluded to avoid excessive numerical diffusion and improve stability and convergence rates for large Courant numbers in implicit and explicit applications.

Brezzi et al. [9] used interpolations with bubble functions to stabilize the advective operator. Using bubble functions in both the advective-diffusive model and the linearized compressible Navier-Stokes equations avoids upwinding or other numerical techniques by linking stabilized finite elements and the Galerkin methods. For the advective-diffusive model, they showed that the Galerkin method with piecewise linear functions and bubble functions is equivalent to the streamline-upwind/Petrov-Galerkin (SUPG) method in the diffusive limit.

Fischer [10] developed a finite element-based additive Schwarz preconditioner for the most challenging pressure operator, contributing to stiffness for unsteady incompressible flows. It utilized overlapping subdomains and a coarse grid projection operator, directly applying pressure to the interior Gauss points. Various preconditioning strategies were discussed for the L^2 Poisson operator in the P_N - P_{N-2} spectral element formulation.

Tomar [11] presented a new h-p spectral element method to solve boundary value problems on non-smooth domains with mixed Neumann and Dirichlet boundary conditions. This method depicts exponentially accurate and asymptotically faster solutions using fully non-conforming SEM functions for pure Dirichlet problems and conforming functions only at the vertices of the elements for mixed problems. This algorithm resulted in a small Schur complement matrix, which, when used with the preconditioned conjugate gradient method and a distributed memory parallel computer, resulted in an optimal solution.

In a recent study by Jiang and Cheng [12], a comparison was made between two open-source codes: OpenFOAM, which is based on the conventional finite volume method, and Nektar++, which employs the high-order spectral/hp element method. The study aimed to assess the computational cost and efficiency of these approaches in achieving mesh convergence in numerical simulations. Upon reaching mesh convergence, they concluded Nektar++ to have approximately 3% of the estimated cost required for the OpenFOAM model.

2.2 Review of the Fractional Step Method

Chorin [13] and Temam introduced the Fractional Step Method (FSM) for decoupling the pressure and velocity components without an iterative approach. This method becomes more helpful in solving unsteady viscous incompressible flow equations, where dealing with iterative approaches like SIMPLE and PISO can be computationally expensive due to multiple iterations within each time step. The FSM ensures continuity by solving a pressure (or pseudo-pressure) Poisson equation only once per time step.

Several second-order projection methods, including Goda [14], Bell et al. [15], Kim and Moin [16], and Van Kan [17], have been presented. Brown et al. [18] have also discussed the accuracy of such methods. Rosenfeld et al. [19] applied an FSM to solve the two- and three-dimensional unsteady incompressible Navier-Stokes equations in generalized coordinate systems using primitive variable formulation and staggered grid. Codina and Blasco introduced a variational formulation of the Stokes problem using equal velocity-pressure finite element interpolation [20]. Limiting time steps ensures stability for the first-order projection approach. Codina [21] explored the stabilized fractional step method to ensure pressure stability in second-order incompressible flow systems where the momentum equation includes a fraction of the pressure gradient to avoid convergence sensitivity with timestep. Perot [22] employed block LU decomposition to investigate the performance of FSM in solving the incompressible Navier-Stokes equations using the Adams-Bashforth scheme for convective terms and the trapezoidal rule for diffusive terms.

2.3 Analysis of Linear Solver and Sparse Storage

Linear Solvers are used to solve the system of linear equations simultaneously, more efficiently to reduce computational time. Various algorithms like Gaussian Elimination, Conjugate Gradient (CG), Preconditioned CG, LU Decomposition, Simplex method, etc., are used to solve systems of linear equations.

Sparse Storage is a matrix representation technique for matrices containing many zero entries. Sparse storage methods can store only nonzero elements and thus reduce memory requirements for huge matrices with a maximum number of null entries.

Many built-in solvers using linear sparse storage algorithms and methods are already available in many programming languages. C++ and MATLAB use different ranges of these solvers like LDLT Decomposition, Cholesky Decomposition, Conjugate Gradient, ILU

Decomposition, etc. Eigen's sparse solvers [23] and PETSC KSP solvers[24] exhibit outstanding effectiveness and prompt determination of solutions for systems of equations containing sparse matrices. Fortran is compatible with PETSC and BLAS & LAPACK libraries, etc., to take advantage of their optimal direct and iterative linear solvers

Liu [25] provided an overview of the multifrontal method for solving large sparse symmetric positive definite linear systems. This method used frontal matrices, update matrices, and an assembly tree. Matrix reordering, supernodes, and other implementation techniques and advancements are discussed. Jacobi and Gauss-Seidel, Gaussian elimination, LU factorization, Cholesky factorization, Conjugate Gradient (CG), BiCGstab, and GMRES, etc., have been covered in Saad and Yousef [26] and Duff et al. [27] uses concepts like fill-reducing algorithms, pivoting techniques, and matrix reordering.

2.4 Review of Taylor-Hood elements and unequal order interpolation velocity & pressure

Taylor-Hood elements in fluid mechanics are used in FEM, which uses 1 order lower for interpolation pressure nodes as compared to velocity nodes in an element. This helps to avoid checkerboarding conditions and avoid oscillation in solutions. Taylor and Hood [28] have discussed the advantages of using this unequal order element approach over the stream function-vorticity approach for constrained flow problems. Taylor-Hood (TH) finite element uses continuous piecewise quadratic velocities and continuous piecewise linear pressures.

Svacek and Vacek [29] have discussed FSI problems using Taylor-Hood and Scott-Vogelius elements. Using elements satisfying the Babuska-Brezzi (BB) inf-sup condition leads to strong governance of the divergence constraint. They tried to simulate nonstationary flow around the vibrating cylinder using the arbitrary Lagrangian-Eulerian (ALE) method.

Douglas and Olaf [30] have presented space-time variational methods using the Taylor-Hood element to avoid inf-sup conditions arising in equal-order interpolation. They have solved the Stokes equation and Taylor-Green vortex and concluded that there is no need for stabilization techniques for diffusion-dominated problems. Hosseini et al. [31] used Taylor-Hood b-spline elements and performed an isogeometric analysis of the Navier-Stokes equation.

2.5 Review on Unstructured Grid

An unstructured grid is widely used in computational simulations and analysis to discretize geometries in parts of varying shapes and sizes, lacking a regular or structured arrangement.

Morgan and Peraire [32] examined the development of unstructured grid-based finite element methods for simulating fluid flows, focusing on solution techniques for the compressible Euler and Navier-Stokes equations. These methods utilize a Galerkin discretization for spatial representation and an appropriate finite-difference approach for time integration. The spatial discretization assumes the use of unstructured triangular assemblies in two dimensions and tetrahedral assemblies in three dimensions. Hughes and Thomas [33] discussed bilinear transformation, which maps quadrilateral shapes onto squares using shape functions. The calculation of derivatives of shape functions and the Jacobian for transformation was explained.

2.6 Objective of the Study

The objectives of this study are as follows:

- To develop unequal order Finite Element Method (FEM) solver for Velocity and Pressure interpolation (Taylor-Hood Element) for the numerical solution of the unsteady, incompressible 2D Navier-Stokes equation using Fractional Step Method.
- Extending the above unequal order solver for Spectral Element Method (SEM) to solve for coarse mesh and increase the accuracy of the solution using higher order basis functions.
- Developing an Element-by-Element level approach to avoid global assembly speeds up the process and reduces memory requirements for efficient computation.
- Utilising Sparse storage for storing matrices after global assembly and using fast linear solvers to accelerate the convergence of solutions.
- Correct the Lift-Drag Calculation part in the code to get the correct forces on the bluff body to solve FSI problems
- Develop an in-house Conjugate Gradient Solver for linear systems of equations using sparse storage format SPD matrices
- To test the solver and validate the results against major fluid flow test problems. To examine the order of accuracy of the Spectral Element solver for the different orders of polynomial expansion and computational time required.

Methodology

3.1. Introduction

In this section, we will deal with 1D and 2D Poisson's equations and apply the spectral element method to obtain discrete equations and solve the system of equations. We will convert the partial differential equation into its weak form to reduce the continuity requirement on the solution variable and obtain an approximate discrete equation. The domain is divided into multiple small subdomains/elements. A polynomial basis function represents the solution within each element. Gauss Quadrature points are used to integrate the equations within each subdomain to obtain Mass, Differential, and Laplacian matrices. A linear system of equations is formed using these matrices, which are solved to obtain a solution at each node within all elements.

3.2. Governing Equations

The following governing equations will be evaluated in this section:

$$\nabla^2 q(x) = f(x) \quad (3.1)$$

$$\frac{\partial^2 q(x, y)}{\partial x^2} + \frac{\partial^2 q(x, y)}{\partial y^2} = f(x, y) \quad (3.2)$$

Here, ∇^2 represents the Laplacian operator, $q(x)/q(x, y)$ are unknowns, and $f(x)/f(x, y)$ is the source term. The domain is represented by $\Omega \in [0, L_x]$ and $\Omega \in [0, L_x] \times [0, L_y]$, where L_x, L_y represent the extent of the domain in the x and y directions, respectively. The Dirichlet boundary condition for q is applied at all boundaries (Γ) with $q(\Gamma) = 0$.

3.3 Discretization of the Governing Equation

3.3.1. Method of Weighted Residual (MWR) and Nodal Expansion

The weighted residual method tries to reduce the difference between the approximate solution and the true solution using test functions. This method operates in two phases. In the first part, an approximate solution is obtained using basis functions. It leads to a residual that is minimized by multiplying with a test function in the second part. Minimizing the residual to a specific tolerance limit yields an approximately accurate solution.

There are different MWRs, but the most popular one in the FEM domain is Galerkin MWR, where the test and basis functions belong to the same polynomial family. This Galerkin MWR approach is also extended to SEM using higher-order test and basis functions.

We will analyse 1D case to understand the discretization and procedure of FEM/SEM method. We first obtain a Weak form of the second-order 1D governing equation as follows to reduce the continuity requirement:

$$\frac{d^2 q(x)}{dx^2} = f(x) \quad (3.3)$$

Here, q and f are approximated by the N^{th} order polynomial q_N and f_N , respectively, using nodal expansion with the test function $\psi_j(x)$:

$$q_N(x) = \sum_{j=0}^N q_j \psi_j(x) \quad (3.4)$$

$$f_N(x) = \sum_{j=0}^N f_j \psi_j(x) \quad (3.5)$$

Where f_j, q_j represent discrete values at the node where they are defined.

Here, the residual is generated as the approximate solution does not satisfy the governing PDE exactly. The Residual function estimated using N^{th} -order polynomial is expressed as follows:

$$\text{Residual} : \epsilon = \frac{d^2 q_N}{dx^2} - f_N \quad (3.6)$$

In MWR, we utilize the orthogonality of ψ_i and ϵ to minimize the error/residual by multiplying the test function and weak form residual function and integrating over the domain:

$$\int_{\Omega} \psi_i \epsilon d\Omega = \int_{\Omega} \psi_i \left(\frac{d^2 q_N}{dx^2} - f_N \right) d\Omega = 0 \quad (3.7)$$

In FEM, the entire domain is divided into small subdomains known as elements, and the discretized equations are evaluated for each element. These elemental equations are then assembled to form a global system of equations. The size of each element depends on the meshing of the domain which depends on the order of the assumed interpolation polynomial test and basis function. Lower order interpolation functions are used in FEM and higher order interpolation functions are used in SEM.

In the Continuous Galerkin method, the basis and test function ψ_j and ψ_i are defined at the element level instead of a global declaration. These functions are defined to be zero elsewhere outside the element.

For each element, equation (3.3) can be transformed as follows using following expansion functions for each variable and source term to get the required equation

$$q_N^{(e)}(x) = \sum_{j=0}^N q_j^{(e)} \psi_j(x) \quad (3.8)$$

$$f_N^{(e)}(x) = \sum_{j=0}^N f_j^{(e)} \psi_j(x) \quad (3.9)$$

$$\int_{\Omega_e} \psi_i \left(\frac{d^2 q_N^{(e)}}{dx^2} - f_N^{(e)} \right) d\Omega_e = 0 \quad (3.10)$$

Ω_e : *Element Domain* (superscript e depicts all quantities at elemental level)

Simplifying first term using Integration by parts, we get

$$\left[\psi_i \frac{dq_N^{(e)}}{dx} \right]_{\Gamma_e} - \int_{\Omega_e} \frac{d\psi_i}{dx} \left(\frac{dq_N^{(e)}}{dx} \right) d\Omega_e = \int_{\Omega_e} \psi_i f_N^{(e)} d\Omega_e \quad (3.11)$$

Where Γ_e is the boundary of the element Ω_e , and the term $\left[\psi_i \frac{dq_N^{(e)}}{dx} \right]_{\Gamma_e}$ vanishes due to homogeneous Dirichlet boundary conditions. This property holds because we will be employing Lagrange polynomials as our basis functions. Lagrange Polynomials are equal to one at point/node where they are defined and zero at all other nodes. This characteristic allows the flux from the interfaces of two elements to cancel each other out mutually, thus reducing equation to required form.

$$- \int_{\Omega_e} \frac{d\psi_i}{dx} \left(\frac{dq_N^{(e)}}{dx} \right) d\Omega_e = \int_{\Omega_e} \psi_i f_N^{(e)} d\Omega_e \quad (3.12)$$

If we have a non-zero Neumann Boundary condition, we must consider this extra term in each equation wherever required. Most of the fluid problems deal with zero Neumann boundary conditions, which makes equation (3.12) valid for almost all cases.

There are two approaches of interpolation – 1) Nodal and 2) Modal expansion functions. They are used to approximate the variable as a summation of basis functions. In Modal expansion, we use waves of distinct frequencies and amplitude to approximate the

variable in each domain. In the Nodal approach, we use the value of the function at specific interpolation points as our expansion coefficients[2] . As we are interested in solutions at some nodes in an element, we use the Nodal approach of interpolation.

The variable and the source term are interpolated using sum of nodal basis functions which are Lagrange polynomials using roots of Lobatto polynomials in each element. They are depicted in equations (3.8) and (3.9).

Using these equations in equation (3.12) lead to

$$- \int_{\Omega_e} \left[\left(\frac{\partial \psi_i}{\partial x} \right) \left(\frac{\partial \sum_{j=0}^N \psi_j q_j^{(e)}}{\partial x} \right) \right] d\Omega_e = \left[\int_{\Omega_e} \psi_i \sum_{j=0}^N \psi_j f_j^{(e)} d\Omega_e \right] \quad (3.12)$$

$$\begin{aligned} \sum_{j=0}^N \left\{ - \int_{\Omega_e} \left[\left(\frac{\partial \psi_i}{\partial x} \right) \left(\frac{\partial (\psi_j q_j^{(e)})}{\partial x} \right) \right] d\Omega_e \right\} &= \sum_{j=0}^N \left\{ \left[\int_{\Omega_e} \psi_i \psi_j f_j^{(e)} d\Omega_e \right] \right\} \\ \sum_{j=0}^N \left\{ - \left[\int_{\Omega_e} \left(\frac{d\psi_i}{dx} \right) \left(\frac{d\psi_j}{dx} \right) d\Omega_e \right] q_j^{(e)} - \left[\int_{\Omega_e} \psi_i \psi_j d\Omega_e \right] f_j^{(e)} \right\} &= 0 \end{aligned} \quad (3.13)$$

(a)

Equation (3.13) is valid for all test functions, $i \in [0, N]$

For linear interpolation function for all terms (N=1), the system of equation will be as follows:

$$\mathbf{i} = 0 \quad \left[\int_{\Omega_e} \left(\frac{d\psi_0}{dx} \right) \left(\frac{d\psi_0}{dx} \right) d\Omega_e \right] q_0^{(e)} - \left[\int_{\Omega_e} \left(\frac{d\psi_0}{dx} \right) \left(\frac{d\psi_1}{dx} \right) d\Omega_e \right] q_1^{(e)} \quad (3.13)$$

$$= \left[\int_{\Omega_e} \psi_0 \psi_0 d\Omega_e \right] f_0^{(e)} + \left[\int_{\Omega_e} \psi_0 \psi_1 d\Omega_e \right] f_1^{(e)} \quad (b)$$

$$\mathbf{i} = 1 \quad \left[\int_{\Omega_e} \left(\frac{d\psi_1}{dx} \right) \left(\frac{d\psi_0}{dx} \right) d\Omega_e \right] q_0^{(e)} - \left[\int_{\Omega_e} \left(\frac{d\psi_1}{dx} \right) \left(\frac{d\psi_1}{dx} \right) d\Omega_e \right] q_1^{(e)} \quad (3.13)$$

$$= \left[\int_{\Omega_e} \psi_1 \psi_0 d\Omega_e \right] f_0^{(e)} + \left[\int_{\Omega_e} \psi_1 \psi_1 d\Omega_e \right] f_1^{(e)} \quad (c)$$

This will form a matrix of systems, which will be explained later.

This approach can be extended in 2D, where each test and basis function will be a function of x and y. We can use an exact representation using tensor-product notation, described as shown below:

$$\Psi_k(x, y) = \psi_i(x) \psi_i(y) \quad (3.14)$$

In 1D case, we will use linear interpolation (1st order Lagrange polynomial) for the test function and basis function. The interpolation functions are expressed over a linear domain $\Omega \in [-1,1] \times \in [-1,1]$ as shown below

$$\psi_0(x) = \frac{(1-x)}{2} \quad \psi_1(x) = \frac{(1+x)}{2} \quad (3.15)$$

Extending this notation to 2D over a square domain, the basis functions can be represented as follows:

$$P_0 \quad \text{Bottom Left corner:} \quad \Psi_0(x, y) = \psi_0(x)\psi_0(y) \quad (3.16)$$

$$P_1 \quad \text{Bottom Right corner:} \quad \Psi_1(x, y) = \psi_1(x)\psi_0(y) \quad (3.17)$$

$$P_2 \quad \text{Top Left corner:} \quad \Psi_2(x, y) = \psi_0(x)\psi_1(y) \quad (3.18)$$

$$P_3 \quad \text{Top Right corner:} \quad \Psi_3(x, y) = \psi_1(x)\psi_1(y) \quad (3.19)$$

These basis functions are defined in domain as shown above and have unit value at its respective corners, everywhere else within the element the basis function value is zero.

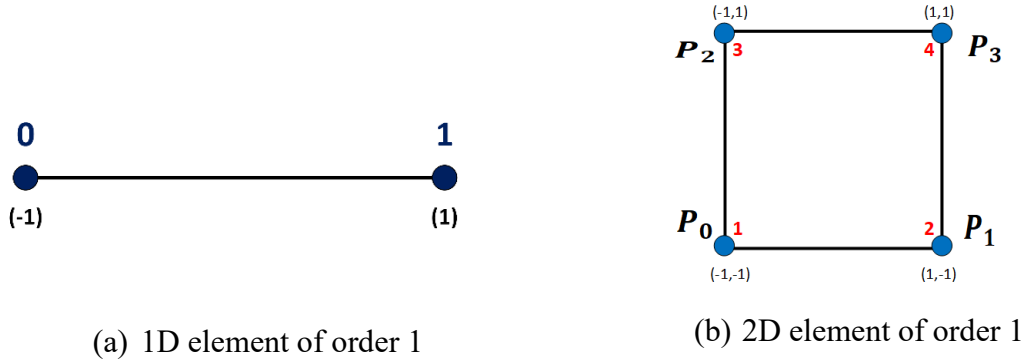


Figure 3.1: Element of order 1

3.3.2. Interpolating Polynomials and Their Roots

Nth-order Lagrange Polynomials are used as basis functions with nodes as (N+1) roots of the Lobatto polynomials. Using Lobatto polynomials' roots leads to a non-uniform distribution of nodes in the domain

The Legendre Polynomial, Lobatto Polynomials, Chebyshev polynomials, etc., are all types of eigenfunction polynomials that are derived from the Sturm-Liouville operator:

$$\frac{d}{dx} \left[p(x) \frac{d}{dx} \phi(x) \right] + \lambda_N w(x) \phi(x) = 0 \quad (3.20)$$

where,

$\phi(x)$: Eigenfunctions, $w(x)$: Weighting functions, λ : Eigenvalues

Which satisfies the differential equation for the specific domain bound (a, b)

For $p(x) = (1 - x^2)$ and $w(x) = 1$, the eigenfunction in domain $[-1,1]$ give solution as legendre polynomials which are defined as below:

$$\phi_0^{Leg}(x) = 1 \quad (3.21)$$

$$\phi_1^{Leg}(x) = x \quad (3.22)$$

$$\phi_N^{Leg}(x) = \frac{2N-1}{N} x \phi_{N-1}^{Leg}(x) - \frac{N-1}{N} \phi_{N-2}^{Leg}(x), \quad \forall N \geq 2 \quad (3.23)$$

These Legendre polynomials are used to calculate Lobatto Polynomials as follows:

$$\phi_N^{Lob}(x) = (1+x)(1-x) \frac{d}{dx} \phi_{N-1}^{Leg}(x) = (1-x^2) \phi_{N-1}'^{Leg}(x) \quad (3.24)$$

The examples of Lobatto and Legendre polynomials for different order are as follows:

Table 3.1: Different Sturm Liouville Eigenfunctions/polynomials

Order of Polynomial	Legendre Polynomial	Lobatto Polynomial
1	x	0
2	$\frac{1}{2}(3x^2 - 1)$	$(1 - x^2)$
3	$\frac{1}{2}(5x^3 - 3x)$	$3x(1 - x^2)$
4	$\frac{1}{8}(35x^4 - 30x^2 + 3)$	$\frac{1}{2}(15x^2 - 3)(1 - x^2)$

The same Lobatto polynomials roots are used as nodes as well as the integration points for Gaussian Quadrature/numerical Integration.

The Lagrange polynomials used as basis and test functions are as follows:

$$\psi_i(x) = L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^N \frac{(x - x_j)}{(x_i - x_j)} \quad (3.25)$$

The gauss quadrature weight associated for Lobatto polynomial roots are calculated using:

$$\omega_i = \frac{2}{N(N+1)(L_n(x_i))^2} \quad (3.26)$$

The roots of the Lobatto polynomial are calculated using Newton Raphson method with initial guess as Chebyshev polynomial roots:

$$\text{Initial guess: } x_i = \cos\left(\left[\frac{2i+1}{2N+2}\right]\pi\right), i = 0, 1, 2, 3, \dots, N \quad (3.27)$$

Lobatto polynomial roots are evaluated using Newton Raphson method as shown below:

$$x_{n+1} = x_n - \frac{\phi_N^{Lob}(x_n)}{\phi_N'^{Lob}(x_n)} = x_n - \frac{(1-x^2)\phi_N'^{Leg}(x_n)}{(1-x^2)\phi_N''^{Leg}(x_n) - (2x)\phi_N'^{Lob}(x_n)} \quad (3.28)$$

Here, ϕ_N^{leg} and ϕ_N^{Lob} represent the N^{th} Legendre and Lobatto polynomials respectively as discussed above and its derivatives are also calculated using equation (3.21-3.24) recursively.

The Lobatto polynomials roots and corresponding gauss quadrature weights are as follows:

Table 3.2: Roots and weights of Lobatto polynomials

Order of Lobatto polynomial	Roots	Weights
2	(-1, 1)	(1, 1)
3	(-1, 0, 1)	(1/3, 4/3, 1/3)
4	(-1, -0.447214, 0.447214, 1)	(5/3, 5/6, 5/6, 5/3)
5	(-1, -0.654654, 0, 0.654654, 1)	(1/10, 49/40, 32/45, 49/90, 1/10)

The Lobatto polynomial roots are used as nodal points as they directly enable us to map boundaries using nodes while transformation. No separate interpolation is required for boundary points. The Lobatto roots always include boundary nodes: -1 & 1. An example of a 2D physical domain mapped to a computational domain always accounts for boundary points as shown below (Transformation has been discussed in a later topic):

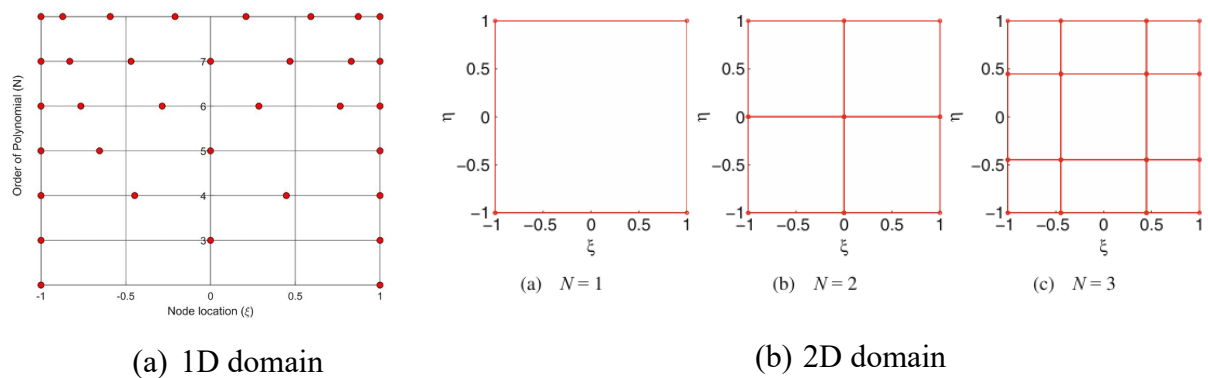


Figure 3.2: Distribution of Lobatto points in in 1D and 2D domain [2]

Another advantage of using the Lobatto polynomial roots is that they don't show any oscillation at domain endpoints while integrating as compared to oscillations depicted while using evenly spaced domain points for FEM. Also, the same nodal points are used as integration points due to a very negligible error in integration.

3.3.3. Numerical Integration

To integrate equation (3.13) inside the element, we use numerical integration as the function value f in $I = \int_{\Omega} f(x, y) d\Omega$ is only known at certain nodal points. To use Gaussian Quadrature Integration, we need to transform from the physical domain to the computational domain as depicted by the figure shown below.

Here, we choose the N th-degree Lobatto polynomial roots as integration points and Gauss-Lobatto-Legendre weights as calculated by equation (3.26). Using N degree Lobatto points can give an accurate result for integration up to $2N - 1$ degree polynomials

The numerical integration is performed on master domain & needs to be performed only once. It is evaluated for various physical domain sizes using corresponding transformation values.

The transformation for the structured grid for 1D & 2D case is achieved using 1st order Lagrange Interpolation shape functions as follows:

$$[1D] \quad x = \frac{1 - \xi}{2} x_0 + \frac{(1 + \xi)}{2} x_n \quad (3.29)$$

$$[x_0, x_n] \rightarrow [-1, 1]$$

$$\int_{x_0}^{x_n} f(x) dx = \int_{-1}^{+1} f(\xi) |J(\xi)| d\xi = \sum_{j=0}^N w_j f(\xi_j) |J(\xi_i)| \quad (3.30)$$



Figure 3.3 Domain transformation in 1D

Lobatto points include the domain endpoints, simplifying the implementation of Dirichlet boundary conditions at the nodes. However, this reduces the number of available quadrature points for integration, as two points are fixed (in 1D), thereby limiting the maximum degree of the integrand that can be accurately integrated.

$$[2D] \quad x = \frac{1 - \xi}{2} x_0 + \frac{(1 + \xi)}{2} x_n \quad y = \frac{1 - \xi}{2} y_0 + \frac{(1 + \xi)}{2} y_n \quad (3.31)$$

$$[x_0, x_n] \times [y_0, y_n] \rightarrow [-1, 1] \times [-1, 1]$$

$$\begin{aligned}
\int_{y_0}^{y_n} \int_{x_0}^{x_n} f(x, y) dx dy &= \int_{-1}^1 \int_{-1}^1 f(\xi, \eta) |J(\xi, \eta)| d\xi d\eta \\
\int_{y_0}^{y_n} \int_{x_0}^{x_n} f(x, y) dx dy &= \sum_{j=0}^{Q_\xi} \sum_{i=0}^{Q_\eta} w_i w_j f(\xi_i, \eta_j) |J(\xi_i, \eta_j)|
\end{aligned} \tag{3.32}$$

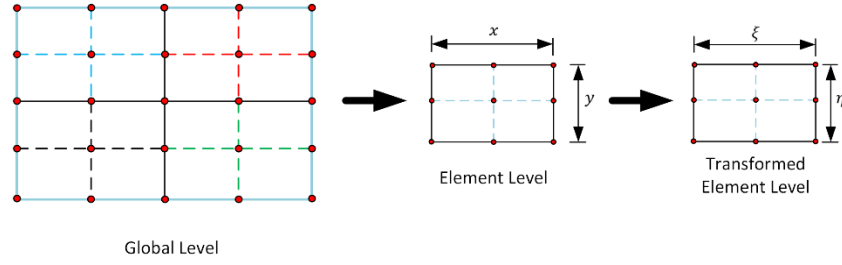


Figure 3.4 Domain transformation in 2D

where ξ_i and η_j are zeros of Q_ξ and Q_η degree polynomials with corresponding Gauss-Lobatto-Legendre weights respectively

Here, we have used Jacobian (J) - $|J(\xi)|$ and $|J(\xi, \eta)|$ for transforming the variables from physical space (x, y) to computational space (ξ, η) .

The determinant of Jacobian from equation (3.28) and (3.30) for structured grid is defined as:

Table 3.3: Determinant of Jacobian for structured grid

1D	$ J = \frac{\partial x}{\partial \xi} = \frac{\Delta x^{(e)}}{2} = \frac{x_n - x_0}{2}$	(3.33)
2D	$ J = \begin{vmatrix} \frac{\partial x}{\partial \xi} & \frac{\partial x}{\partial \eta} \\ \frac{\partial y}{\partial \xi} & \frac{\partial y}{\partial \eta} \end{vmatrix} = \frac{\Delta x^{(e)} \Delta y^{(e)}}{4} = \frac{(x_n - x_0)(y_n - y_0)}{4}$	(3.34)

3.3.4. Element Level Assembly

Now integrating equation (3.13) for each i (summation part (j) will be considered in form of column entries in element level matrices) using above transformation we get,

$$\sum_{j=0}^N \left\{ - \left[\int_{\Omega^e} \left(\frac{d\psi_i}{dx} \right) \left(\frac{d\psi_j}{dx} \right) d\Omega_e \right] q_j^{(e)} - \left[\int_{\Omega^e} \psi_i \psi_j d\Omega_e \right] f_j^{(e)} \right\} = 0$$

For 1D case: $x \in [x_0, x_n] \rightarrow \xi \in [-1, 1]$

$$\begin{aligned}
& - \left[\int_{\Omega^e} \left(\frac{d\psi_i(x)}{dx} \right) \left(\frac{d\psi_j(x)}{dx} \right) dx \right] q_j^{(e)} = \left[\int_{\Omega^e} \psi_i(x) \psi_j(x) dx \right] f_j^{(e)} \\
& - \left[\int_{-1}^1 \left(\frac{d\psi_i(\xi)}{d\xi} \right) \left(\frac{d\xi}{dx} \right) \left(\frac{d\psi_j(\xi)}{d\xi} \right) \left(\frac{d\xi}{dx} \right) \left(\frac{dx}{d\xi} \right) d\xi \right] q_j^{(e)} = \left[\int_{-1}^1 \psi_i(\xi) \psi_j(\xi) \left(\frac{dx}{d\xi} \right) d\xi \right] f_j^{(e)} \\
& - \left[\frac{2}{\Delta x^{(e)}} \int_{-1}^1 \left(\frac{d\psi_i(\xi)}{d\xi} \right) \left(\frac{d\psi_j(\xi)}{d\xi} \right) d\xi \right] q_j^{(e)} = \left[\frac{\Delta x^{(e)}}{2} \int_{-1}^1 \psi_i(\xi) \psi_j(\xi) d\xi \right] f_j^{(e)} \quad (3.35)
\end{aligned}$$

Using numerical integration, we get,

$$\begin{aligned}
& - \left[\frac{2}{\Delta x^{(e)}} \sum_{k=0}^{Q_\xi} w_k \left(\frac{d\psi_i(\xi_k)}{d\xi} \right) \left(\frac{d\psi_j(\xi_k)}{d\xi} \right) \right] q_j^{(e)} \\
& = \left[\frac{\Delta x^{(e)}}{2} \sum_{k=0}^{Q_\xi} w_k \psi_i(\xi_k) \psi_j(\xi_k) \right] f_j^{(e)} \quad (3.36)
\end{aligned}$$

Here, each ψ_i and ψ_j are N^{th} order polynomials, so the combined degree of $\psi_i \psi_j$ is $2N$, as discussed earlier we will require $Q=N+1$ point for integration which accurately integrates up to $2N+1$ order polynomial.

The use of Lobatto points as integration points do not account for significant error while performing integration with higher order as compared to gauss quadrature points.

Equation (3.36) can be expressed in matrix form as follows:

$$- \frac{2}{\Delta x^{(e)}} [L_{ij}] q_j^{(e)} = \frac{\Delta x^{(e)}}{2} [M_{ij}] f_j^{(e)} \quad (3.37)$$

$$- [L_{ij}^{(e)}] q_j^{(e)} = [M_{ij}^{(e)}] f_j^{(e)} \quad (3.38)$$

Here, the $L_{ij}^{(e)}$ and $M_{ij}^{(e)}$ are Laplacian and mass matrices defined at element level, i denotes the row index which corresponds to different test functions ψ_i and j denotes the column index which corresponds to different basis functions as depicted in (3.13) (a) and (3.13) (b). (Further discussion on Laplacian and Mass Matrix is performed in next chapter)

For 1D problem with 1st order basis functions,

$$\psi_0(\xi) = \frac{(1-\xi)}{2}, \quad \psi_1(\xi) = \frac{(1+\xi)}{2}$$

$$\frac{d\psi_0(\xi)}{d\xi} = \frac{(-1)}{2}, \quad \frac{d\psi_1(\xi)}{d\xi} = \frac{(1)}{2}$$

Each term of the Mass and Laplacian matrix can be expressed as follows

Table 3.4:1D Mass and Laplacian element matrix entries

Mass Matrix (M_{ij})	Laplacian Matrix (L_{ij})
$\int_{-1}^{+1} \psi_0 \psi_0 d\xi = \int_{-1}^{+1} \frac{1}{4} (1 - \xi)(1 - \xi) d\xi = \frac{2}{3}$	$\int_{-1}^{+1} \frac{d\psi_0}{d\xi} \frac{d\psi_0}{d\xi} d\xi = \int_{-1}^{+1} \frac{1}{4} (-1)(-1) d\xi = \frac{1}{2}$
$\int_{-1}^{+1} \psi_0 \psi_1 d\xi = \int_{-1}^{+1} \frac{1}{4} (1 - \xi)(1 + \xi) d\xi = \frac{1}{3}$	$\int_{-1}^{+1} \frac{d\psi_0}{d\xi} \frac{d\psi_1}{d\xi} d\xi = \int_{-1}^{+1} \frac{1}{4} (-1)(1) d\xi = -\frac{1}{2}$
$\int_{-1}^{+1} \psi_1 \psi_0 d\xi = \int_{-1}^{+1} \frac{1}{4} (1 + \xi)(1 - \xi) d\xi = \frac{1}{3}$	$\int_{-1}^{+1} \frac{d\psi_1}{d\xi} \frac{d\psi_0}{d\xi} d\xi = \int_{-1}^{+1} \frac{1}{4} (1)(-1) d\xi = -\frac{1}{2}$
$\int_{-1}^{+1} \psi_1 \psi_1 d\xi = \int_{-1}^{+1} \frac{1}{4} (1 + \xi)(1 + \xi) d\xi = \frac{2}{3}$	$\int_{-1}^{+1} \frac{d\psi_1}{d\xi} \frac{d\psi_1}{d\xi} d\xi = \int_{-1}^{+1} \frac{1}{4} (1)(1) d\xi = \frac{1}{2}$

Thus, 1st order element level Mass matrix and Laplacian matrix are as follows:

$$M_{ij}^{(e)} = \frac{\Delta x^{(e)}}{2} \begin{bmatrix} \int_{-1}^{+1} \psi_0 \psi_0 d\xi & \int_{-1}^{+1} \psi_0 \psi_1 d\xi \\ \int_{-1}^{+1} \psi_1 \psi_0 d\xi & \int_{-1}^{+1} \psi_1 \psi_1 d\xi \end{bmatrix} = \frac{\Delta x^{(e)}}{2} \begin{bmatrix} 2/3 & 1/3 \\ 1/3 & 2/3 \end{bmatrix} \quad (3.39)$$

$$L_{ij}^{(e)} = \frac{2}{\Delta x^{(e)}} \begin{bmatrix} \int_{-1}^{+1} \frac{d\psi_0}{d\xi} \frac{d\psi_0}{d\xi} d\xi & \int_{-1}^{+1} \frac{d\psi_0}{d\xi} \frac{d\psi_1}{d\xi} d\xi \\ \int_{-1}^{+1} \frac{d\psi_1}{d\xi} \frac{d\psi_0}{d\xi} d\xi & \int_{-1}^{+1} \frac{d\psi_1}{d\xi} \frac{d\psi_1}{d\xi} d\xi \end{bmatrix} = \frac{2}{\Delta x^{(e)}} \begin{bmatrix} 1/2 & -1/2 \\ -1/2 & 1/2 \end{bmatrix} \quad (3.40)$$

3.3.5. Direct Summation Stiffness or Global Assembly

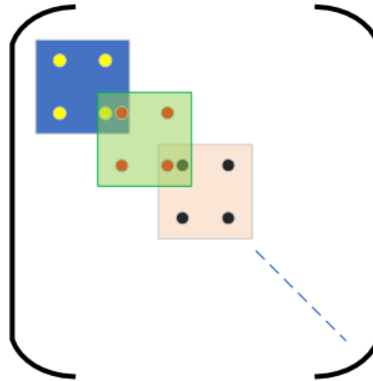


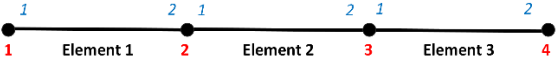
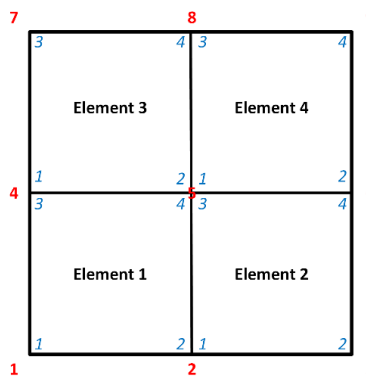
Figure 3.5 Global Assembly

As the domain is divided into subdomains/elements, the use of Lobatto points as nodes leads to the sharing of nodes between elements at boundaries. These boundary nodes share the information between elements and have contributions from all elements to which they belong when the system of equations is formed. Global assembly, also known as direct stiffness summation (DSS), is the process of adding up the components that contribute to a grid point. A global matrix is created by adding all of the elements' contributions at shared grid points.

This Global assembly is accompanied by a Connectivity matrix that stores the corresponding global node numbers for each element in the sequence of its local nodes. The connectivity matrix is a 2D matrix whose rows depict the element numbers and whose columns represent the corresponding global nodes in each element. Total number of elements in the domain are equal to number of rows in connectivity matrix, and the total number of nodes in each element are equal to number of columns in connectivity matrix.

The Global Node numbering and local node numbering is carried out as shown in figure below:

Table 3.5: Local and Global Node numbering and connectivity matrix

1D	2D
	
$C = \begin{bmatrix} 1 & 2 \\ 2 & 3 \\ 3 & 4 \end{bmatrix}$	$C = \begin{bmatrix} 1 & 2 & 4 & 5 \\ 2 & 3 & 4 & 6 \\ 4 & 5 & 7 & 8 \\ 5 & 6 & 8 & 9 \end{bmatrix}$

Here, the blue colour nodes depict local nodes, and red colour nodes depict global nodes, and the connectivity matrix is as shown in the table above.

Global Matrix can be represented as:

$$M_{IJ} = \bigwedge_{e=1}^{N_e} M_{ij}^{(e)} \quad (3.41)$$

$$L_{IJ} = \bigwedge_{e=1}^{N_e} L_{ij}^{(e)} \quad (3.42)$$

Here, I, J denote the global node numbers and i, j denote local node numbers which are assembled using DSS Λ operator.

Each entry of the local matrix $M_{ij}^{(e)}$ contribute to M_{IJ}

Where $I = C(e, i) - 1$ and $J = C(e, j) - 1$,

and C is the connectivity matrix, and -1 is subtracted for zero-based indexing.

Equation (3.37) after global assembly becomes,

$$-[L_{IJ}]q_J = [M_{IJ}]f_J \quad (3.43)$$

The contribution to the global matrix using the connectivity matrix is depicted in the figure below:

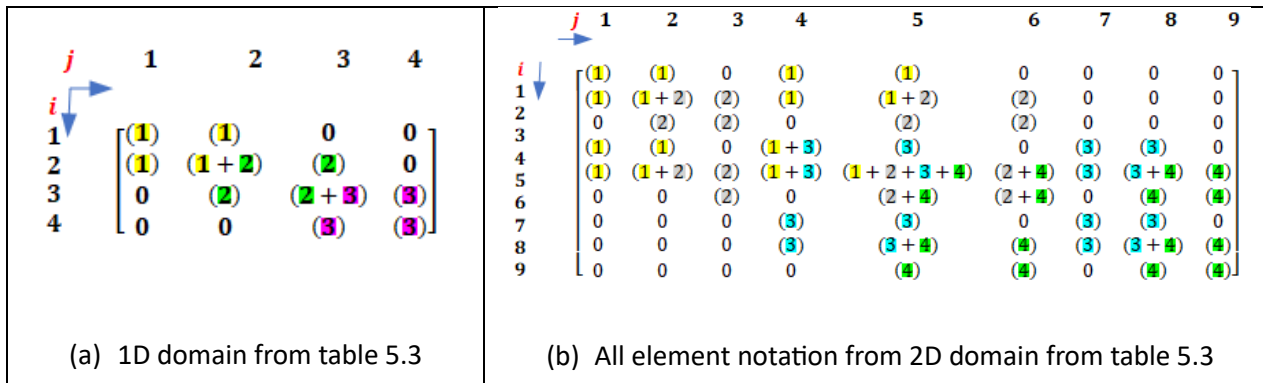


Figure 3.6 Global Assembly in 1D and 2D domain

Doing global assembly, using element level matrices from equation (3.39) and (3.40), and using connectivity matrix from table (3.5) for 1D domain with 3 elements and 4 nodes, we get the global mass and Laplacian matrices as follows:

$$M_{IJ} = \begin{bmatrix} 2/3 & 1/3 & 0 & 0 \\ 1/3 & 4/3 & 1/3 & 0 \\ 0 & 1/3 & 4/3 & 1/3 \\ 0 & 0 & 1/3 & 2/3 \end{bmatrix} \quad (3.44)$$

$$L_{IJ} = \begin{bmatrix} 1/2 & -1/2 & 0 & 0 \\ -1/2 & 1 & -1/2 & 0 \\ 0 & -1/2 & 1 & -1/2 \\ 0 & 0 & -1/2 & 1/2 \end{bmatrix} \quad (3.45)$$

Thus, equation (3.43) can be now represented in matrix form using equations (3.44) & (3.45) as follows:

$$\begin{bmatrix} -1/2 & 1/2 & 0 & 0 \\ 1/2 & -1 & 1/2 & 0 \\ 0 & 1/2 & -1 & 1/2 \\ 0 & 0 & 1/2 & -1/2 \end{bmatrix} \begin{Bmatrix} q_1 \\ q_2 \\ q_3 \\ q_4 \end{Bmatrix} = \begin{bmatrix} 2/3 & 1/3 & 0 & 0 \\ 1/3 & 4/3 & 1/3 & 0 \\ 0 & 1/3 & 4/3 & 1/3 \\ 0 & 0 & 1/3 & 2/3 \end{bmatrix} \begin{Bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \end{Bmatrix} \quad (3.46)$$

The above equation is linear system of equations:

$$Ax = b$$

where $A_{ij} = -L_{ij}$, $x_j = q_j$, and $b_i = M_{ij}f_j$. Here x_j 's are unknowns, f_j 's are known quantities which are used to find b_i 's. We have Dirichlet Homogeneous boundary condition at both ends for 1D domain. We need to update this known variable in the system of equations to get modified system of equations: $\bar{A}x = \bar{b}$. For this system, rows of A matrix for known variable values are made zero except the diagonal entry which is made 1.0. The b_i 's for these known variables is directly assigned the known value. And for each unknown variable nodes, the respective coefficients corresponding to all known variables are multiplied by the corresponding variable coefficient values from A and subtracted from that node's RHS(b_i 's) before making them zero. This approach is similar to ignoring the corresponding rows and columns of known variables from system of equations and reducing the system of equations, but the modification on the RHS needs to be performed to create accurate system of equations. The operation to eliminate boundary nodes from equations is depicted below:

$$\begin{bmatrix} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \\ a_{30} & a_{31} & a_{32} & a_{33} \end{bmatrix} \begin{Bmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \end{Bmatrix} = \begin{Bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{Bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & a_{11} & a_{12} & 0 \\ 0 & a_{21} & a_{22} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{Bmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \end{Bmatrix} = \begin{Bmatrix} q_0 \\ b_1 - a_{10}q_0 - a_{13}q_3 \\ b_2 - a_{20}q_0 - a_{23}q_3 \\ q_3 \end{Bmatrix}$$

Now this system of equations can be solved by various methods and solvers to increase the speed of calculation to reduce the number of iterations if solved by an iterative method.

Unsteady Navier-Stokes Equation using the Fractional Step Method

4.1 Introduction

This section deals with the unsteady Navier-Stokes equation in its weak form. The domain is divided into multiple small subdomains/elements. Polynomial basis functions for each local nodal variable are assumed within each element to approximate the solution. A local system of equations formed from local nodes is globally assembled to generate a global system of equations using global Mass, Laplacian, and Differential matrices after applying all required boundary conditions. Integrals are evaluated using numerical integration and linear systems of equation are solved using an iterative method to arrive at a Transient solution for velocity and pressure.

4.2 Unsteady Incompressible Navier-Stokes Equation

The Navier-Stokes (NS) equation for incompressible flow in non-dimensional form can be expressed as:

- Continuity Equation (2D)

$$\nabla \cdot \mathbf{U} = 0 \quad (4.1)$$

- Momentum Equation(2D)

$$\frac{\partial \mathbf{U}}{\partial t} + (\mathbf{U} \cdot \nabla) \mathbf{U} = -\nabla p + \frac{1}{Re^2} \nabla^2 \mathbf{U} \quad (4.2)$$

where $\mathbf{U} = u\hat{i} + v\hat{j} + w\hat{k}$, represent the fluid velocity in vector form, p is pressure, and Re is the Reynolds number: $Re = \frac{\rho U L_c}{\mu}$

The NS equation for 2D system can be expressed as

- Continuity equation:

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0 \quad (4.3)$$

- x-momentum equation ($\hat{i}$ direction)

$$\frac{\partial u}{\partial t} + \left(u \frac{\partial u}{\partial x} + v \frac{\partial u}{\partial y} \right) = -\nabla p + \frac{1}{Re^2} \left(\frac{\partial u}{\partial x^2} + \frac{\partial u}{\partial y^2} \right) \quad (4.4)$$

- y-momentum equation (*j direction*)

$$\frac{\partial v}{\partial t} + \left(u \frac{\partial v}{\partial x} + v \frac{\partial v}{\partial y} \right) = -\nabla p + \frac{1}{Re^2} \left(\frac{\partial v}{\partial x^2} + \frac{\partial v}{\partial y^2} \right) \quad (4.5)$$

4.3 Fractional Step Method

The fractional step method performs only 1 iteration at each time step, unlike other iterative methods. It consists of 3 steps in each iteration: a) Computation of intermediate velocity (u^* and v^*), b) computation of pressure (p), & c) computation of corrected velocity (u^{n+1} and v^{n+1}).

We will use the Adams Bashforth Method for the convective term, Crank-Nicolson method for the diffusive term, and first-order forward time discretization. Using these schemes, and discretizing the momentum and continuity equations (4.1) and (4.2), we get:

$$\nabla \cdot U^{n+1} = 0 \quad (4.6)$$

$$\begin{aligned} \frac{U^{n+1} - U^n}{\Delta t} + \left[\frac{3}{2} (U^n \cdot \nabla) U^n - \frac{1}{2} (U^{n-1} \cdot \nabla) U^{n-1} \right] \\ = -\nabla p^{n+1} + \frac{1}{2Re} \nabla^2 (U^{n+1} + U^n) \end{aligned} \quad (4.7)$$

Step 1: We employ Kim and Moin/Gresho [19] method for dealing with pressure term in momentum equation. We utilize their P1 method to calculate intermediate velocity U^* , where we ignore the pressure term (pressure term is set to zero) in momentum equation to determine intermediate velocity.

So, using intermediate velocity as new velocity in equation (4.7), we get:

$$\frac{U^* - U^n}{\Delta t} + \left[\frac{3}{2} (U^n \cdot \nabla) U^n - \frac{1}{2} (U^{n-1} \cdot \nabla) U^{n-1} \right] = \frac{1}{2Re} \nabla^2 (U^* + U^n) \quad (4.8)$$

Step 2: Subtracting equation (4.8) from (4.7) and ignoring the difference of diffusive term, we get,

$$\frac{U^{n+1} - U^*}{\Delta t} = -\nabla p^{n+1} \quad (4.9)$$

The next time step velocity (U^{n+1}) will be divergence free due to incompressibility condition and continuity constraint ($\nabla \cdot U^{n+1} = 0$).

Taking divergence on both sides, Equation (4.9) reduces to the pressure Poisson's equation:

$$(\nabla \cdot \nabla)p^{n+1} = \frac{\nabla \cdot \mathbf{U}^*}{\Delta t} \quad (4.10)$$

Thus, we calculate the pressure in the domain using intermediate velocity values from the above Poisson pressure equation.

Step 3: Using the calculated pressure value and the intermediate velocity value in equation (4.9), we get the next time step velocity.

$$\mathbf{U}^{n+1} = \mathbf{U}^* - \nabla p^{n+1} \Delta t \quad (4.11)$$

4.4 Discretization of Governing Equation & Taylor Hood Element

We will obtain the weak formulation of the above 3 equations using Galerkin MWR by multiplying the residual with the test function and using the orthogonality principle to find the approximate solution using assumed basis polynomial functions for velocity and pressure.

Checkerboarding conditions become prominent for high Reynolds number due to nonphysical and oscillatory pressure modes, which can disrupt fluid motion and compromise numerical accuracy. The staggered grid approach was developed to overcome this issue with the collocated grid, where linear interpolation of velocities in adjacent Control volumes (CV) and calculating the pressure gradient using central differencing caused a wrong solution. In a Staggered grid, the velocity and pressure nodes do not coincide in a CV, and thus checkerboarding can be avoided without using higher order interpolation. The SIMPLE scheme was developed by Patankar [34] to solve the NS equation by an iterative approach using a staggered grid.

Since, here we are using similar lower-order schemes for discretizing NS equation, therefore, to avoid a similar checkerboarding condition for the SEM approach, we will assume different order basis functions for pressure ($j = 0 - N_p$) and velocity ($j = 0 - N_v$) as shown below (order for 1D case). This approach gives us the freedom to use either a staggered grid ($N_v \neq N_p$) or collocated grid ($N_v = N_p$) as per our requirement.

To understand and validate our approach, we will solve NS equation using 2nd order ($N_v = 2$) approximation for velocity (*3X3 nodes in each element*) and 1st order ($N_p = 1$) approximation for pressure (*2X2 nodes in each element*), also known as Taylor Hood element. [28]. This lower order test and basis functions make the method FEM and not SEM

due to use of fine mesh to avoid divergence of solution. We will design the solver/algorithm for a general SEM (higher order) method with unequal order for velocity and pressure, but we will do our current analysis with $N_v = 2$, and $N_p = 1$.

Assuming basis functions for velocity and pressure as shown below, where $M_{N_v} = (N_{v\xi} + 1)(N_{v\eta} + 1) - 1$, and $N_{v\xi}$ and $N_{v\eta}$ are the orders of polynomial in the x and y directions, respectively. We are assuming equal order for velocity for same in both x and y direction, i.e. $N_{v\eta} = N_{v\xi} = N_v$, which implies $M_{N_v} = (N_v + 1)(N_v + 1) - 1$

Similarly, for pressure $M_{N_p} = (N_p + 1)(N_p + 1) - 1$, where N_p denotes the order of polynomial in both x and y direction.

Let $M_{N_v} = N_1$, and $M_{N_p} = N_2$

$$U_N^* = \sum_{j=0}^{M_{N_v}} \Psi_j^{N_v} U_j^* = \sum_{j=0}^{N_1} \Psi_j U_j^* \quad (4.12)$$

$$U_N^n = \sum_{j=0}^{M_{N_v}} \Psi_j^{N_v} U_j^n = \sum_{j=0}^{N_1} \Psi_j U_j^n \quad (4.13)$$

Velocity basis function:

$$U_N^{n-1} = \sum_{j=0}^{M_{N_v}} \Psi_j^{N_v} U_j^{n-1} = \sum_{j=0}^{N_1} \Psi_j U_j^{n-1} \quad (4.14)$$

$$U_N^{n+1} = \sum_{j=0}^{M_{N_v}} \Psi_j^{N_v} U_j^{n+1} = \sum_{j=0}^{N_1} \Psi_j U_j^{n+1} \quad (4.15)$$

Pressure basis function:

$$p_N^{n+1} = \sum_{j=0}^{M_{N_p}} \Psi_j^{N_p} p_j^{n+1} = \sum_{j=0}^{N_2} \Psi_j p_j^{n+1} \quad (4.16)$$

Integrating equation (4.8) with test function: Ψ_i , where $i = 0$ to N_1 (since velocity is unknown term in this equation), we get

$$\begin{aligned} \int_{\Omega} \Psi_i^{N_v} \left\{ \frac{U_N^* - U_N^n}{\Delta t} + \left[\frac{3}{2} (U_N^n \cdot \nabla) U_N^n - \frac{1}{2} (U_N^{n-1} \cdot \nabla) U_N^{n-1} \right] \right\} d\Omega \\ = \int_{\Omega} \Psi_i^{N_1} \left\{ \frac{1}{2Re} \nabla^2 (U_N^* + U_N^n) \right\} d\Omega \end{aligned} \quad (4.17)$$

Utilising integration by parts, and using Homogeneous Neumann boundary conditions, we can simplify the equation (4.17) at element level as follows:

$$\begin{aligned}
& \frac{1}{\Delta t} \int_{\Omega_e} \Psi_i^{N_v} (U_N^* - U_N^n) d\Omega_e \\
& + \left[\frac{3}{2} \int_{\Omega_e} \Psi_i^{N_v} (U_N^n \cdot \nabla) U_N^n d\Omega_e - \frac{1}{2} \int_{\Omega_e} \Psi_i^{N_v} (U_N^{n-1} \cdot \nabla) U_N^{n-1} d\Omega_e \right] \quad (4.18) \\
& = \frac{-1}{2Re} \int_{\Omega_e} \nabla \Psi_i^{N_v} \cdot \nabla U_N^* d\Omega_e - \frac{1}{2Re} \int_{\Omega_e} \nabla \Psi_i^{N_v} \cdot \nabla U_N^n d\Omega_e
\end{aligned}$$

Using equations (4.12), (4.13), and (4.14) in above equation we get,

$$\begin{aligned}
& \sum_{j=0}^{N_1} \left[\frac{1}{\Delta t} \left\{ \int_{\Omega_e} \Psi_i^{N_v} \Psi_j^{N_v} d\Omega_e \right\} (U_j^* - U_j^n) \right. \\
& + \frac{3}{2} \left[(u_i^n) \left\{ \int_{\Omega_e} \Psi_i^{N_v} \frac{\partial \Psi_j^{N_v}}{\partial x} d\Omega_e \right\} + (v_i^n) \left\{ \int_{\Omega_e} \Psi_i^{N_v} \frac{\partial \Psi_j^{N_v}}{\partial y} d\Omega_e \right\} \right] (U_j^n) \\
& - \frac{1}{2} \left[(u_i^{n-1}) \left\{ \int_{\Omega_e} \Psi_i^{N_v} \frac{\partial \Psi_j^{N_v}}{\partial x} d\Omega_e \right\} + (v_i^{n-1}) \left\{ \int_{\Omega_e} \Psi_i^{N_v} \frac{\partial \Psi_j^{N_v}}{\partial y} d\Omega_e \right\} \right] (U_j^{n-1}) \left. \right] \quad (4.19) \\
& = \sum_{j=0}^{N_1} \left[\frac{-1}{2Re} \left\{ \int_{\Omega_e} \nabla \Psi_i^{N_v} \cdot \nabla \Psi_j^{N_v} d\Omega_e \right\} (U_N^*) - \frac{1}{2Re} \left\{ \int_{\Omega_e} \nabla \Psi_i^{N_v} \cdot \nabla \Psi_j^{N_1} d\Omega_e \right\} (U_N^n) \right]
\end{aligned}$$

The $\{\}$ bracted terms refer to 3 types of matrices at element level as follows:

Table 4.1: Mass, Laplacian and Differentiation matrix

Mass Matrix:	$M_{ij}^{(e)} = M_{ij}^{(e, N_1)} = \int_{\Omega_e} \Psi_i^{N_v} \Psi_j^{N_v} d\Omega_e = \int_{\Omega_e} \Psi_i \Psi_j d\Omega_e$	(4.20)
Laplacian Matrix:	$L_{ij}^{(e)} = L_{ij}^{(e, N_1)} = \int_{\Omega_e} \nabla \Psi_i^{N_v} \cdot \nabla \Psi_j^{N_v} d\Omega_e = \int_{\Omega_e} \nabla \Psi_i \cdot \nabla \Psi_j d\Omega_e$	(4.21)
Differential Matrix:	$D_{ij}^{(e)} = D_{ij}^{(e, N_1)} = \int_{\Omega_e} U \cdot (\Psi_i^{N_v} \nabla \Psi_j^{N_v}) d\Omega_e = \int_{\Omega_e} U \cdot (\Psi_i \nabla \Psi_j) d\Omega_e$	(4.22)

(We are ignoring the size of square matrix (N_1) now for simplicity from superscript)

Here, each basis function is of order N_v so we are ignoring the superscript N_1 in further discussion for this section. Also, as the Differential matrix ($D_{ij}^{(e)}$) and Laplacian matrix ($L_{ij}^{(e)}$) contains partial derivative, they can be further expanded as follows:

$$D_{ij}^{(e, x)} = \int_{\Omega_e} \Psi_i \frac{\partial \Psi_j}{\partial x} d\Omega_e \quad D_{ij}^{(e, y)} = \int_{\Omega_e} \Psi_i \frac{\partial \Psi_j}{\partial y} d\Omega_e \quad (4.23)$$

$$L_{ij}^{(e, x)} = \int_{\Omega_e} \frac{\partial \Psi_i}{\partial x} \frac{\partial \Psi_j}{\partial x} d\Omega_e \quad L_{ij}^{(e, y)} = \int_{\Omega_e} \frac{\partial \Psi_i}{\partial y} \frac{\partial \Psi_j}{\partial y} d\Omega_e \quad (4.24)$$

Here, while evaluating all these integrals and matrices, we need to transform from physical domain $[x_0, x_n] \times [y_0, y_n]$ to computational domain $[-1, +1] \times [-1, +1]$

Using equation (3.30), for structured grid, the transformed variables will be as follows:

$$\xi(x) = \frac{2x - x_0}{\Delta x^{(e)}} - 1 \quad \eta(y) = \frac{2y - y_0}{\Delta y^{(e)}} - 1 \quad (4.25)$$

$$\frac{d\xi}{dx} = \frac{2}{\Delta x^{(e)}} \quad , \quad \frac{d\xi}{dy} = 0 \quad \frac{d\eta}{dy} = \frac{2}{\Delta y^{(e)}} \quad , \quad \frac{d\eta}{dx} = 0 \quad (4.26)$$

$$|J(\xi, \eta)| = \frac{\Delta x^{(e)} \Delta y^{(e)}}{4} \quad (4.27)$$

where $\Delta x^{(e)}$ and $\Delta y^{(e)}$ are dimensions of element in physical domain

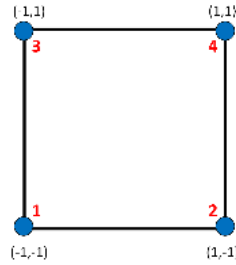


Figure 4.1 Pressure element in 1D domain

Instead of second order interpolation which is used for velocity, we will try to express above equations and matrices using first order interpolation for simplicity for understanding. Here, the basis functions which are used also correspond to pressure instead of velocity ($\Psi_i = \Psi_i^{N_p}$ & $\Psi_j = \Psi_j^{N_p}$).

For a 2D element of first order (these types of elements are used for interpolating pressure in Taylor-hood elements ($N_p = 1$)), there are 4 nodes in an element located at $(-1, -1)$, $(1, -1)$, $(-1, 1)$, & $(1, 1)$, as shown in figure.

$$\begin{aligned} \psi_{i\xi} &= \frac{1 + \xi_0 \xi}{2} & \psi_{i\eta} &= \frac{1 + \eta_0 \eta}{2} \\ \xi_0 &= (-1, 1) & \eta_0 &= (-1, 1) \\ \Psi_i &= \psi_{i\xi} \otimes \psi_{i\eta} \\ \Psi_i &= \left(\frac{1 + \xi_0 \xi}{2} \right) \left(\frac{1 + \eta_0 \eta}{2} \right) \\ (\xi_0, \eta_0) &= [(-1, -1), (1, -1), (-1, 1), (1, 1)] \end{aligned} \quad (4.28)$$

$$\frac{\partial \Psi_i}{\partial \xi} = \left(\frac{\xi_0}{2}\right) \left(\frac{1 + \eta_0 \eta}{2}\right) \quad \frac{\partial \Psi_i}{\partial \eta} = \left(\frac{1 + \xi_0 \xi}{2}\right) \left(\frac{\eta_0}{2}\right)$$

Now, equation (4.20) using transformation, Gaussian Quadrature for integration and equation (4.27) can be written as:

$$\begin{aligned} M_{ij}^{(e)} &= \int_{\Omega_e} \Psi_i(x, y) \Psi_j(x, y) d\Omega_e \\ M_{ij}^{(e)} &= \int_{-1}^{+1} \int_{-1}^{+1} \Psi_i(\xi, \eta) \Psi_j(\xi, \eta) |J(\xi, \eta)| d\xi d\eta \\ M_{ij}^{(e)} &= \sum_{s=0}^{Q_\eta} \sum_{r=0}^{Q_\xi} w_r w_s \Psi_i(\xi_r, \eta_s) \Psi_j(\xi_r, \eta_s) \left(\frac{\Delta x^{(e)} \Delta y^{(e)}}{4}\right) \end{aligned} \quad (4.29)$$

Here, $M_{ij}^{(e)}$ is mass matrix of dimension $(N_p + 1)^2 \times (N_p + 1)^2$, $|J(\xi, \eta)|$ is determinant of Jacobian matrix, Ψ_i and Ψ_j are polynomials of order N_p in both x and y direction, and $M(i, j)$ entry of matrix denotes when i^{th} node test function Ψ_i is multiplied by j^{th} node basis function Ψ_j and integrated over entire domain.

It results in a matrix of size M_{ij} of size 4×4 , as shown below, whose entries are calculated by numerical integration using Lobatto polynomial roots as integration points (Q_ξ & Q_η) similar to 1D case of equation (3.38), but using equation (4.28) as the basis function:

$$M_{ij}^{(e)} = \frac{\Delta x^{(e)} \Delta y^{(e)}}{36} \begin{bmatrix} 4 & 2 & 2 & 1 \\ 2 & 4 & 1 & 2 \\ 2 & 1 & 4 & 2 \\ 1 & 2 & 2 & 4 \end{bmatrix} \quad (4.30)$$

The Differential ($D_{ij}^{(e)}$) and Laplacian ($L_{ij}^{(e)}$) matrices are formed in similar manner, but they also use the derivatives of Ψ_i and Ψ_j in their equation. They both are of dimension $(N_p + 1)^2 \times (N_p + 1)^2$

Here, $D_{ij}^{(e,x)}$ and $D_{ij}^{(e,y)}$ are calculated as shown below which use the x and y derivate of the basis function Ψ_j respectively. All notations are the same as explained above for mass matrix.

The $D_{ij}^{(e,x)}$ matrix is calculated as follows:

$$D_{ij}^{(e,x)} = \int_{\Omega_e} \Psi_i(x, y) \frac{\partial \Psi_j(x, y)}{\partial x} d\Omega_e \quad (4.31)$$

$$D_{ij}^{(e,x)} = \int_{-1}^1 \int_{-1}^1 \Psi_i(\xi, \eta) \frac{\partial \Psi_j(\xi, \eta)}{\partial \xi} \frac{\partial \xi}{\partial x} |J(\xi, \eta)| d\xi d\eta$$

$$D_{ij}^{(e,x)} = \sum_{s=0}^{Q_\eta} \sum_{r=0}^{Q_\xi} w_r w_s \Psi_i(\xi_r, \eta_s) \frac{\partial \Psi_j}{\partial \xi} \Big|_{(\xi_r, \eta_s)} \left(\frac{2}{\Delta x^{(e)}} \right) \left(\frac{\Delta x^{(e)} \Delta y^{(e)}}{4} \right)$$

Similarly, $D_{ij}^{(e,y)}$ matrix is calculated as follows:

$$D_{ij}^{(e,y)} = \sum_{s=0}^{Q_\eta} \sum_{r=0}^{Q_\xi} w_r w_s \Psi_i(\xi_r, \eta_s) \frac{\partial \Psi_j}{\partial \eta} \Big|_{(\xi_r, \eta_s)} \left(\frac{2}{\Delta y^{(e)}} \right) \left(\frac{\Delta x^{(e)} \Delta y^{(e)}}{4} \right) \quad (4.32)$$

The differentiation matrix $D_{ij}^{(e,x)}$ and $D_{ij}^{(e,y)}$ in the x direction can be represented as:

$$D_{ij}^{(e,x)} = \frac{\Delta y^{(e)}}{12} \begin{bmatrix} -2 & 2 & -1 & 1 \\ -2 & 2 & -1 & 1 \\ -1 & 1 & -2 & 2 \\ -1 & 1 & -2 & 2 \end{bmatrix} \quad D_{ij}^{(e,y)} = \frac{\Delta x^{(e)}}{12} \begin{bmatrix} -2 & -1 & 2 & 1 \\ -1 & -2 & 1 & 2 \\ -2 & -1 & 2 & 1 \\ -1 & -2 & 1 & 2 \end{bmatrix} \quad (4.33)$$

The Laplacian Matrix $L_{ij}^{(e,x)}$ & $L_{ij}^{(e,y)}$ can be evaluated using partial derivatives of both test & basis functions with respect to x & y respectively as given by equation (4.24). All notations & terms remain same as explained earlier, & the matrix can be calculated as follows:

$$L_{ij}^{(e,x)} = \int_{\Omega_e} \frac{\partial \Psi_i(x, y)}{\partial x} \frac{\partial \Psi_j(x, y)}{\partial x} d\Omega_e$$

$$L_{ij}^{(e,x)} = \int_{-1}^1 \int_{-1}^1 \frac{\partial \Psi_i(\xi, \eta)}{\partial \xi} \frac{\partial \xi}{\partial x} \frac{\partial \Psi_j(\xi, \eta)}{\partial \xi} \frac{\partial \xi}{\partial x} |J(\xi, \eta)| d\xi d\eta$$

$$L_{ij}^{(e,x)} = \int_{-1}^1 \int_{-1}^1 \frac{\partial \Psi_i(\xi, \eta)}{\partial \xi} \left(\frac{2}{\Delta x^{(e)}} \right) \frac{\partial \Psi_j(\xi, \eta)}{\partial \xi} \left(\frac{2}{\Delta x^{(e)}} \right) \left(\frac{\Delta x^{(e)} \Delta y^{(e)}}{4} \right) d\xi d\eta \quad (4.34)$$

$$L_{ij}^{(e,x)} = \sum_{s=0}^{Q_\eta} \sum_{r=0}^{Q_\xi} w_r w_s \frac{\partial \Psi_i}{\partial \xi} \Big|_{(\xi_r, \eta_s)} \frac{\partial \Psi_j}{\partial \xi} \Big|_{(\xi_r, \eta_s)} \left(\frac{\Delta y^{(e)}}{\Delta x^{(e)}} \right)$$

Similarly, $L_{ij}^{(e,y)}$ matrix is calculated as follows:

$$L_{ij}^{(e,y)} = \sum_{s=0}^{Q_\eta} \sum_{r=0}^{Q_\xi} w_r w_s \frac{\partial \Psi_i}{\partial \eta} \Big|_{(\xi_r, \eta_s)} \frac{\partial \Psi_j}{\partial \eta} \Big|_{(\xi_r, \eta_s)} \left(\frac{\Delta x^{(e)}}{\Delta y^{(e)}} \right) \quad (4.35)$$

The Laplacian matrix $L_{ij}^{(e,x)}$ and $L_{ij}^{(e,y)}$ in the x & y direction can be represented as:

$$L_{ij}^{(e,x)} = \frac{\Delta y^{(e)}}{6\Delta x^{(e)}} \begin{bmatrix} 2 & -2 & 1 & -1 \\ -2 & 2 & -1 & 1 \\ 1 & -1 & 2 & -2 \\ -1 & 1 & -2 & 2 \end{bmatrix} \quad L_{ij}^{(e,y)} = \frac{\Delta x^{(e)}}{6\Delta y^{(e)}} \begin{bmatrix} 2 & 1 & -2 & -1 \\ 1 & 2 & -1 & -2 \\ -2 & -1 & 2 & 1 \\ -1 & -2 & 1 & 2 \end{bmatrix} \quad (4.36)$$

All above matrices are calculated for pressure nodal approximation as pressure is approximated using first order basis functions.

For velocity, we use second order basis functions, for which the interpolating polynomial is expressed as follows:

Table 4.2: Basis functions in 2D

	Basis/test functions in ξ direction	Basis/test functions in η direction	(4.37)
$i = 0$	$\psi_{0\xi} = \frac{(\xi)(\xi - 1)}{2}$	$\psi_{0\eta} = \frac{(\eta)(\eta - 1)}{2}$	
$i = 1$	$\psi_{1\xi} = \frac{(\xi + 1)(\xi - 1)}{-1}$	$\psi_{1\eta} = \frac{(\eta + 1)(\eta - 1)}{-1}$	
$i = 2$	$\psi_{2\xi} = \frac{(\xi)(\xi + 1)}{2}$	$\psi_{2\eta} = \frac{(\eta)(\eta + 1)}{2}$	
$\Psi_i = \psi_{i\xi} \otimes \psi_{i\eta}$			

Expressing equation using the mass, Laplacian, and differential matrices for velocity $U = u\hat{i} + v\hat{j}$, we get following two u and v velocity correction equations:

$$\begin{aligned}
M_{ij}^{(e)}(U_j^*) - \frac{\Delta t}{2Re} [L_{ij}^{(e)}] U_j^* \\
= \left[M_{ij}^{(e)} - \frac{\Delta t}{2Re} [L_{ij}^{(e)}] - \frac{3\Delta t}{2} D_{ij}^{(e)(n)} \right] U_j^n + \left[\frac{\Delta t}{2} D_{ij}^{(e)(n-1)} \right] U_j^{n-1}
\end{aligned}$$

$$\begin{aligned}
M_{ij}^{(e)}(u_j^*) - \frac{\Delta t}{2Re} [L_{ij}^{(e)}] u_j^* \\
= \left[M_{ij}^{(e)} - \frac{\Delta t}{2Re} [L_{ij}^{(e)}] - \frac{3\Delta t}{2} D_{ij}^{(e)(n)} \right] u_j^n + \left[\frac{\Delta t}{2} D_{ij}^{(e)(n-1)} \right] u_j^{n-1}
\end{aligned} \tag{4.38}$$

$$\begin{aligned}
M_{ij}^{(e)}(v_j^*) - \frac{\Delta t}{2Re} [L_{ij}^{(e)}] v_j^* \\
= \left[M_{ij}^{(e)} - \frac{\Delta t}{2Re} [L_{ij}^{(e)}] - \frac{3\Delta t}{2} D_{ij}^{(e)(n)} \right] v_j^n + \left[\frac{\Delta t}{2} D_{ij}^{(e)(n-1)} \right] v_j^{n-1}
\end{aligned} \tag{4.39}$$

Now we need to solve equation (4.10) to update pressure using intermediate velocity values.

Integrating equation (4.10) with test function: Ψ_i , where $i = 0$ to N_2 (since pressure is unknown term in this equation) at elemental level, we get

$$\begin{aligned}
\int_{\Omega_e} \Psi_i^{N_p} (\nabla^2 p_N^{n+1}) d\Omega_e &= \frac{1}{\Delta t} \int_{\Omega_e} \Psi_i^{N_p} (\nabla \cdot U_N^*) d\Omega_e \\
\int_{\Omega_e} \Psi_i^{N_p} (\nabla^2 p_N^{n+1}) d\Omega_e &= \frac{1}{\Delta t} \int_{\Omega_e} \Psi_i^{N_p} \left(\frac{\partial u_N^*}{\partial x} + \frac{\partial v_N^*}{\partial y} \right) d\Omega_e
\end{aligned} \tag{4.40}$$

Using basis function for velocity and pressure as defined in equations (4.13) and (4.16) and simplifying using integration by parts, we can express the updated pressure as follows:

$$\begin{aligned}
& - \sum_{j=0}^{N_2} \left\{ \int_{\Omega_e} \nabla \Psi_i^{N_p} \cdot \nabla \Psi_j^{N_p} d\Omega_e \right\} (p_j^{n+1}) \\
&= \frac{1}{\Delta t} \left[\sum_{j=0}^{N_1} \left[\left\{ \Psi_i^{N_p} \frac{\partial \Psi_j^{N_v}}{\partial x} d\Omega_e \right\} (u_j^*) + \left\{ \Psi_i^{N_p} \frac{\partial \Psi_j^{N_v}}{\partial y} d\Omega_e \right\} (v_j^*) \right] \right]
\end{aligned} \tag{4.41}$$

Here, the $\{\}$ term on LHS represents the Laplacian term for pressure, but the two $\{\}$ terms on RHS are not representing a square matrix, on other hand they are of dimensions $(N_2 \times N_1)$. These non-square matrices are formed due to difference in interpolation order of velocity and pressure and to convert the quantity evaluated on RHS in terms of velocity-based nodes into pressure-based nodes on LHS.

The above expression can be expressed as follows:

$$- L_{ij}^{(e, N_2)} (p_j^{n+1}) = \frac{1}{\Delta t} \left[D_{ij}^{(e, x, N_2, N_1)} (u_j^*) + D_{ij}^{(e, y, N_2, N_1)} (v_j^*) \right] \tag{4.42}$$

Here, the $D_{ij}^{(e, x, N_2, N_1)}$ and $D_{ij}^{(e, y, N_2, N_1)}$ matrices have dimension $(N_p + 1) \times (N_v + 1)$, Ψ_i is polynomials of order N_p in both x and y direction, Ψ_j is polynomials of order N_v in both x and y direction on RHS and $D^{(e, x, N_2, N_1)}(i, j)$ entry of matrix denotes when i^{th} node test function Ψ_i is multiplied by j^{th} node basis function Ψ_j and integrated over entire domain

$$\begin{aligned}
D_{ij}^{(e, x, N_2, N_1)} &= \int_{\Omega_e} \Psi_i^{N_p} (x, y) \frac{\partial \Psi_j^{N_v} (x, y)}{\partial x} d\Omega_e \\
D_{ij}^{(e, x, N_2, N_1)} &= \int_{-1}^1 \int_{-1}^1 \Psi_i^{N_p} (\xi, \eta) \frac{\partial \Psi_j^{N_v} (\xi, \eta)}{\partial \xi} \frac{\partial \xi}{\partial x} |J(\xi, \eta)| d\xi d\eta \\
D_{ij}^{(e, x, N_2, N_1)} &= \sum_{s=0}^{Q_\eta} \sum_{r=0}^{Q_\xi} w_r w_s \Psi_i^{N_p} (\xi_r, \eta_s) \frac{\partial \Psi_j^{N_v}}{\partial \xi} \bigg|_{(\xi_r, \eta_s)} \left(\frac{2}{\Delta x^{(e)}} \right) \left(\frac{\Delta x^{(e)} \Delta y^{(e)}}{4} \right)
\end{aligned} \tag{4.43}$$

Similarly, $D_{ij}^{(e,y,N_2,N_1)}$ matrix is calculated as follows:

$$D_{ij}^{(e,y,N_2,N_1)} = \sum_{s=0}^{Q_\eta} \sum_{r=0}^{Q_\xi} w_r w_s \Psi_i^{N_p}(\xi_r, \eta_s) \frac{\partial \Psi_j^{N_v}}{\partial \eta} \bigg|_{(\xi_r, \eta_s)} \left(\frac{2}{\Delta y^{(e)}} \right) \left(\frac{\Delta x^{(e)} \Delta y^{(e)}}{4} \right) \quad (4.44)$$

It results in a differential matrix of size 4×9 , as shown below, whose entries are calculated by numerical integration using Lobatto polynomial roots as integration points (Q_ξ & Q_η) similar to 1D case of equation (3.38), but using equation (4.28) as the basis function:

$$D_{ij}^{(e,x,N_2,N_1)} = \frac{\Delta y^{(e)}}{36\Delta} \begin{bmatrix} -5 & 4 & 1 & -10 & 8 & 2 & 0 & 0 & 0 \\ -1 & -4 & 5 & -2 & -8 & 10 & 0 & 0 & 0 \\ 0 & 0 & 0 & -10 & 8 & 2 & -5 & 4 & 1 \\ 0 & 0 & 0 & -2 & -8 & 10 & -1 & -4 & 5 \end{bmatrix} \quad (4.45)$$

$$D_{ij}^{(e,y,N_2,N_1)} = \frac{\Delta x^{(e)}}{36\Delta} \begin{bmatrix} -5 & -10 & 0 & 4 & 8 & 0 & 1 & 2 & 0 \\ 0 & -10 & -5 & 0 & 8 & 4 & 0 & 2 & 1 \\ -1 & -2 & 0 & -4 & -8 & 0 & 5 & 10 & 0 \\ 0 & -2 & -1 & 0 & -8 & -4 & 0 & 10 & 5 \end{bmatrix} \quad (4.46)$$

Now, Solving equation (4.11), we get the new corrected velocity

We need to solve equation (4.11) to get corrected velocity using intermediate velocity values and calculated pressure values using equation (4.48).

Integrating equation (4.11) with test function: Ψ_i , where $i = 0$ to N_1 (since velocity is unknown term in this equation) at elemental level, we get

$$\begin{aligned} \frac{1}{\Delta t} \int_{\Omega_e} \Psi_i^{N_v} (U_N^{n+1} - U_N^*) d\Omega_e &= - \int_{\Omega_e} \Psi_i^{N_v} (\nabla p_N^{n+1}) d\Omega_e \\ \frac{1}{\Delta t} \int_{\Omega_e} \Psi_i^{N_v} (U_N^{n+1} - U_N^*) d\Omega_e &= - \int_{\Omega_e} \Psi_i^{N_v} \left(\frac{\partial p_N^{n+1}}{\partial x} + \frac{\partial p_N^{n+1}}{\partial y} \right) d\Omega_e \\ \frac{1}{\Delta t} \left\{ \int_{\Omega_e} \Psi_i^{N_v} \Psi_j^{N_v} d\Omega_e \right\} (U_j^{n+1} - U_j^*) & \\ &= - \left[\left\{ \int_{\Omega_e} \Psi_i^{N_v} \frac{\partial \Psi_j^{N_v}}{\partial x} d\Omega_e \right\} + \left\{ \int_{\Omega_e} \Psi_i^{N_v} \frac{\partial \Psi_j^{N_v}}{\partial y} d\Omega_e \right\} \right] (p_j^{n+1}) \end{aligned} \quad (4.47)$$

Segregating the velocity U into components we get,

$$\frac{1}{\Delta t} \left\{ \int_{\Omega_e} \Psi_i^{N_v} \Psi_j^{N_v} d\Omega_e \right\} (u_j^{n+1} - u_j^*) = - \left[\left\{ \int_{\Omega_e} \Psi_i^{N_v} \frac{\partial \Psi_j^{N_p}}{\partial x} d\Omega_e \right\} \right] (p_j^{n+1})$$

$$\frac{1}{\Delta t} M_{ij}^{(e,N_1)} (u_j^{n+1} - u_j^*) = -D_{ij}^{(e,x,N_1,N_2)} (p_j^{n+1}) \quad (4.48)$$

$$M_{ij}^{(e,N_1)} (u_j^{n+1}) = M_{ij}^{(e,N_1)} (u_j^*) - \Delta t D_{ij}^{(e,x,N_1,N_2)} (p_j^{n+1})$$

$$\frac{1}{\Delta t} \left\{ \int_{\Omega_e} \Psi_i^{N_v} \Psi_j^{N_v} d\Omega_e \right\} (v_j^{n+1} - v_j^*) = - \left[\left\{ \int_{\Omega_e} \Psi_i^{N_v} \frac{\partial \Psi_j^{N_p}}{\partial y} d\Omega_e \right\} \right] (p_j^{n+1})$$

$$\frac{1}{\Delta t} M_{ij}^{(e,N_1)} (v_j^{n+1} - v_j^*) = -D_{ij}^{(e,y,N_1,N_2)} (p_j^{n+1}) \quad (4.49)$$

$$M_{ij}^{(e,N_1)} (v_j^{n+1}) = M_{ij}^{(e,N_1)} (v_j^*) - \Delta t D_{ij}^{(e,y,N_1,N_2)} (p_j^{n+1})$$

Here, the $D_{ij}^{(e,x,N_1,N_2)}$ and $D_{ij}^{(e,y,N_1,N_2)}$ matrices have dimension $(N_1 + 1) \times (N_2 + 1)$, Ψ_i is polynomials of order N_v in both x and y direction, Ψ_j is polynomials of order N_p in both x and y direction on RHS and $D^{(e,x,N_1,N_2)}$ (i,j) entry of matrix denotes when i^{th} node test function Ψ_i is multiplied by j^{th} node basis function Ψ_j and integrated over entire domain

$$D_{ij}^{(e,x,N_1,N_2)} = \int_{\Omega_e} \Psi_i^{N_v} (x, y) \frac{\partial \Psi_j^{N_p} (x, y)}{\partial x} d\Omega_e$$

$$D_{ij}^{(e,x,N_1,N_2)} = \int_{-1}^1 \int_{-1}^1 \Psi_i^{N_v} (\xi, \eta) \frac{\partial \Psi_j^{N_p} (\xi, \eta)}{\partial \xi} \frac{\partial \xi}{\partial x} |J(\xi, \eta)| d\xi d\eta \quad (4.50)$$

$$D_{ij}^{(e,x,N_1,N_2)} = \sum_{s=0}^{Q_\eta} \sum_{r=0}^{Q_\xi} w_r w_s \Psi_i^{N_v} (\xi_r, \eta_s) \frac{\partial \Psi_j^{N_p}}{\partial \xi} \bigg|_{(\xi_r, \eta_s)} \left(\frac{2}{\Delta x^{(e)}} \right) \left(\frac{\Delta x^{(e)} \Delta y^{(e)}}{4} \right)$$

Similarly, $D_{ij}^{(e,y,N_1,N_2)}$ matrix is calculated as follows:

$$D_{ij}^{(e,y,N_1,N_2)} = \sum_{s=0}^{Q_\eta} \sum_{r=0}^{Q_\xi} w_r w_s \Psi_i^{N_v} (\xi_r, \eta_s) \frac{\partial \Psi_j^{N_p}}{\partial \eta} \bigg|_{(\xi_r, \eta_s)} \left(\frac{2}{\Delta y^{(e)}} \right) \left(\frac{\Delta x^{(e)} \Delta y^{(e)}}{4} \right) \quad (4.51)$$

It results in a differential matrix of size 9×4 , as shown below, whose entries are calculated by numerical integration using Lobatto polynomial roots as integration points (Q_ξ & Q_η) similar to 1D case of equation (3.38), but using equation (4.28) as the basis function:

$$D_{ij}^{(e,x,N_2,N_1)} = \frac{\Delta y^{(e)}}{36} \begin{bmatrix} -1 & 1 & 0 & 0 \\ -4 & 4 & 0 & 0 \\ -2 & 1 & 0 & 0 \\ -2 & 2 & -2 & 2 \\ -8 & 8 & -8 & 8 \\ -2 & 2 & -2 & 2 \\ 0 & 0 & -1 & 1 \\ 0 & 0 & -4 & 4 \\ 0 & 0 & -2 & 1 \end{bmatrix} \quad D_{ij}^{(e,y,N_2,N_1)} = \frac{\Delta x^{(e)}}{36} \begin{bmatrix} 1 & 0 & 1 & 0 \\ -2 & -2 & 2 & 2 \\ 0 & -1 & 0 & 1 \\ -4 & 0 & 4 & 0 \\ -8 & -8 & 8 & 8 \\ 0 & -4 & 0 & 4 \\ -1 & 0 & 1 & 0 \\ -2 & -2 & 2 & 2 \\ 0 & -1 & 0 & 1 \end{bmatrix} \quad (4.52)$$

Thus, the final set of 5 equations we need to solve is as follows:

$$\begin{aligned} M_{ij}^{(e,N_1,N_1)}(u_j^*) - \frac{\Delta t}{2Re} [L_{ij}^{(e,N_1,N_1)}] u_j^* \\ = \left[M_{ij}^{(e,N_1,N_1)} - \frac{\Delta t}{2Re} [L_{ij}^{(e,N_1,N_1)}] - \frac{3\Delta t}{2} D_{ij}^{(e,N_1,N_1)(n)} \right] u_j^n \\ + \left[\frac{\Delta t}{2} D_{ij}^{(e,N_1,N_1)(n-1)} \right] u_j^{n-1} \end{aligned} \quad (4.53)$$

$$\begin{aligned} M_{ij}^{(e,N_1,N_1)}(v_j^*) - \frac{\Delta t}{2Re} [L_{ij}^{(e,N_1,N_1)}] v_j^* \\ = \left[M_{ij}^{(e,N_1,N_1)} - \frac{\Delta t}{2Re} [L_{ij}^{(e,N_1,N_1)}] - \frac{3\Delta t}{2} D_{ij}^{(e,N_1,N_1)(n)} \right] v_j^n \\ + \left[\frac{\Delta t}{2} D_{ij}^{(e,N_1,N_1)(n-1)} \right] v_j^{n-1} \end{aligned} \quad (4.54)$$

$$-L_{ij}^{(e,N_2,N_2)}(p_j^{n+1}) = \frac{1}{\Delta t} [D_{ij}^{(e,x,N_2,N_1)}(u_j^*) + D_{ij}^{(e,y,N_2,N_1)}(v_j^*)] \quad (4.55)$$

$$M_{ij}^{(e,N_1,N_1)}(u_j^{n+1}) = M_{ij}^{(e,N_1,N_1)}(u_j^*) - \Delta t D_{ij}^{(e,x,N_1,N_2)}(p_j^{n+1}) \quad (4.56)$$

$$M_{ij}^{(e,N_1,N_1)}(v_j^{n+1}) = M_{ij}^{(e,N_1,N_1)}(v_j^*) - \Delta t D_{ij}^{(e,y,N_1,N_2)}(p_j^{n+1}) \quad (4.57)$$

Assembling the above matrices globally, we get following set of equations:

$$\begin{aligned} \mathbf{M}_{IJ}^{(N_1,N_1)}(\mathbf{u}_J^*) - \frac{\Delta t}{2Re} [\mathbf{L}_{IJ}^{(N_1,N_1)}] \mathbf{u}_J^* \\ = \left[\mathbf{M}_{IJ}^{(N_1,N_1)(e)} - \frac{\Delta t}{2Re} [\mathbf{L}_{IJ}^{(N_1,N_1)}] - \frac{3\Delta t}{2} \mathbf{D}^{(N_1,N_1)(n)}_{IJ} \right] \mathbf{u}_J^n \\ + \left[\frac{\Delta t}{2} \mathbf{D}^{(N_1,N_1)(n-1)}_{IJ} \right] \mathbf{u}_J^{n-1} \end{aligned} \quad (4.58)$$

$$\begin{aligned}
\mathbf{M}_{IJ}^{(N_1, N_1)}(\mathbf{v}_j^*) - \frac{\Delta t}{2Re} [\mathbf{L}_{IJ}^{(N_1, N_1)}] \mathbf{v}_j^* \\
= \left[\mathbf{M}_{IJ}^{(N_1, N_1)(e)} - \frac{\Delta t}{2Re} [\mathbf{L}_{IJ}^{(N_1, N_1)}] - \frac{3\Delta t}{2} (N_1, N_1) \right] \mathbf{v}_j^n \\
+ \left[\frac{\Delta t}{2} \mathbf{D}_{IJ}^{(N_1, N_1)(n-1)} \right] \mathbf{v}_j^{n-1}
\end{aligned} \tag{4.59}$$

$$-\mathbf{L}_{IJ}^{(N_2, N_2)}(\mathbf{p}_j^{n+1}) = \frac{1}{\Delta t} \left[\mathbf{D}_{IJ}^{(x, N_2, N_1)}(\mathbf{u}_j^*) + \mathbf{D}_{IJ}^{(y, N_2, N_1)}(\mathbf{v}_j^*) \right] \tag{4.60}$$

$$\mathbf{M}_{IJ}^{(N_1, N_1)}(\mathbf{u}_j^{n+1}) = \mathbf{M}_{IJ}^{(N_1, N_1)}(\mathbf{u}_j^*) - \Delta t \mathbf{D}_{IJ}^{(x, N_1, N_2)}(\mathbf{p}_j^{n+1}) \tag{4.61}$$

$$\mathbf{M}_{IJ}^{(N_1, N_1)}(\mathbf{v}_j^{n+1}) = \mathbf{M}_{IJ}^{(N_1, N_1)}(\mathbf{v}_j^*) - \Delta t \mathbf{D}_{IJ}^{(y, N_1, N_2)}(\mathbf{p}_j^{n+1}) \tag{4.62}$$

Now above 5 set of equations will form system of equations, which need to be solved by suitable technique. The Mass matrix using inexact integration generates diagonal matrix. Last 2 equations mentioned above have only Mass matrix on LHS. We make use of inexact differentiation for the last 2 set of equations, since it develops a diagonal matrix A for $Ax = b$, which makes calculation of u^{n+1} and v^{n+1} easy by direct division and we don't need to solve last 2 set of equations by any specific technique.

For 2D system, the diagonal mass matrix for velocity calculated using inexact differentiation, is as follows:

$$\mathbf{M}_{IJ}^{(e)} = \frac{\Delta x^{(e)}}{2} \begin{bmatrix} 1 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & 1 \end{bmatrix}_{9 \times 9}$$

All the above square matrices represented above will be used for Pressure space as they are constructed using 1st order interpolation. Velocity using 2nd-order expansion function will create matrices of size (9×9) . Velocity matrices can be constructed in a similar manner. We will now examine the Taylor-Hood element now which uses this unequal order interpolation as discussed above.

4.5 Taylor Hood Element

Taylor-Hood element uses 2nd order velocity interpolation and 1st order pressure interpolation as discussed above. This unequal order interpolation is frequently used to guarantee accuracy and stability while discretizing pressure and velocity fields in incompressible flow. This element satisfies the inf-sup (Ladyzhenskaya-Babuška-Brezzi, or LBB) stability condition. This stability ensures that the discretization of the velocity and

pressure fields does not lead to spurious pressure modes (in regions with sharp pressure gradients) or oscillations and avoids numerical instabilities.

The above matrices of size (4×4) generated represent pressure space, and similar (9×9) Matrices can be generated to represent velocity space. The rectangular difference matrices shown in the earlier discussion help to couple different pressure and velocity nodes.

The Nodal representation of the Taylor-hood element is shown below:

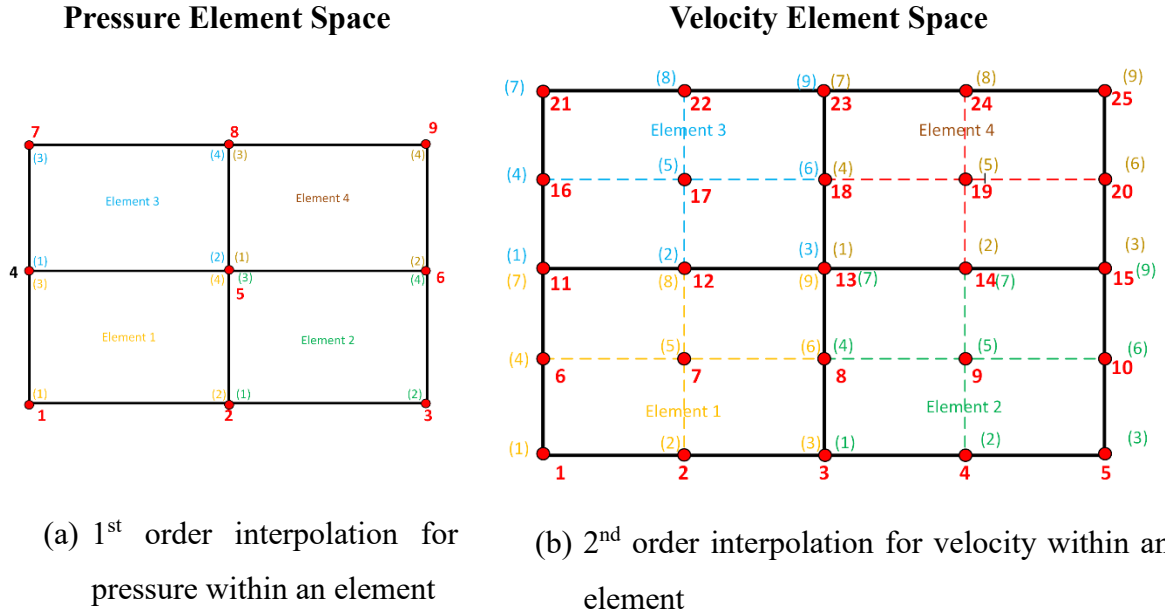


Figure 4.2 Taylor-Hood Element

4.6 Element by Element (EBE) Conjugate Gradient Method

As discussed above, all the element-level matrices need to be globally assembled to form a system of equations. If an element has N nodes, then for any node, its solution depends on a maximum $4 \times N$ number of surrounding nodes. (i.e. maximum nonzero entries in a row). This is due to any node on the boundary can be part of 4 elements if it is at a corner. So, in a global system of equations, almost all of the matrix is sparse with very less nonzero entries. For a mainly structured grid, where each element matrix is calculated from general element level matrices with a multiple of Jacobian used for transformation. For optimum memory utilisation, we can just store the general element level mass, Laplacian and Differential matrix, and store the Jacobian, its derivatives and element sizes for each element. Instead of assembling the global stiffness matrices explicitly, the matrices are stored in terms of their elemental contributions. Conjugate Gradient (CG) method can be efficiently utilised to deal with such a

approach, as for every step in CG, we need the column vectors (like Ax , residual, direction vector, etc.) and not the A matrix directly.

All other operations in a standard conjugate gradient algorithm involve vector-vector operations, except for the crucial matrix-vector product. The matrix-vector product Ax takes a vector x as input and produces another vector as output. In the Element-by-Element (EBE) approach, this product is computed by summing the contributions from each individual element:

$$A_{IJ}x_j = \sum_{\text{elements}} A_{ij}^{(e)} x_j$$

where A_{IJ} denotes the global matrix, and $A_{ij}^{(e)}$ is the element matrix. This is illustrated using an example of a three-element 1D domain below (Figure 3.7). The global matrix-vector product is split into three separate element matrix-vector products.



Figure 4.3 1D Element-by-Element Assembly

A general matrix system for above 1D case can be represented as follows:

$$A_{IJ}x_j = \begin{bmatrix} a_{11} & a_{12} & 0 & 0 \\ a_{21} & a_{22} & a_{23} & 0 \\ 0 & a_{32} & a_{33} & a_{34} \\ 0 & 0 & a_{43} & a_{44} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} Ax_1 \\ Ax_2 \\ Ax_3 \\ Ax_4 \end{bmatrix}$$

In above equation for calculating Ax_1 , instead of calculating it from A matrix, we will make use of the general element level matrix and connectivity matrix as shown below

$$C = \begin{bmatrix} 1 & 2 \\ 2 & 3 \\ 3 & 4 \end{bmatrix} \quad A_{ij}^{(e)} = \begin{bmatrix} a_{11}^e & a_{11}^e \\ a_{11}^e & a_{11}^e \end{bmatrix}$$

$$A_{IJ}x_j = A_{ij}^{(1)}x_j + A_{ij}^{(2)}x_j + A_{ij}^{(3)}x_j$$

Here, the superscript depicts the element number.

Here, the yellow colour denotes contribution from element 1. Green colour denotes contribution from element 2, and the magenta colour denotes contribution from element 3. The grey colour entries have contribution from both adjacent elements.

Table 4.3: Element-by-Element Assembly

$A_{ij}^{(1)} x_j = \begin{bmatrix} a_{11}^1 & a_{12}^1 \\ a_{21}^1 & a_{22}^1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$	$A_{IJ} x_J \rightarrow \begin{bmatrix} Ax_1^1 \\ Ax_2^1 \\ 0 \\ 0 \end{bmatrix}$
$A_{ij}^{(2)} x_j = \begin{bmatrix} a_{22}^2 & a_{23}^2 \\ a_{32}^2 & a_{33}^2 \end{bmatrix} \begin{bmatrix} x_2 \\ x_3 \end{bmatrix}$	$A_{IJ} x_J \rightarrow \begin{bmatrix} 0 \\ Ax_2^2 \\ Ax_3^2 \\ 0 \end{bmatrix}$
$A_{ij}^{(3)} x_j = \begin{bmatrix} a_{33}^3 & a_{34}^3 \\ a_{43}^3 & a_{44}^3 \end{bmatrix} \begin{bmatrix} x_3 \\ x_4 \end{bmatrix}$	$A_{IJ} x_J \rightarrow \begin{bmatrix} 0 \\ 0 \\ Ax_3^3 \\ Ax_4^3 \end{bmatrix}$
$A_{IJ} x_J = A_{ij}^{(1)} x_j + A_{ij}^{(2)} x_j + A_{ij}^{(3)} x_j = \begin{bmatrix} Ax_1^1 \\ Ax_2^1 + Ax_2^2 \\ Ax_3^2 + Ax_3^3 \\ Ax_4^3 \end{bmatrix}$	

Here, all the entries Ax_l^k are calculated using the corresponding entry from general element level matrix to locate the local node no. from global node no. using connectivity matrix and multiplying it by required multiplier.

In the conjugate gradient method, operations such as calculating residuals, direction vectors, and updating the solution vector are performed using above element-wise technique to eliminate the need for explicitly forming the global matrix.

Since, this we need to add each element contribution to $A_{IJ}X_J$, we can use parallel processing to optimize this calculation process as it is independent of each other.

This method is most useful for structured grid as the element level matrix is same for all elements. But for unstructured grid, we need to store the element matrix for each element as it is not unique due to non-orthogonal elements. Still storing individual element level matrices is more efficient than storing 1 global matrix with all zero entries. For unstructured grid, Coordinate Matric storage solver, Eigen solver, LDLT solver can give faster results and less storage requirement than EBE approach.

4.5. Unstructured Grid

For structured grid, we can define or generalise the grid details, element sizes and spacing explicitly in the solver. But for unstructured grid, defining these details is difficult, and we need to extract all element, node and boundary information from some meshing software. In current study, we are using GMSH software to mesh geometry and extract the data.

This direct data is very useful and doesn't need any additional operation while solving 1st order FEM equations, but while using SEM, where we approximate the solution inside an element using higher order basis functions with multiple nodes (more than conventional 4 nodes located at boundary), we need to generate more modes inside the element, and store all the information related to their coordinates, node numbering, element numbering, connectivity, etc in an efficient manner. The current solver is designed to accommodate all these requirements. After reading all element and its boundary nodes data from the GMSH file, this solver rearranges all the nodes in an element in a sequential manner as required for SEM. It also generates multiple interior nodes required for approximation the solution based on order of interpolating polynomial. It makes use of 1st order shape functions (bilinear transformation) for mapping these interior points from computational domain to physical domain. It also tries to generate separate set of nodes for velocity and pressure and utilise this information when we have unequal order approximation for both variable.

The shape functions used to calculate the values of interior points from corner nodes are expressed as follows:

Table 4.4: Shape functions for linear elements

$N_1 = \frac{(1 - \xi)(1 - \eta)}{4}$	$N_2 = \frac{(1 + \xi)(1 - \eta)}{4}$	$N_3 = \frac{(1 + \xi)(1 + \eta)}{4}$	$N_4 = \frac{(1 - \xi)(1 + \eta)}{4}$	(4.63)
$x = N_1x_1 + N_2x_2 + N_3x_3 + N_4x_4$				
$y = N_1y_1 + N_2y_2 + N_3y_3 + N_4y_4$				

While generating all nodes, the nodes on boundary are shared by two elements. We can avoid repetitive counting of these common shared nodes using some key-value based data structure to track the existence of a node from its coordinates. Some examples of these are Maps and Hash tables.

As discussed earlier, the generation of element-level matrices for unstructured grid requires information of the jacobian and its components, and these parameters change continuously at each node with changing values of ξ and η . For an unstructured grid, both x and y are functions of both ξ and η .

Using the chain rule of differentiation, we can express various terms and its derivatives as shown in table below:

Table 4.5: Bilinear Transformation

$x = x(\xi, \eta)$	$y = y(\xi, \eta)$	(4.64)
$dx = \frac{\partial x}{\partial \xi} d\xi + \frac{\partial x}{\partial \eta} d\eta$	$dy = \frac{\partial y}{\partial \xi} d\xi + \frac{\partial y}{\partial \eta} d\eta$	(4.65)
$dx = x_{,\xi} d\xi + x_{,\eta} d\eta$	$dy = y_{,\xi} d\xi + y_{,\eta} d\eta$	
$\begin{pmatrix} dx \\ dy \end{pmatrix} = \begin{pmatrix} \frac{\partial x}{\partial \xi} & \frac{\partial x}{\partial \eta} \\ \frac{\partial y}{\partial \xi} & \frac{\partial y}{\partial \eta} \end{pmatrix} \begin{pmatrix} d\xi \\ d\eta \end{pmatrix} = J \begin{pmatrix} d\xi \\ d\eta \end{pmatrix}$		(4.66)
$\det(J) = Jac = J^{(e)} = \frac{\partial x}{\partial \xi} \frac{\partial y}{\partial \eta} - \frac{\partial y}{\partial \xi} \frac{\partial x}{\partial \eta}$		(4.67)
$\xi = \xi(x, y)$	$\eta = \eta(x, y)$	(4.68)
$d\xi = \frac{\partial \xi}{\partial x} dx + \frac{\partial \xi}{\partial y} dy$	$d\eta = \frac{\partial \eta}{\partial x} dx + \frac{\partial \eta}{\partial y} dy$	(4.69)
$d\xi = \xi_{,x} dx + \xi_{,y} dy$	$d\eta = \eta_{,x} dx + \eta_{,y} dy$	
$\begin{pmatrix} d\xi \\ d\eta \end{pmatrix} = J^{-1} \begin{pmatrix} dx \\ dy \end{pmatrix}$		(4.70)
$J^{-1} = \frac{1}{ J^{(e)} } \begin{pmatrix} \frac{\partial y}{\partial \eta} & -\frac{\partial x}{\partial \eta} \\ -\frac{\partial y}{\partial \xi} & \frac{\partial x}{\partial \xi} \end{pmatrix}$		(4.71)
$\frac{\partial}{\partial x} = \frac{\partial}{\partial \xi} \frac{\partial \xi}{\partial x} + \frac{\partial}{\partial \eta} \frac{\partial \eta}{\partial x}$	$\frac{\partial}{\partial y} = \frac{\partial}{\partial \xi} \frac{\partial \xi}{\partial y} + \frac{\partial}{\partial \eta} \frac{\partial \eta}{\partial y}$	(4.72)
$\frac{\partial \xi}{\partial x} = \frac{1}{ J^{(e)} } \frac{\partial y}{\partial \eta}$	$\frac{\partial \xi}{\partial y} = -\frac{1}{ J^{(e)} } \frac{\partial x}{\partial \eta}$	(4.73)
$\frac{\partial \eta}{\partial x} = -\frac{1}{ J^{(e)} } \frac{\partial y}{\partial \xi}$	$\frac{\partial \eta}{\partial y} = \frac{1}{ J^{(e)} } \frac{\partial x}{\partial \xi}$	

For simplification, we will consider different notation for the basis functions in x or ξ and y or η direction as follows:

Table 4.6: Notations for transformed test/basis functions

$\Psi_j'(x, y) \Rightarrow \Psi_j(\xi, \eta) = \psi_j(\xi)\phi_j(\eta) \rightarrow$ Orthogonal Coordinate System	
$\Psi_j(\xi, \eta) = \psi_j(\xi)\phi_j(\eta)$	
$\Psi_{j,\xi}(\xi, \eta) = \psi_{j,\xi}(\xi)\phi_j(\eta)$	$\Psi_{j,\eta}(\xi, \eta) = \psi_j(\xi)\phi_{j,\eta}(\eta)$
<u>Test/Trial Function</u>	<u>Basis Function</u>
$\Psi_i(\xi, \eta) = \psi_i(\xi)\phi_i(\eta)$	$\Psi_j(\xi, \eta) = \psi_j(\xi)\phi_j(\eta)$

Considering equal order approximation for velocity and pressure and assuming $\psi_i^N = \psi_i^N(\xi)$

& $\phi_i^N = \phi_i^N(\eta)$, the 5 matrices for unstructured grid can be represented as follows:

$$M_{ij}^{(e,N)} = \int_{-1}^1 \int_{-1}^1 [\psi_i^N(\xi)\phi_i^N(\eta)\psi_j^N(\xi)\phi_j^N(\eta)] |J^{(e)}| d\xi d\eta \quad (4.74)$$

$$L_{ij}^{(e,N)(x)} = \int_{-1}^1 \int_{-1}^1 [(\psi_{i,\xi}^N \phi_i^N \psi_{j,\xi}^N \phi_j^N) y_{,\eta} y_{,\eta} - (\psi_{i,\xi}^N \phi_i^N \psi_j^N \phi_{j,\eta}^N) y_{,\eta} y_{,\xi} - (\psi_i^N \phi_{i,\eta}^N \psi_{j,\xi}^N \phi_j^N) y_{,\eta} y_{,\xi} + (\psi_i^N \phi_{i,\eta}^N \psi_j^N \phi_{j,\eta}^N) y_{,\xi} y_{,\xi}] \frac{1}{|J^{(e)}|} d\xi d\eta \quad (4.75)$$

$$L_{ij}^{(e,N)(y)} = \int_{-1}^1 \int_{-1}^1 [(\psi_{i,\xi}^N \phi_i^N \psi_{j,\xi}^N \phi_j^N) x_{,\eta} x_{,\eta} - (\psi_{i,\xi}^N \phi_i^N \psi_j^N \phi_{j,\eta}^N) x_{,\eta} x_{,\xi} - (\psi_i^N \phi_{i,\eta}^N \psi_{j,\xi}^N \phi_j^N) x_{,\eta} x_{,\xi} + (\psi_i^N \phi_{i,\eta}^N \psi_j^N \phi_{j,\eta}^N) x_{,\xi} x_{,\xi}] \frac{1}{|J^{(e)}|} d\xi d\eta \quad (4.76)$$

$$D_{ij}^{(e,N)(x)} = \int_{-1}^1 \int_{-1}^1 [(\psi_i^N \phi_i^N \psi_{j,\xi}^N \phi_j^N) y_{,\eta} - (\psi_i^N \phi_i^N \psi_j^N \phi_{j,\eta}^N) y_{,\eta}] d\xi d\eta \quad (4.77)$$

$$D_{ij}^{(e,N)(y)} = \int_{-1}^1 \int_{-1}^1 [-(\psi_i^N \phi_i^N \psi_{j,\xi}^N \phi_j^N) x_{,\eta} + (\psi_i^N \phi_i^N \psi_j^N \phi_{j,\eta}^N) x_{,\eta}] d\xi d\eta \quad (4.78)$$

For unequal order the differentiation matrix is give as:

$D_{ij}^{(e,(N_1,N_2))(x)} = \int_{-1}^1 \int_{-1}^1 [(\psi_i^{N_1} \phi_i^{N_1} \psi_{j,\xi}^{N_2} \phi_j^{N_2}) y_{,\eta} - (\psi_i^{N_1} \phi_i^{N_1} \psi_j^{N_2} \phi_{j,\eta}^{N_2}) y_{,\eta}] d\xi d\eta$	(4.79)
$D_{ij}^{(e,(N_1,N_2))(y)} = \int_{-1}^1 \int_{-1}^1 [-(\psi_i^{N_1} \phi_i^{N_1} \psi_{j,\xi}^{N_2} \phi_j^{N_2}) x_{,\eta} + (\psi_i^{N_1} \phi_i^{N_1} \psi_j^{N_2} \phi_{j,\eta}^{N_2}) x_{,\eta}] d\xi d\eta$	(4.80)

4.6 Linear Mesh / Straight Edge Mesh Elements

Standard FEM uses equidistant points, but SEM tries to fit non-equidistant polynomial roots in the domain. For transforming physical coordinates at nodal DOF from physical to computational domain, we make use of shape functions. These shape functions should be smooth (at least C^1) on the element interior Γ^e and complete. The shape function for 1st order straight lines in 2D is given by:

$$N_a(\xi, \eta) = \text{Lag}^1(\xi) \text{Lag}^1(\eta) \quad (4.81)$$

The transformation for 2D domain is shown below. The computational domain in 1D is from -1 to 1. For 2D, it is a square with coordinates: (-1, -1), (1, -1), (1, 1), and (-1, 1). The transformation from computational to physical domain to get the SEM mesh physical coordinates is as follows:

$$\begin{aligned} x(\xi, \eta) &= \sum_{a=1}^4 N_a(\xi, \eta) x_a^e \\ y(\xi, \eta) &= \sum_{a=1}^4 N_a(\xi, \eta) y_a^e \end{aligned} \quad (4.82)$$

The shape function is bilinear expression in terms of ξ and η , and can be expressed as:

Table 4.7: Shape functions for Linear/Straight edge 2D elements

$N_1 = \frac{(1 - \xi)(1 - \eta)}{4}$	$N_2 = \frac{(1 + \xi)(1 - \eta)}{4}$
$N_3 = \frac{(1 + \xi)(1 + \eta)}{4}$	$N_4 = \frac{(1 - \xi)(1 + \eta)}{4}$

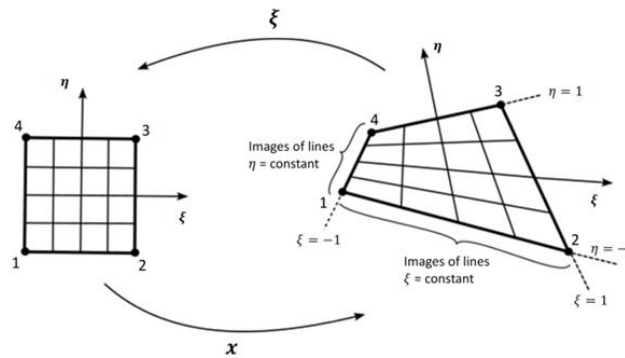


Figure 4.4: Transformation of straight edge quadrilaterals (constant ξ and η are mapped into straight lines)

The Linear straight line edge elements are not able to track the curved edges and surfaces. So, we make use of the Curved Mesh Elements. The details are discussed in further section.

Code Restructuring and Solver Analysis

This section marks the beginning of the work carried out during Phase 2 of the MTP. It discusses the outcome of code parallelization and the need for restructuring the code. It also highlights the shortcomings of the Element-by-Element Conjugate Gradient approach and the advantages of using the global assembly approach. Linear solvers like PETSC and in-house Conjugate Gradient (CG) solver are examined and compared.

5.1 CPP Code Parallelization

Parallelization of the code can reduce the computational time. Most of the computational time of the solver is consumed by the time iteration loop. For each timestep, we need to solve 3 systems of equations ($Ax=b$), 1 each for u^* , v^* and pressure. In the EBE CG approach, parallelization can be used for assembling each element contribution as well as parallelizing all for loops. Two parallelization approaches have been discussed further.

1st Parallelization: OpenMP using multiple threads of cores was used for parallelizing EBE Jacobi Preconditioned_Conjugate_Gradient (PCG) Solver. Assembling element-level contribution from each element in matrix-vector multiplication to global vectors along parallelization of multiple “for loops” in PCG Solver being called multiple times in time iteration loop was primary target. The gain in reducing computational time for 600 timesteps from 12 minutes to approximately 4 minutes for this operation is shown table 5.1 below.

2nd Parallelization: The calculation of u^* and v^* can also be parallelized because they are independent of each other. Same is the case for calculation of u and v . In each timestep the 5 serial calculations can be reduced to 3 serial calculations – (u^* , v^*); (p); (u , v).

We faced difficulty in implementing 2nd parallelization due to unstructured Code. Also, as global matrix is not created, we cannot make use of other preconditioners like ILU, FEM, GAMG, etc. except Jacobi preconditioner. Geometric Multigrid can be used only for few specific choices of mesh. So, the entire code was rewritten and restructured in Fortran along with Global assembly to create sparse matrices and use linear solvers with various preconditioners on global Matrix-Vector system.

Table 5.1: Parallelisation for EBE PCG Solver

	#pragma omp for (Threads = 4)	#pragma omp parallel for collapse (2) (Threads = 4)	#pragma ivdep (Threads = 8)	#pragma unroll(10) (Threads = 8)	#pragma simd (Threads = 8)
Time	11m46.222s	6m50.206s	5m18.873s	5m10.537s	3m39.973s
Speed up	1.09x	1.88x	2.42x	2.48x	3.51x

5.2 Code Restructuring and Use of Other Solvers

We have left behind the Element-by-element level approach due to the absence of preconditioners & other direct/iterative solvers, and repetitive assembly operations. The gain in reducing space requirement is negligible compared to the increase in computational time for each iteration of the time step.

The CPP code was written in 1 single file, which exceeded 5000+ lines. Therefore, we switched to Fortran using make makefile concept to restructure the code properly. Also, CPP code made use of various data structures like HashMap and Vectors, which are not available in Fortran. C++ Code used the Eigen library's iterative solvers for solving globally assembled sparse matrix systems of equations, $Ax = b$. The eigen library is not supported in Fortran. So, we have made use of the PETSC Library for iterative solvers. The Further section gives details of the PETSC Solver. We can implement parallelization in the PETSC Solver, but for that, the whole code needs to be written using PETSC variables, arrays, and matrices. Also, it will be strictly restricted to systems having the PETSC solver and cannot run on all systems universally. So, we are currently using the PETSC Solver only for solving the system of equations $Ax = b$ sequentially.

As an alternative to PETSC and to decrease the dependence on the PETSC library, we have tried to develop a solver using the Conjugate Gradient method using sparse storage matrices. The CG_SOLVER details are discussed in a further section. As this Solver is completely controlled by the user, it can be parallelized as per the user's requirement. 2nd parallelization can be implemented using this approach.

5.3 PETSC Solver

‘Portable, Extensible Toolkit for Scientific Computation’ abbreviated as **PETSC** is an open-source computing library designed for solving scientific and engineering problems, particularly those involving partial differential equations in a scalable and parallel manner. It is highly extensible, built with MPI, and supports various numerical methods, including linear and nonlinear solvers, time integrators, and optimization tools. PETSC provides interfaces for C/C++, Fortran, and Python. We mainly focus on **KSP (Krylov Space Methods)** objects used as iterative linear solvers (Conjugate Gradient and Preconditioners).

KSPCG is used for solving the linear system of equation $\mathbf{Ax}=\mathbf{b}$ in Navier-stokes equation (2 Intermediate velocity equations, & 1 Pressure Poisson equation). It stores the A matrix in global sense in CSR (Compressed Sparse Row) format for efficient memory utilisation (All element level matrices are used for assembly). Various preconditioners are available to speed up the CG method like JACOBI, ILU, GAMG (General Algebraic Multigrid), etc. We can make use of various other preconditioners, decomposition methods, direct solvers, etc. from PETSC library. A short comparison of time and iteration count by various preconditioners is done ahead in Results section.

The Workflow of the PETSC Solver is shown in the diagram below

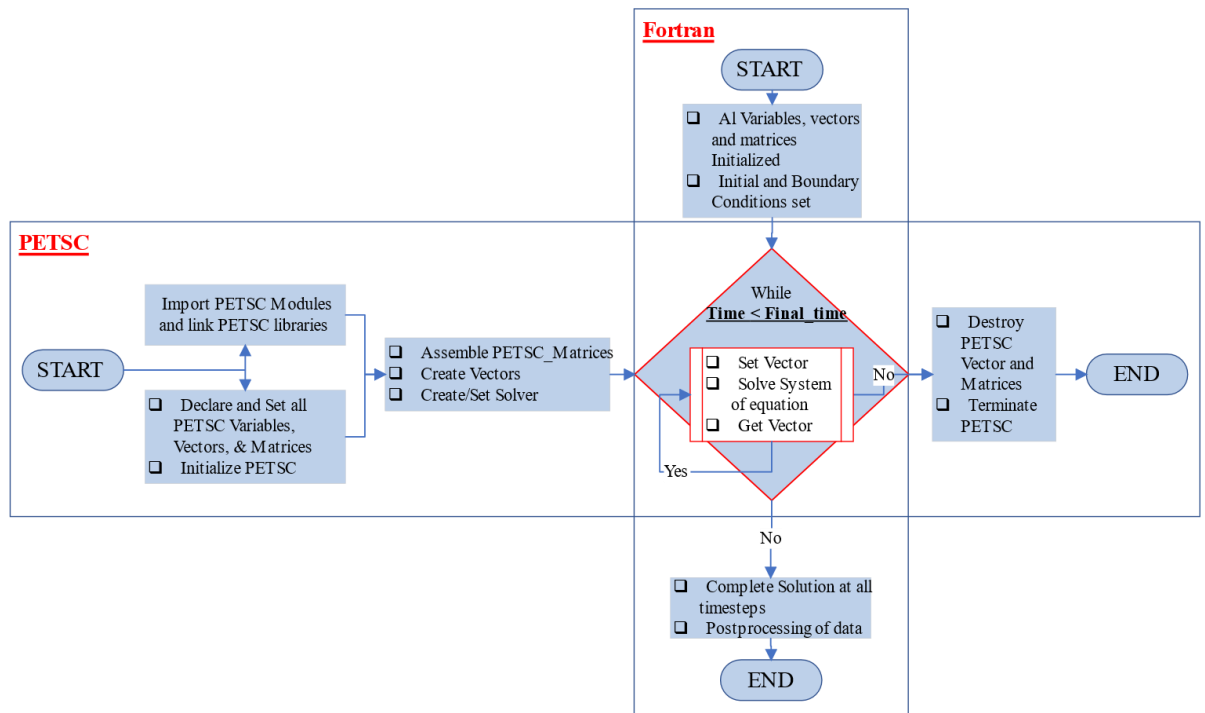


Figure 5.1: Workflow of PETSC Solver

Use of PETSC puts the following restrictions:

1) The RHS (b vector) in $Ax=b$ is calculated using Fortran vectors and matrices. Therefore, the initial and final vectors need to be copied before and after each solve (SET_VECTOR and GET_VECTOR) between Fortran vectors and PETSC vectors. This adds up to the overhead cost. (Alternative: The whole code (all arrays and matrices) can be rewritten in PETSC layout using PETSC Variables and functions)

2) The current process is running sequentially on a single core. We can distribute some of the operations on various cores as discussed above, but the non-PETSC part of the code needs to be modified to suit the MPI requirement and execute only once. It will parallelize the iterative solver on its own, and will restrict us to having control of parallelizing the iteration sub-steps and the choice to have the parallelization & preconditioner of our own.

3) The code depends on the PETSC libraries and modules for execution and cannot be executed on any system where the PETSC libraries are not installed.

5.4 CG Solver

To overcome the above solver issue, we are trying to develop a Conjugate Gradient (CG) Solver (CG_SOLVER) as an alternative solver. It helps to decrease the dependency on the PETSC Solver. It makes use of global matrices in CSR format with BLAS functionality from **Intel one API Library** for multiplying a CSR matrix with a vector. It uses a general CG structure with a residual check ($residual = b - Ax$) and tolerance limit of 10^{-9} as the stopping criteria.

The CSR matrices are created from the element matrices. The element-level Mass, Laplacian, and Differential matrices are used to create element-level LHS matrices. These matrices are then individually updated for boundary conditions to make the required row and column entries zero corresponding to the Dirichlet boundary nodes, like eliminating those rows and columns (explained in section 3.3.5). Then these modified element-level LHS matrices are used to assemble global LHS matrices in Compressed Matrix Format (MATLAB sparse storage format)[35]. These Sparse global matrices in compressed format are then converted to CSR format easily. A basic example of the Compressed Matrix and CSR Matrix is shown below. The assembly of element-level matrices to form a Compressed Matrix and a CSR matrix is further explained.

The Flowchart for the Solver is as shown below:

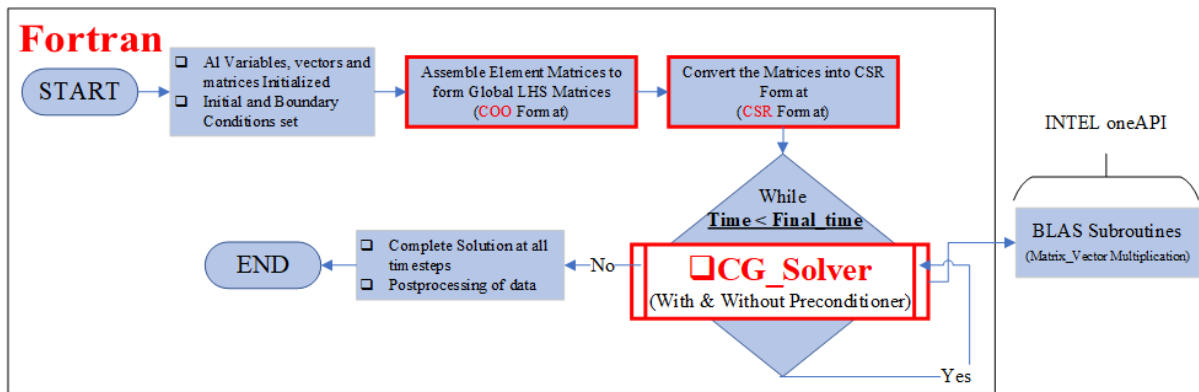


Figure 5.2: Workflow of in-house CG Solver

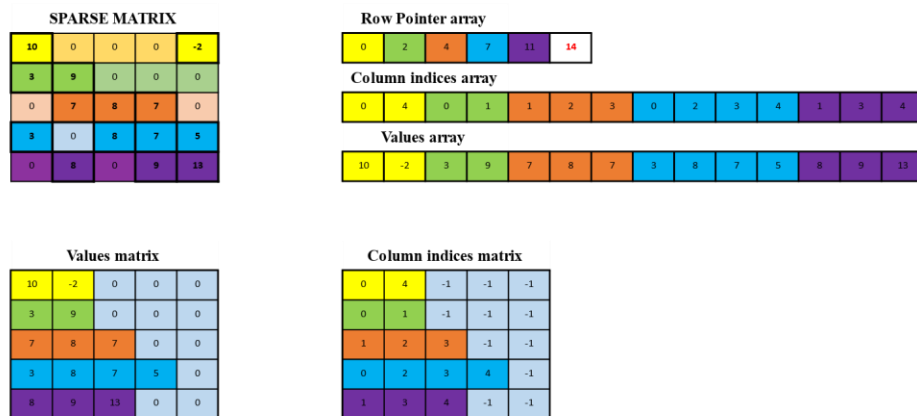


Figure 5.3: Structure of transformation of Matrix to Compressed Matrix & CSR Matrix

BLAS (Basic Linear Algebra Subprograms) routines are used to multiply CSR matrix and vector in CG_sparse_solver subroutine. Relative residual norm is used as stopping criteria for each iteration.

$$Relative\ Residual\ Norm = \frac{\|res\|}{\|b\|}, \text{ where } \|\cdot\| \text{ depicts L2 norm and } res = b - Ax \quad (5.1)$$

Preconditioner: The Conjugate Gradient method reduces the residual slowly without any preconditioner. Preconditioners accelerate the residual reduction in less iterations. We have currently applied Jacobi Preconditioner to reduce iterations. It makes use of the diagonal matrix of the global matrix as the preconditioner.

Further, we plan to implement other preconditioners like ILU, algebraic multigrid etc. Since, we have the matrix now in CSR format, we can make use of direct solvers like Cholesky

solvers by extracting the L & L^T matrices using LAPACK subroutine from Intel one API library. We can also apply lower order SEM/FEM matrices as preconditioners. Geometric Multigrid can also be utilized in overset method for the background uniform structured mesh.

An example of Assembly of 1D element level Mass matrices to form Compressed Matrix and CSR matrices (with stages) is shown in figure 5.4 below:

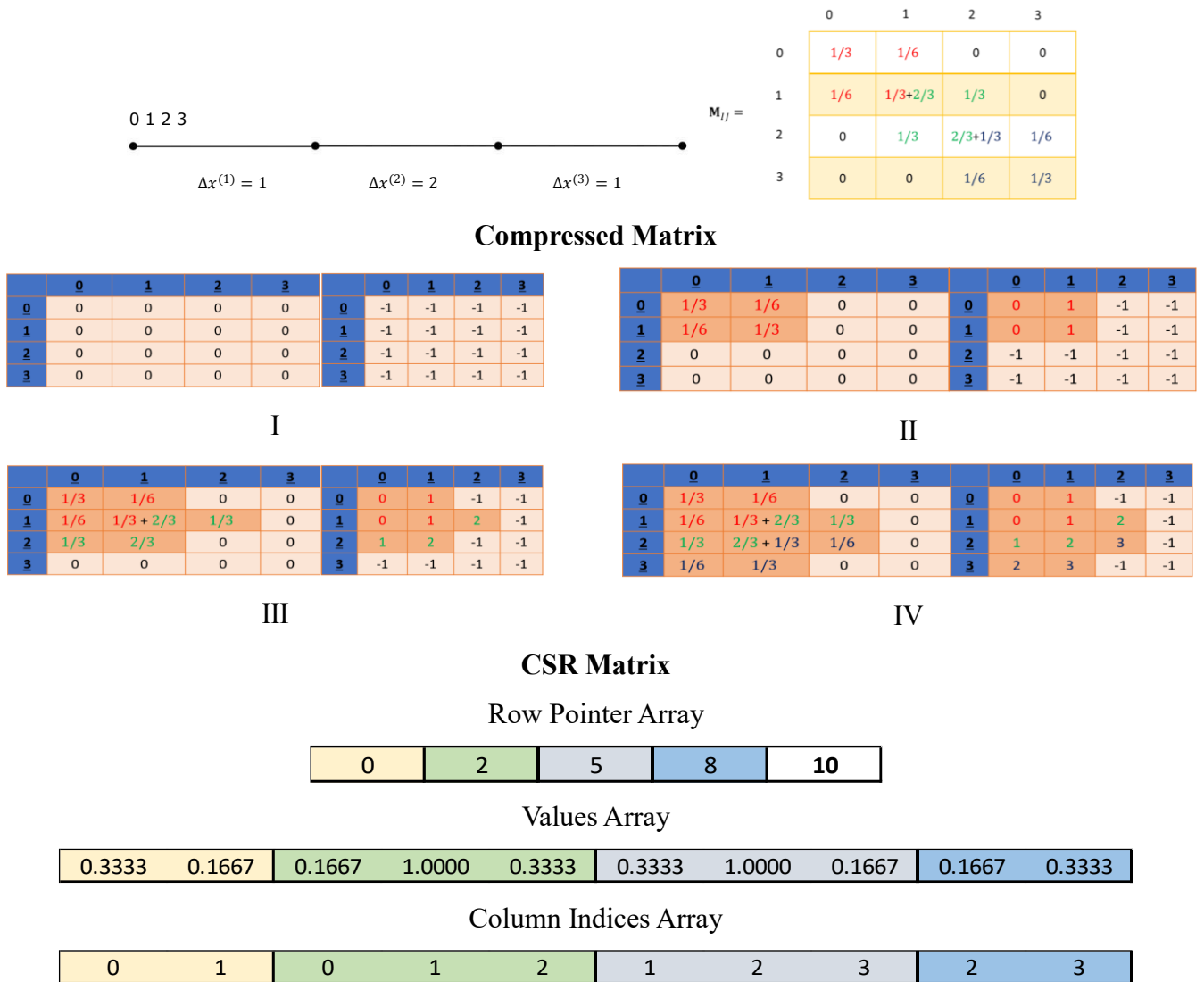


Figure 5.4: Global Assembly to form CSR Matrix

The Comparison between PETSC and CG_SOLVER (using FEM straight edge mesh) in terms of number of iterations, computational time is shown below. Also, the figures show the comparison of total lift and total drag plot for both solvers using Jacobi preconditioner.

Table 5.2: Results for CG with Jacobi Preconditioner

Method	Average Iterations			Computational Time
	u^*	v^*	Pressure	
PETSC CG (Jac)	4	5	288	3678.5464
Solver CG (Jac)	2	3	289	2185.8981

Table 5.3: Results for CG with No Preconditioner

Method	Average Iterations			Computational Time
	u^*	v^*	Pressure	
PETSC CG (None)	181	344	312	19327.276
Solver CG (None)	193	358	319	10930.507

Numerical Simulation Parameters

No. of timesteps	Tolerance	Order	Nodes	Quad Elements
50000	1.00E-09	2,1	21826, 5513	5400

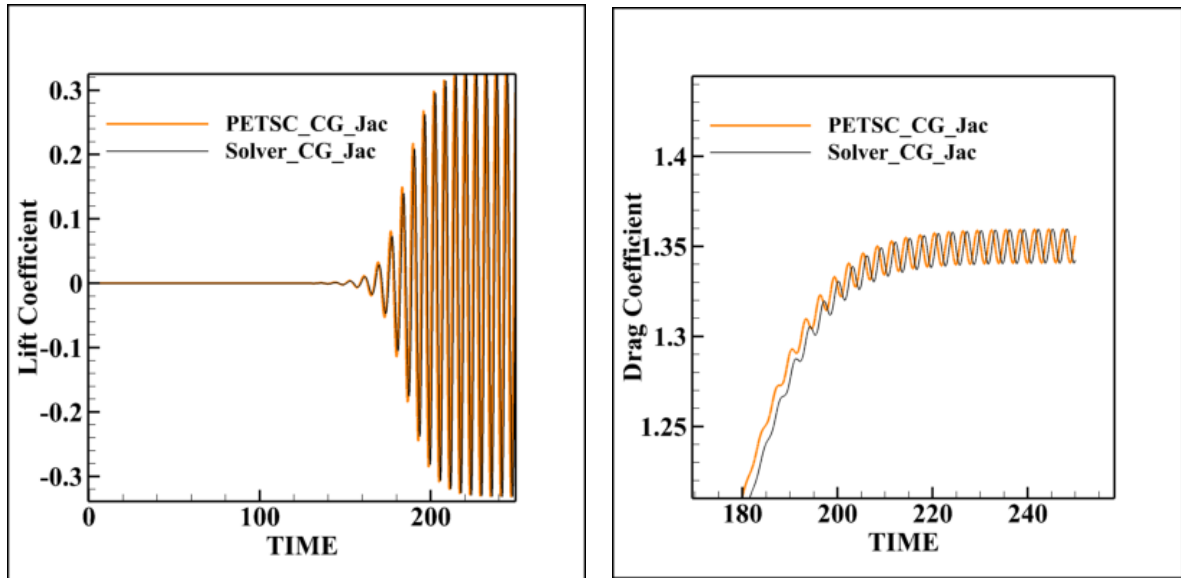


Figure 5.5: Comparison of lift-drag coefficient of in-house CG Solver with PETSC solver

Hydrodynamic Forces and Mesh Generation

6.1 Hydrodynamic Forces

6.1.1 Lift and Drag Forces

The alternating von Kármán vortices induce unsteady forces on the cylinder. The force acting in the direction of the flow is known as the drag force, while the force in the transverse (or cross-flow) direction is called the lift force. Both lift and drag forces arise from two components: pressure and viscous effects. The pressure distribution contributes to the **form drag** and **form lift**, while variations in velocity normal to the surface cause shear stresses, which result in **skin friction drag** and **skin friction lift**. Summation of skin friction and form lift and drag results into total lift and drag.



Figure 6.1: Lift and Drag forces on bluff body

The equations for calculating Lift and drag forces are presented in table 6.1 below.

Table 6.1: Equations of Lift and Drag Forces

	INTEGRATION	NUMERICAL INTEGRATION	
Lift Force	$= \int_0^{2\pi} -p \sin(\theta) d\theta + \int_0^{2\pi} \tau_w \cos(\theta) d\theta$	$= \sum_{i=0}^N w_i \{-p(\theta_i) \sin(\theta_i) + \tau_w(\theta_i) \cos(\theta_i)\}$	(6.1)
Drag Force	$= \int_0^{2\pi} -p \cos(\theta) d\theta - \int_0^{2\pi} \tau_w \sin(\theta) d\theta$	$= \sum_{i=0}^N w_i \{-p(\theta_i) \cos(\theta_i) - \tau_w(\theta_i) \sin(\theta_i)\}$	(6.2)

- **Form Lift/Drag**

Calculating the lift and drag resulting from pressure forces is relatively straightforward. The pressure at the boundary of the cylinder is known at each discretized node. Since pressure forces act normal to the surface, their components in the flow (drag) and transverse (lift) directions can be obtained by projecting the normal on each axis accordingly, as illustrated below in figure 6.2. The integration over the cylinder surface is performed numerically using Gauss–Lobatto quadrature weights w_i for each line element.

- **Skin Friction Lift/Drag**

Calculating the contribution from viscous forces is slightly tedious, as shear stress values at the cylinder boundary nodes are not directly available. To obtain them, we use differential matrices within each element to compute the velocity gradients: $\frac{\partial u}{\partial x}$, $\frac{\partial u}{\partial y}$, $\frac{\partial v}{\partial x}$, $\frac{\partial v}{\partial y}$. These gradients are then used to evaluate the shear stress components in the x–y plane (σ_x , σ_y , and τ_{xy}). Finally, using the Mohr’s circle formulation (plane stress transformation), we determine the shear stress acting tangentially along the cylinder surface at each point.

$$\tau_{x'y'} = -\frac{1}{2}(\sigma_x - \sigma_y)\sin(2\theta) + \tau_{xy}\cos(2\theta) \quad (6.3)$$

where $\sigma_x = E \frac{\partial u}{\partial x}$, $\sigma_y = E \frac{\partial v}{\partial y}$, $\tau_{xy} = E \left(\frac{\partial u}{\partial y} + \frac{\partial v}{\partial x} \right)$

After reducing equation into non-dimensional, we get the shear stress at cylinder surface node along the tangent as follows:

$$\tau_{nt} = \frac{1}{Re} \left[\left(\frac{\partial v}{\partial y} - \frac{\partial u}{\partial x} \right) \sin(2\theta) + \left(\frac{\partial u}{\partial y} + \frac{\partial v}{\partial x} \right) \cos(2\theta) \right] \quad (6.4)$$

This τ_{nt} will be the τ_w at each cylinder surface node.

The Lift and Drag force are made non-dimensional by converting into lift coefficient and drag coefficient as follows:

$$\text{Lift Coefficient:} \quad C_L = \frac{\text{Lift}}{\frac{1}{2}\rho U_\infty^2 D} \quad (6.5)$$

$$\text{Drag Coefficient:} \quad C_D = \frac{\text{Drag}}{\frac{1}{2}\rho U_\infty^2 D} \quad (6.6)$$

The Strouhal number characterizing oscillatory flows is defined as shown below, where f is the shedding frequency of the lift / drag.

Strouhal number:
$$S_t = \frac{fD}{U} \quad (6.7)$$

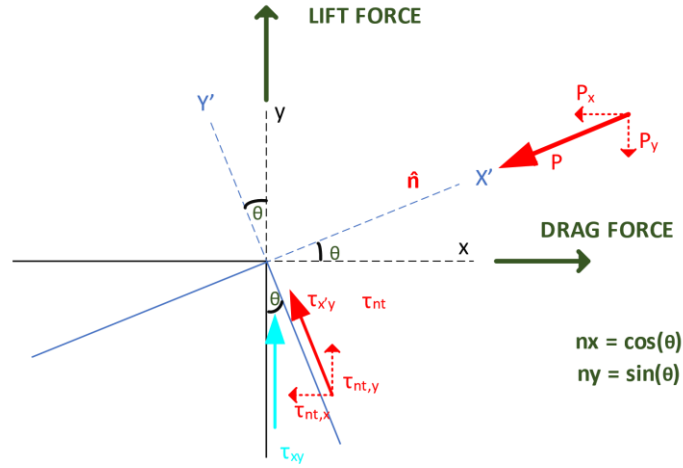


Figure 6.2: Lift and Drag forces components

Derivative Matrix Calculation: The strain vectors like $(\frac{\partial u}{\partial x})$ used above in shear stress calculation are constructed using basis functions as shown below.

$$\frac{\partial u}{\partial x} = \frac{\partial (\sum_{i=0}^N \psi_j u_j)}{\partial x} = \sum_{i=0}^N u_j \frac{\partial \psi_j}{\partial x} \quad (6.8)$$

$$\psi_j(x, y) = \psi_{j\xi}(\xi) \otimes \psi_{j\eta}(\eta) \quad (6.9)$$

Physical domain $(x, y) \rightarrow$ Computational domain (ξ, η)

$$\therefore \frac{\partial u}{\partial x} = \sum_{i=0}^N u_j \left(\psi_{j\eta} \frac{\partial \psi_{j\xi}}{\partial \xi} \frac{\partial \xi}{\partial x} + \psi_{j\xi} \frac{\partial \psi_{j\eta}}{\partial \eta} \frac{\partial \eta}{\partial x} \right) \quad (6.10)$$

Similarly, we can calculate $\frac{\partial u}{\partial y}$, $\frac{\partial v}{\partial x}$, & $\frac{\partial v}{\partial y}$

The quantities $(D_\xi = \psi_{j\eta} \frac{\partial \psi_{j\xi}}{\partial \xi})$ and $(D_\eta = \psi_{j\xi} \frac{\partial \psi_{j\eta}}{\partial \eta})$ are stored as general derivative matrices and all Jacobian quantities $(\frac{\partial \xi}{\partial x}, \frac{\partial \xi}{\partial y}, \frac{\partial \eta}{\partial x}, \frac{\partial \eta}{\partial y})$ are multiplied as required as per each element coordinate entities to compute τ_{nt}

Arrangement of the nodes on the center: The ξ & η needs to be tracked for each node on the cylinder as per the corresponding surrounding quad element as we required them in locating the right row in D_ξ & D_η confirming with the correct local node number. This is illustrated in figure 6.3 (a).

6.1.2 Normal Calculation

We require to calculate the normal (or angle θ) at each node to resolve the lift and drag forces in proper components for integration. Using the parametric equation of the body is not the best choice as we are not aware of the parametric equation of every bluff body. Instead, we make use of the shape functions in 1D case for the same purpose.

For linear mesh we can use of transformation ($x \rightarrow \xi$) and for curvilinear mesh, we can use the transformation $((x, y) \rightarrow \xi)$. We have tried to use the general transformation $((x, y) \rightarrow \xi)$ as it also takes into consideration the linear mesh and adjust accordingly. We make use of the Lagrange functions for calculating shape functions and its derivatives as shown in below table 6.2.

Table 6.2: 1D shape functions and derivatives for lift calculation

$N_1 = \frac{(\xi)(\xi - 1)}{2}$	$N_2 = \frac{(\xi - 1)(\xi + 1)}{-1}$	$N_3 = \frac{(\xi)(\xi + 1)}{2}$
$N_{1,\xi} = \frac{(2\xi - 1)}{2}$	$N_{2,\xi} = \frac{(2\xi)}{-1}$	$N_{3,\xi} = \frac{(2\xi + 1)}{2}$

We can find the derivatives at any point of 1D element using these shape functions as shown below:

$$\frac{\partial x}{\partial \xi} = \sum_{i=1}^3 N_{i,\xi} * x_i \quad \& \quad \frac{\partial y}{\partial \xi} = \sum_{i=1}^3 N_{i,\xi} * y_i \quad (6.11)$$

Where, x_i and y_i are the start, middle and end coordinates of the 1D element (GMSH data).

The Tangent vector, Normal vector and angle of Normal with x-axis can be calculated as follows. This θ can be used for resolving the forces into component:

$$Tangent = \left(\frac{\partial y}{\partial \xi}, \frac{\partial x}{\partial \xi} \right) \quad \& \quad Normal = \left(-\frac{\partial x}{\partial \xi}, \frac{\partial y}{\partial \xi} \right) \quad (6.12)$$

$$\theta = \tan^{-1} \left(-\frac{\partial y / \partial \xi}{\partial x / \partial \xi} \right) \quad (6.13)$$

$$n_x = \cos(\theta) \quad \& \quad n_y = \sin(\theta) \quad (6.14)$$

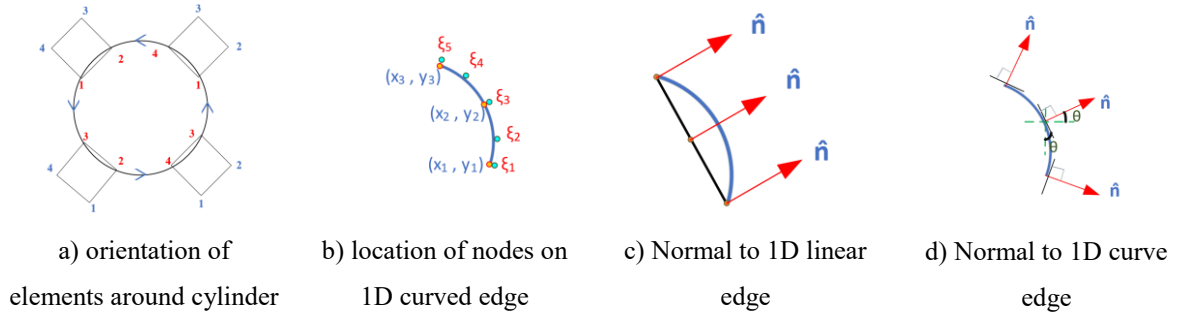


Figure 6.3: Lift-Drag Parameters

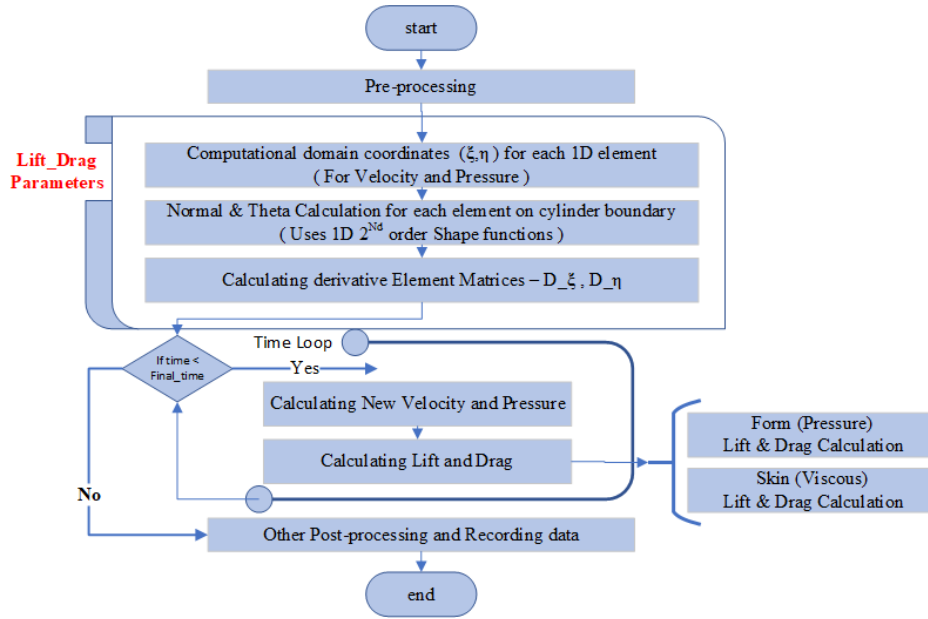


Figure 6.4: Lift and Drag forces Calculation Flowchart

6.2 Spectral Element Mesh

GMSH provides a basic quadrilateral element mesh suitable for lower-order finite element methods (FEM), including first- & second-order elements. However, for spectral element methods (SEM), the node distribution within each element must follow specific rules—typically using non-equidistant nodes or Lobatto points. Therefore, a refined GMSH mesh cannot be directly used as a refined mesh for SEM

To generate an SEM-compatible mesh, we place nodes at Lobatto roots within each quad element. The coordinates of these nodes are defined in the computational domain (ξ, η) & then mapped to the physical domain (x, y) using shape functions & the Jacobian. However, boundary nodes may be duplicated across adjacent elements, which must be avoided for

consistency. In C++, this issue can be addressed using data structures like HashMap to track unique nodes. In Fortran, due to limited built-in data structures, this becomes challenging.

Moreover, generating nodes independently within each element does not preserve global ordering. As a result, Dirichlet boundary nodes can be intermixed with interior nodes, complicating the elimination of corresponding rows & columns in the global matrix. Directly removing such nodes disrupts the structure of both the unknown vector & the global system matrix. To preserve matrix symmetry & structure, and to eliminate boundary nodes contribution, we use reduction as shown in section 3.3.5

To address these challenges, a SEM mesh generator is being developed in Fortran. It generates global node numbering with a specific order: boundary nodes first, followed by interface nodes, & finally, interior nodes. This ordering simplifies Dirichlet node elimination & enables straightforward renumbering. Interior nodes within an element can be solved independently, making them ideal for parallel element-level computations. Only interface nodes require a global solution approach, which can be further optimized in the project.

For this node numbering scheme, we first generate nodes along the edges of each element, followed by the internal nodes. By keeping track of edge nodes explicitly, we avoid the need for map-like structures. Instead, simple arrays can store node-edge associations, element-edge mappings, & edge boundary information. This structure also makes it easier to store neighbouring element data—an essential feature for implementing fluid-structure interaction (FSI) methods such as overset & stair-step techniques. A diagram illustrating this structure is provided below.

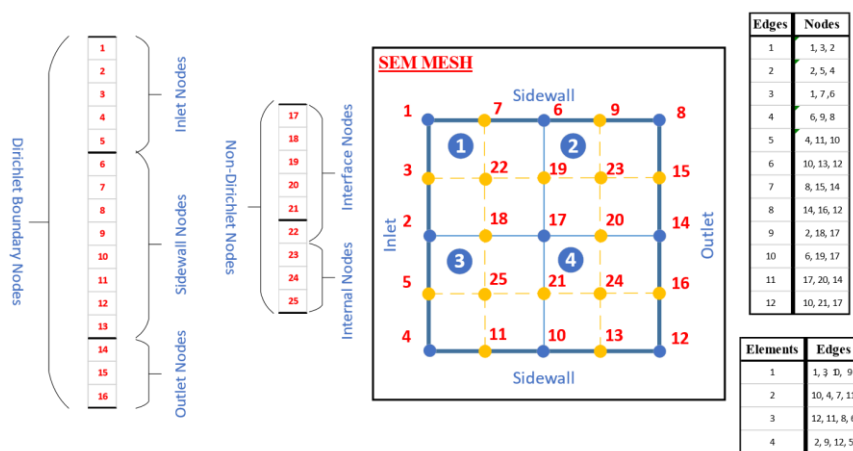


Figure 6.5: SEM Mesh Generation in Fortran

6.3 Curvilinear Mesh / Curved Mesh Elements

The shortcoming of using straight-edge elements is their inability to represent curved and complex geometries accurately. For accurate representation of these elements, curved mesh elements are computationally less expensive and better represent complex boundaries or surfaces with fewer elements.

Second-order shape functions are used instead of first-order ones in each direction. For curved elements, they are represented as products of quadratic Lagrange polynomials.

$$Na(\xi, \eta) = Lag^2(\xi)Lag^2(\eta)$$

Thus, the overall order of shape function is $2 + 2 = 4$ for the 9-noded quadrilateral. As the 9th node is always $(\xi = \eta = 0)$, we eliminate it and convert the shape function into 3rd order function for the mid-side nodes. The shape function corresponding to corner nodes- 1, 2, 3, 4 are modified using the shape functions of mid-side nodes (from existing 1st order shape functions) ensuring summation of the value of the shape function is 1 at all nodes. The modified shape functions are as follows:

Table 6.3: 2D shape functions for curved elements

$N_5(\xi, \eta) = \frac{1}{2}(1 - \xi^2)(1 - \eta)$	$N_6(\xi, \eta) = \frac{1}{2}(1 + \xi)(1 - \eta^2)$
$N_7(\xi, \eta) = \frac{1}{2}(1 - \xi^2)(1 + \eta)$	$N_8(\xi, \eta) = \frac{1}{2}(1 - \xi)(1 - \eta^2)$
$N_1 \leftarrow N_1 - \frac{1}{2}(N_5 + N_8)$	$N_2 \leftarrow N_2 - \frac{1}{2}(N_5 + N_6)$
$N_3 \leftarrow N_3 - \frac{1}{2}(N_6 + N_7)$	$N_4 \leftarrow N_4 - \frac{1}{2}(N_7 + N_8)$

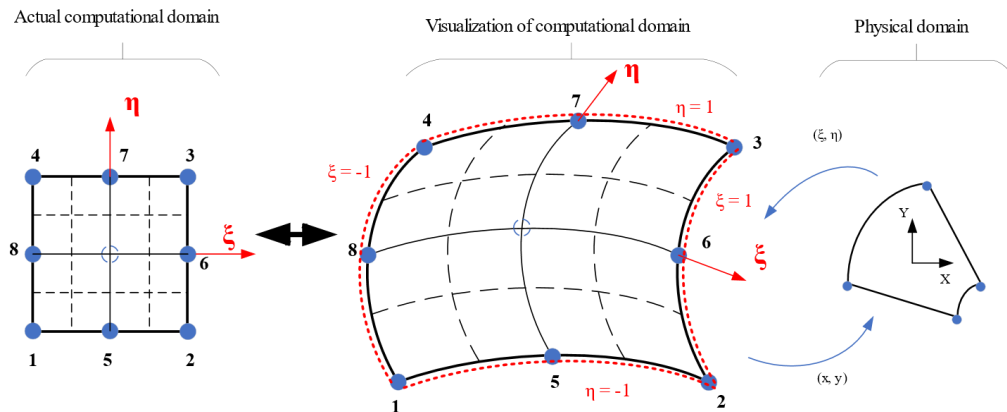


Figure 6.6: Curved Mesh Elements

6.3 Program Structure / Flowchart

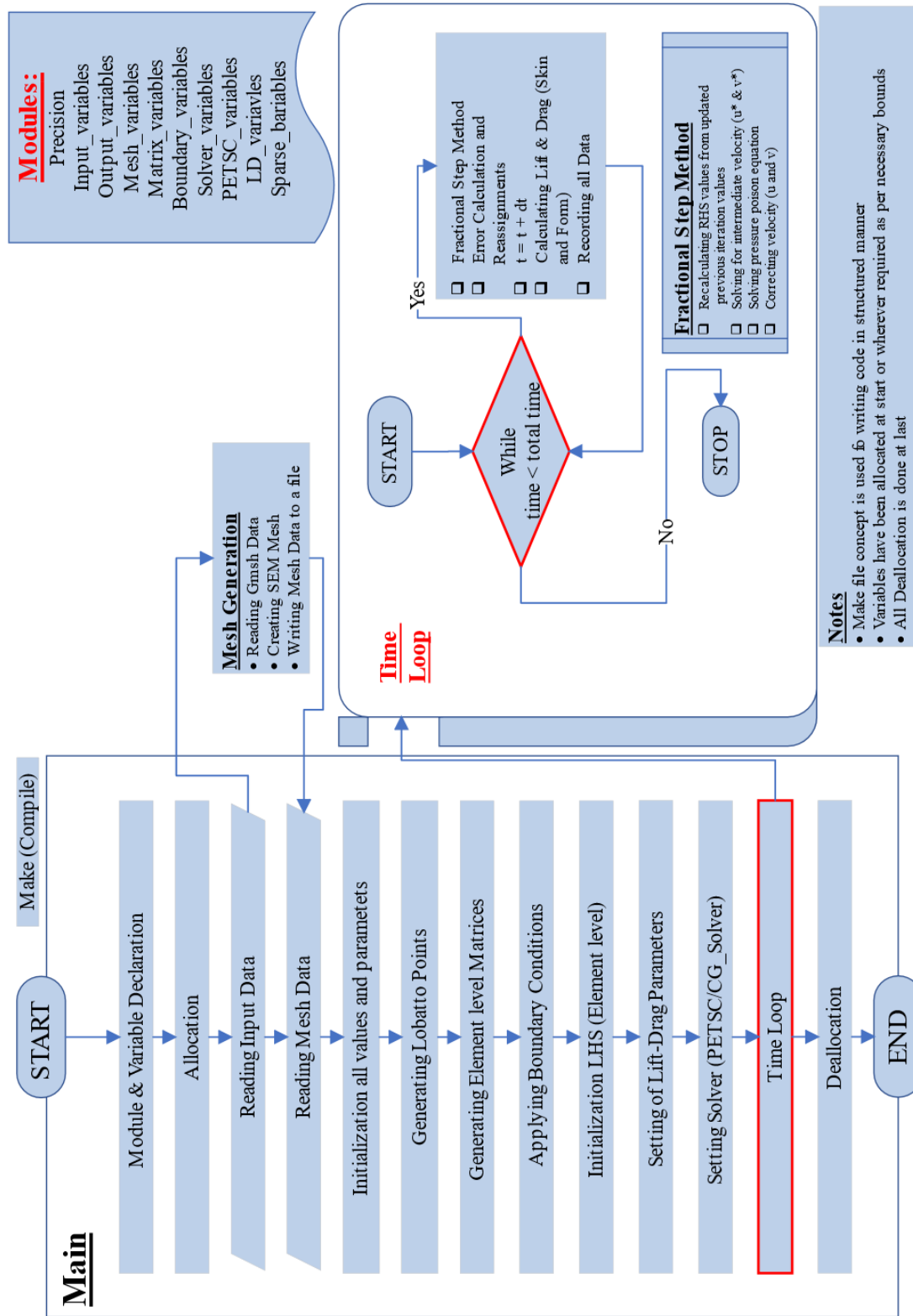


Figure 6.7: Code Flowchart

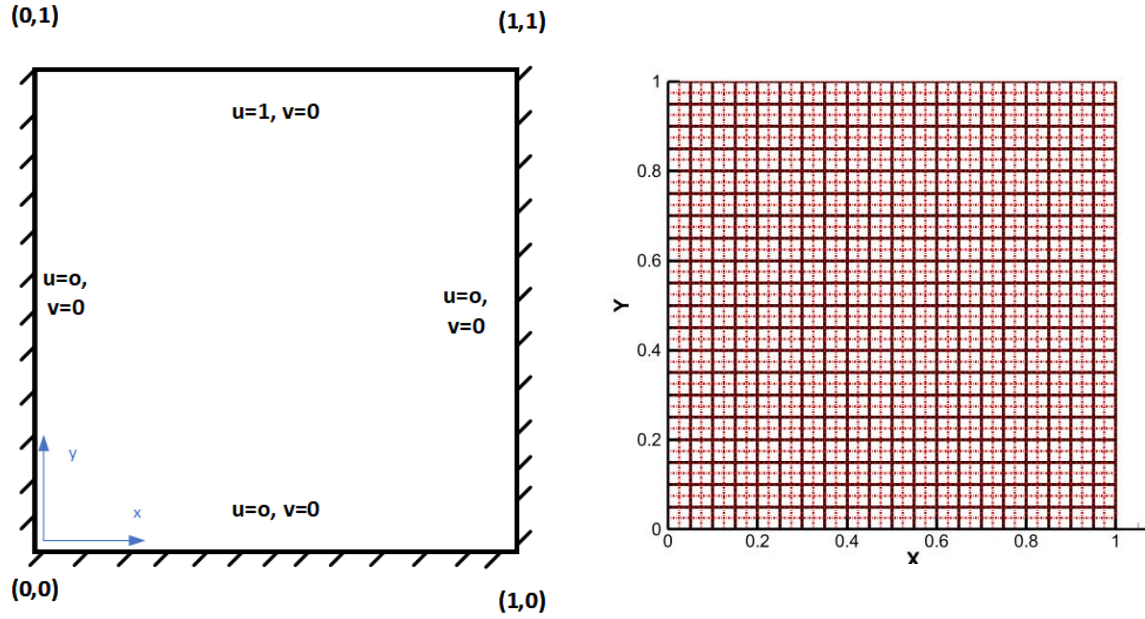
Results and Discussion

In this section, the solver is tested against the basic CFD problems. We perform transient analysis for all the problems using the Fractional step method as discussed before. The initial problems of Lid-Driven Cavity problem and Poiseuille's flow problem are solved by EBE approach. The cases for flow around square, circular and elliptical cylinder are solved by global assembly approach. We have validated our results wherever possible comparing it with renowned data and relevant literature. We have considered only application of Dirichlet boundary condition in whole discussion. If no boundary condition is mentioned, the homogeneous Neumann boundary conditions are automatically applied, otherwise the non-zero Neumann BCs needs to specifically specified wherever required.

7.1 Lid-Driven Cavity flow problem

The Lid-driven cavity flow problem is the most common benchmark constrained steady state flow problem to test all the basic methods, schemes, and solvers. It is a non-dimensional problem involving a square domain of unit size with all sides having homogeneous Dirichlet boundary conditions for velocity, except the top surface. The top surface moves with unit velocity along x direction and is stationary in y direction. The pressure gradient along normal direction to all walls is zero, i.e. $\frac{\partial p}{\partial \hat{n}} = 0$. The Reynold's number for this case is given by $Re = \frac{UL}{\nu}$, where $U=1$ is the velocity of top surface, characteristic length (dimension of square cavity): $L=1$, and ν is kinematic viscosity. There are 20 elements along each x and y direction.

While solving above problem by iterative approach, we have considered the $L2$ norm for error with a tolerance limit of 10^{-6} for both pressure and velocity convergence to reach steady state



(a) Lid Driven Cavity Geometry and BC

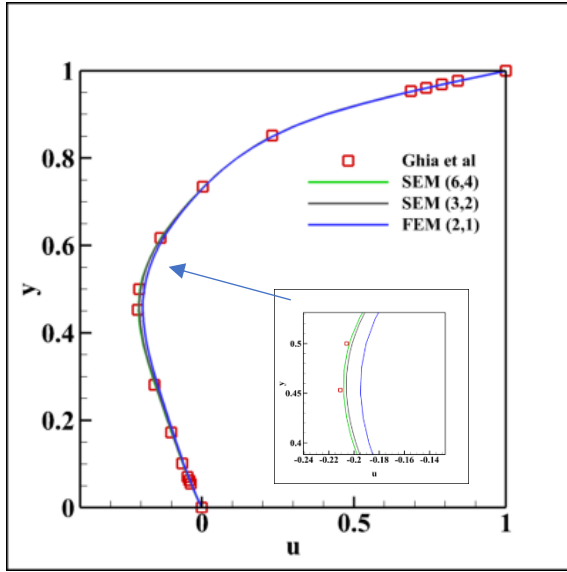
(b) Mesh details - velocity mesh nodes (dotted & solid lines) and pressure mesh nodes (solid lines)

Figure 7.1: Lid-Driven Cavity Flow

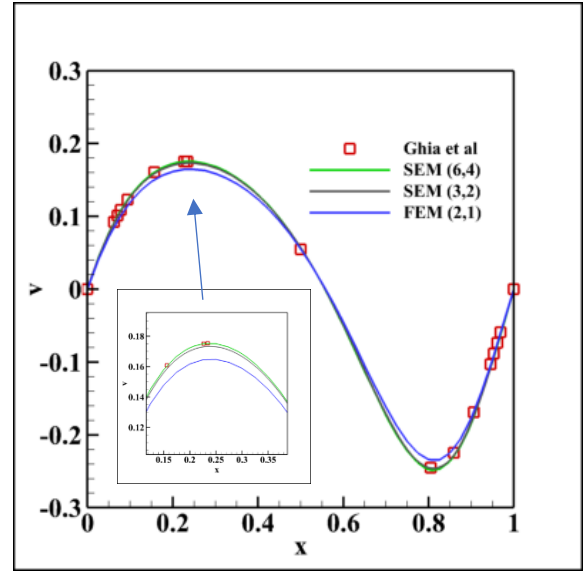
We have analysed the solution for lower unequal order interpolation for velocity and pressure to test the Taylor-Hood element model. The results are compared with Ghia. et.al [36]. We have plotted contours for vorticity, stream function, the variation of u velocity with y along the mid vertical line ($x = 0.5$) of the cavity, and the variation of v velocity with x along the mid-horizontal line ($y = 0.5$) of the cavity, as illustrated in figures below for Reynold's number = 100, 400 and 1000. Above simulation

For lower Reynolds numbers, a lower polynomial order can be employed, while for higher Reynolds numbers, a higher polynomial order should be utilized. As we increase the order of SEM, the results become more accurate, but the calculation cost increases. As, we can see at lower order, we can get significant results, without much compromise in accuracy as seen from plots for u vs y and v vs x graphs, as shown below.

The plot in figure 7.2 denotes the velocity plots at mid plane for $Re=100$ using different order of interpolation of polynomial as described in bracket (velocity order, pressure order). The figure 7.3 presents the velocity at mid line profiles for $Re=100, 400$ and 1000 for 4th order expansion function for velocity and 3rd order expansion function for pressure. The figures 7.4, 7.5, & 7.6 depict the velocity contour, stream function and vorticity contours at $Re=100, 400$ and 1000 respectively. All results seem to agree with literature data.

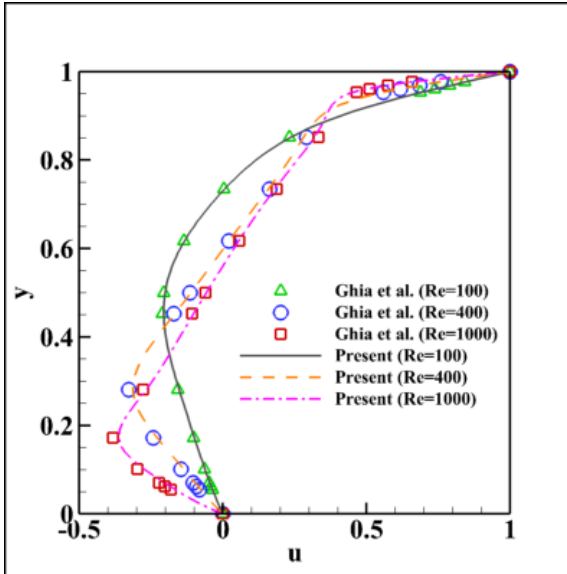


(a) u velocity variation with y at mid-vertical line ($x=0.5$) at $Re=100$

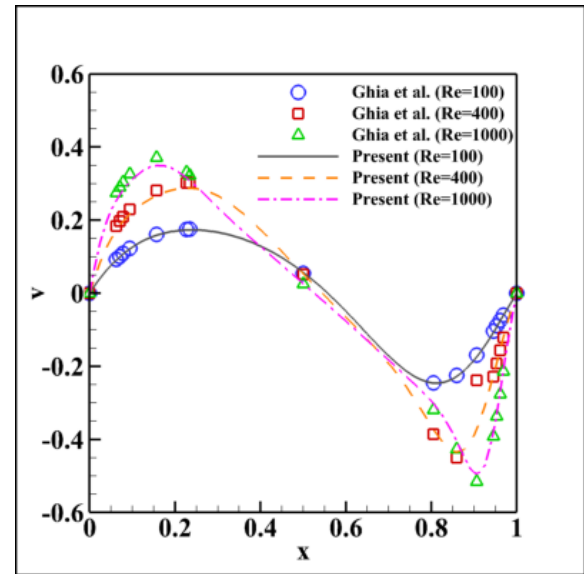


(b) v velocity variation with x at mid-horizontal line ($y=0.5$) at $Re=100$

Figure 7.2: Velocity variation at mid-lines at $Re=100$

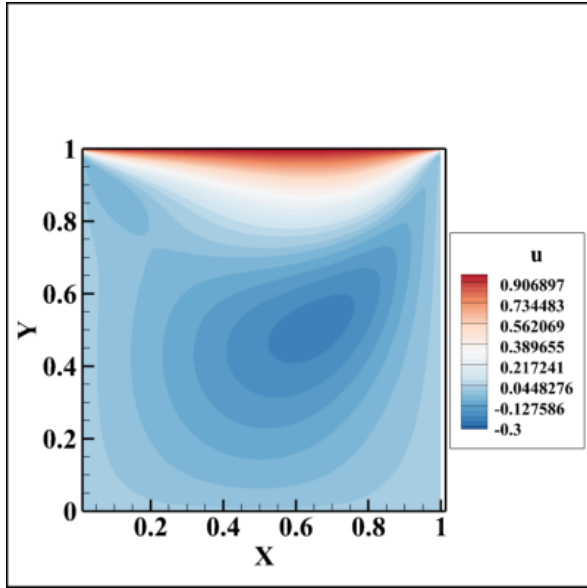


(a) Variation of u velocity with mid-vertical line ($x=0.5$)

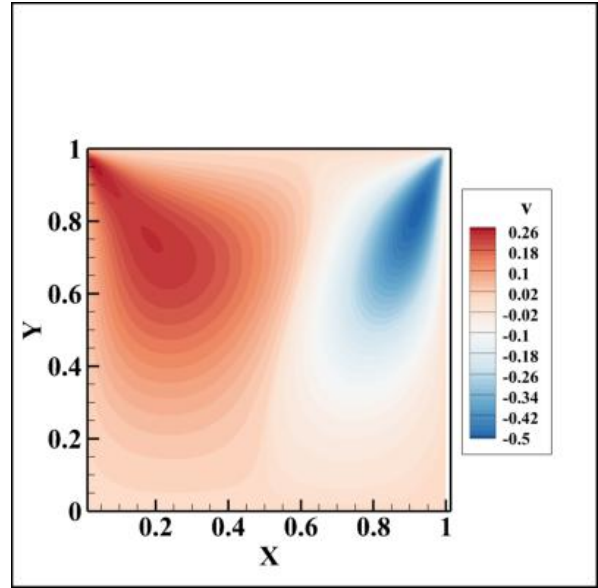


(b) Variation of v velocity with mid-horizontal line ($y=0.5$)

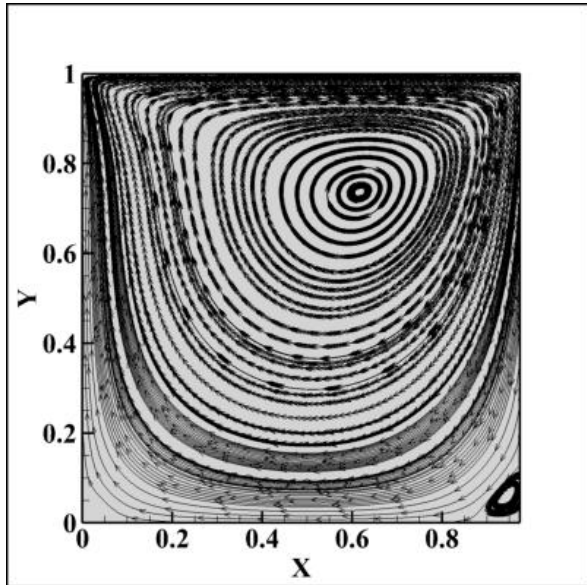
Figure 7.3: Velocity variation at mid-lines at $Re=100, 400, \& 1000$



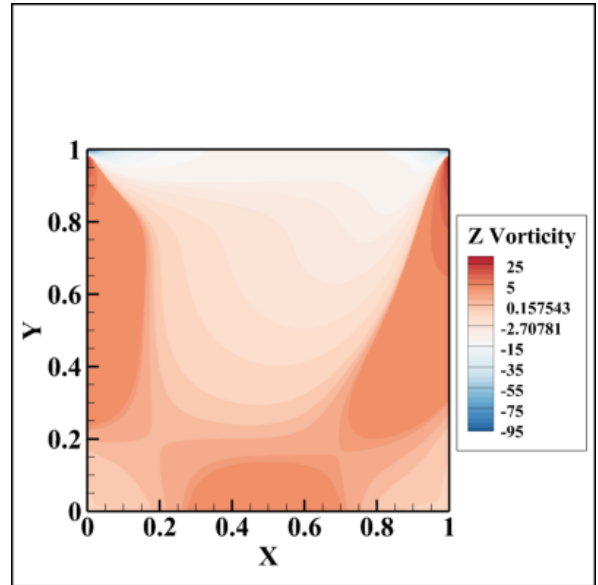
(a) u velocity profile



(b) v velocity profile

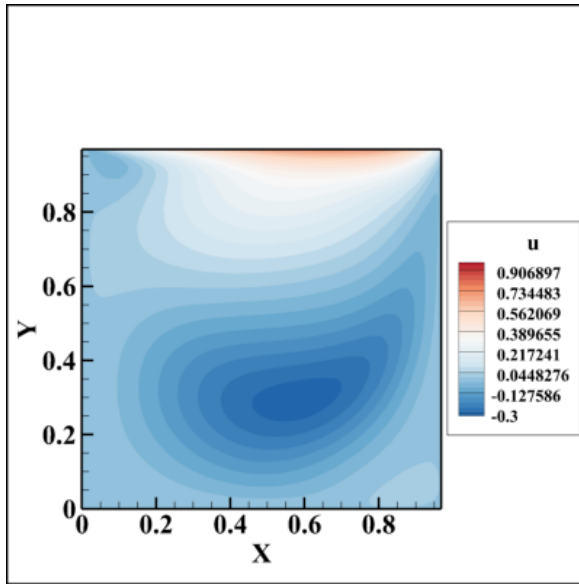


(c) Stream function

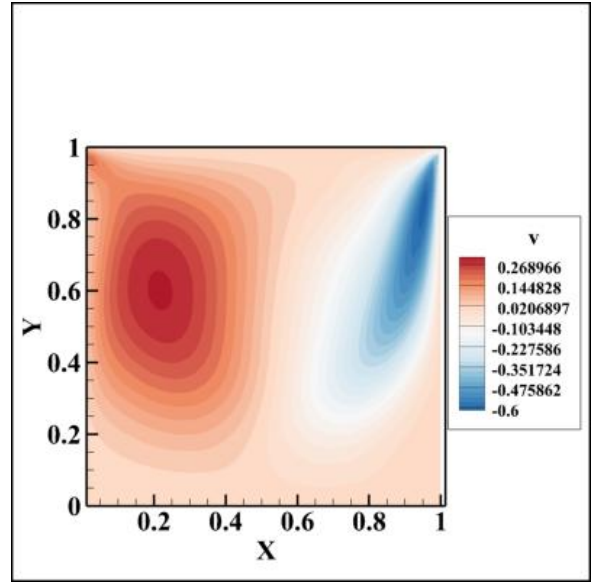


(d) Vorticity Contour

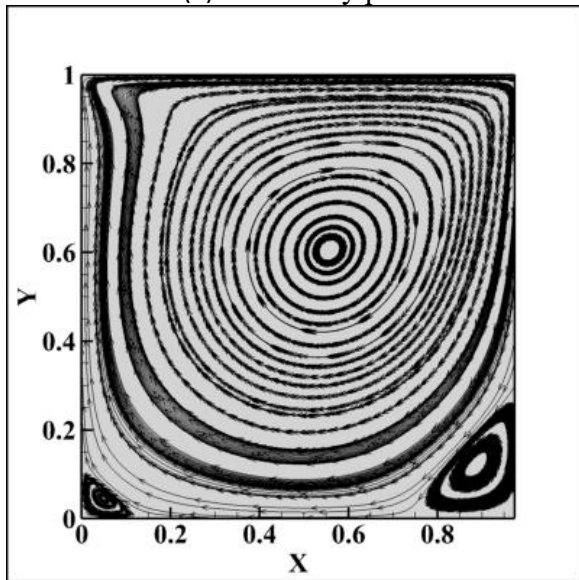
Figure 7.4: Velocity contours, stream function and vorticity contours at $Re=100$



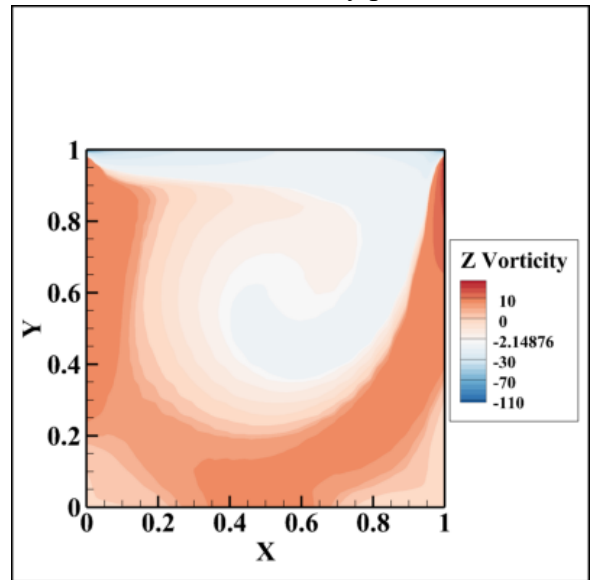
(a) u velocity profile



(b) v velocity profile

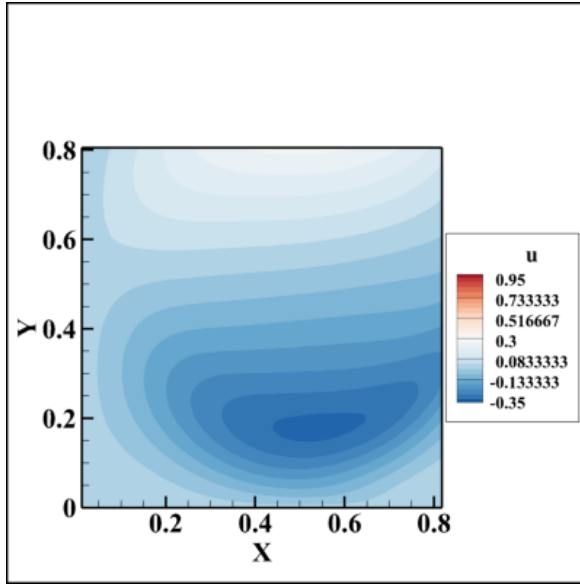


(c) Stream function

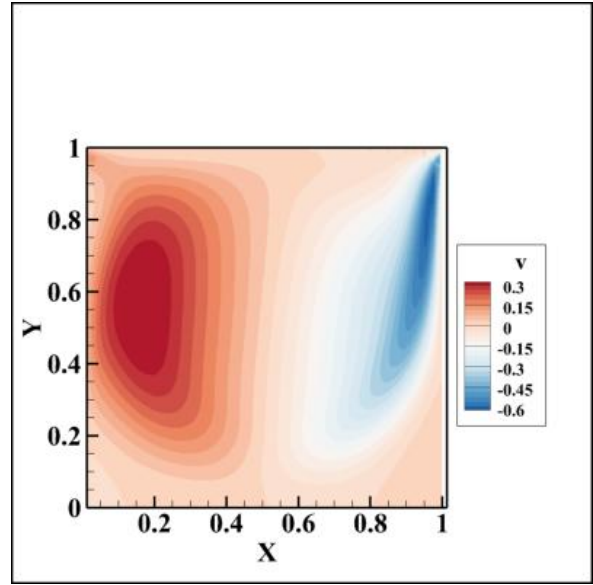


(d) Vorticity Contour

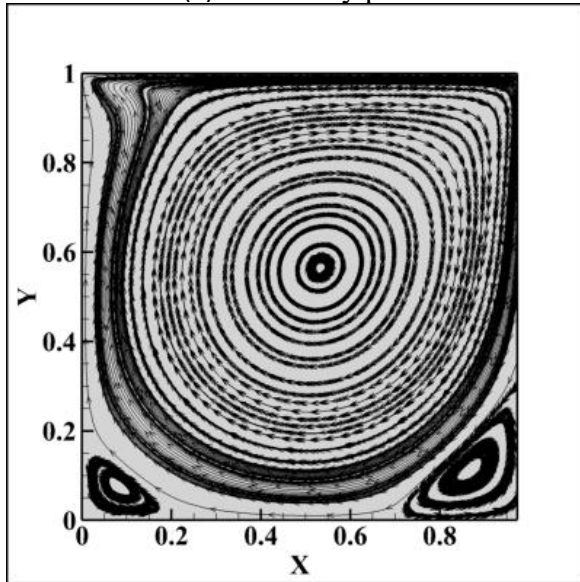
Figure 7.5: Velocity contours, stream function and vorticity contours at $Re=400$



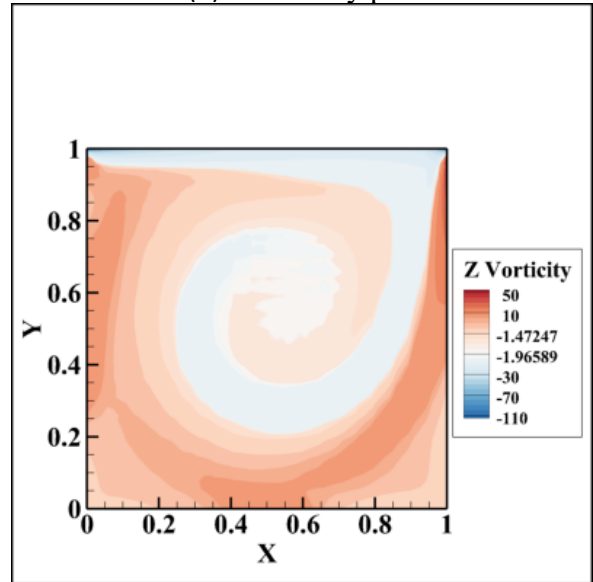
(a) u velocity profile



(b) v velocity profile



(c) Stream function



(d) Vorticity Contour

Figure 7.6: Velocity contours, stream function and vorticity contours at $Re=1000$

The 4th order interpolation results in 81×81 nodes, as compared to 128×128 nodes for Ghia, but the results are still in good sync with the literature data. The computational time reduced by almost 3 times. SEM can work with any coarse mesh and it can use higher order interpolation as required to get accurate results. Similar fast and accurate results have been obtained for the Temperature distribution across 2D plat problem

7.2 Plane Poiseuille flow problem

This problem is studied to assess the inflow and outflow of fluid through the domain. Here, the L and H represents the length and height of the channel. Here, u and v are the velocity components in x and y direction. The upper and bottom walls of channel are fixed, which lead to no slip boundary conditions at wall ($u = 0, v = 0$). The inlet velocity is uniform and equal to U_{avg} . The outlet velocity and pressure have Neumann BC.

To assess the development of flow till it fully develops, we consider the formula $L_e = 0.057 * Re * D$ to calculate the entrance length of flow, and thus try to keep the length of channel more than L_e . Here, we are analysing the flow for $Re=100$, and we have taken the height of channel as 1 unit and length of channel as 15 units. We cannot guarantee the velocity at midplane in fully developed flow region $U_{max} \leq 1.5U_{avg}$, due to small introduction of turbulence into the flow. But still the flow at midplane is close to this value. 100 elements are considered along length of channel and 20 elements along width of channel with second order approximation for velocity and 1st order approximation for pressure for approximating solution within the domain. We have considered the $L2$ norm for error with a tolerance limit of 10^{-6} for both pressure and velocity convergence to reach steady state. The velocity profile plots along a vertical axis in developed region and in developing region are as follows:

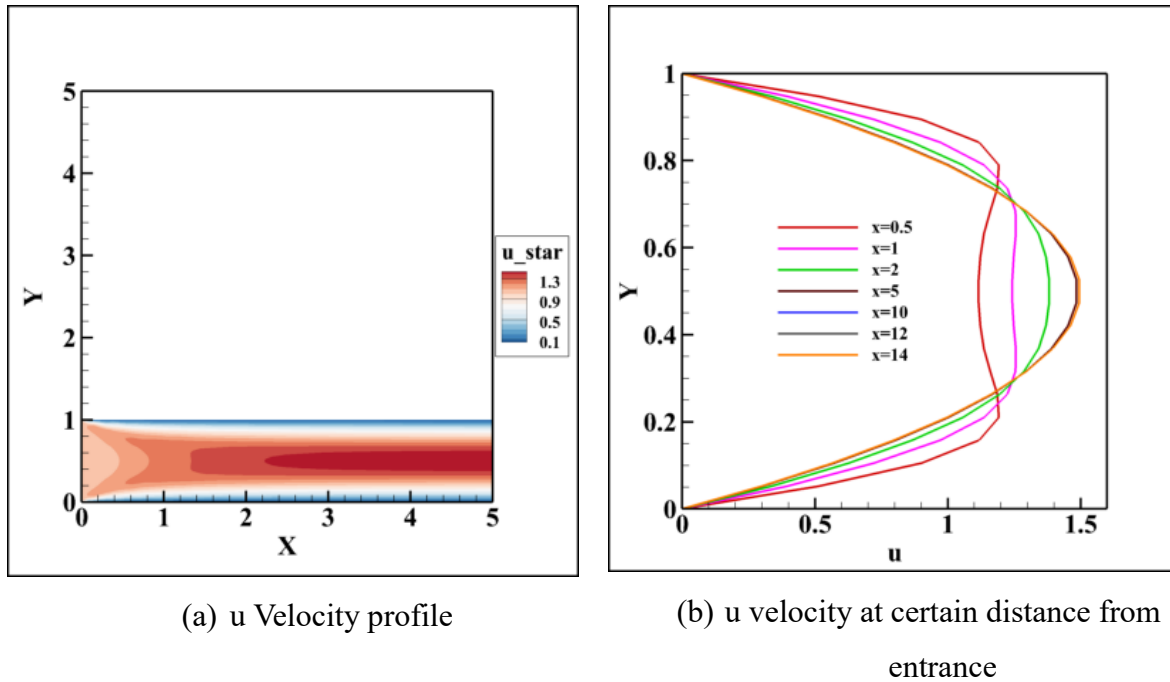


Figure 7.7: Velocity profile for Poiseuille's flow at $Re=100$

7.3 Flow around Circular cylinder:

For all above cases, we have used structured mesh where each element is square or rectangular and oriented in specific manner. For Circular cylinder case, all mesh elements need not be made of straight edges. To capture the actual geometry, we need to make use of very fine straight edge mesh (FEM), or we can make use of curved coarse higher order FEM /SEM mesh. We will compare the results from both mesh. All the flow parameters and geometric boundary conditions are depicted in figure 7.8. The Cylinder diameter is D and all other dimensions are expressed in terms of D .

Cylinder has no slip boundary condition ($u = v = 0$). The outlet has zero Neumann boundary condition ($\frac{\partial u}{\partial n} = \frac{\partial v}{\partial n} = 0$). The pressure at inlet, outlet, sidewall, and cylinder has Neumann boundary condition ($\frac{\partial p}{\partial n} = 0$). The simulation is run for 50000 timesteps with a time step of 0.005 for $Re = 100$. The Schematic diagram for both FEM and SEM case is shown below.

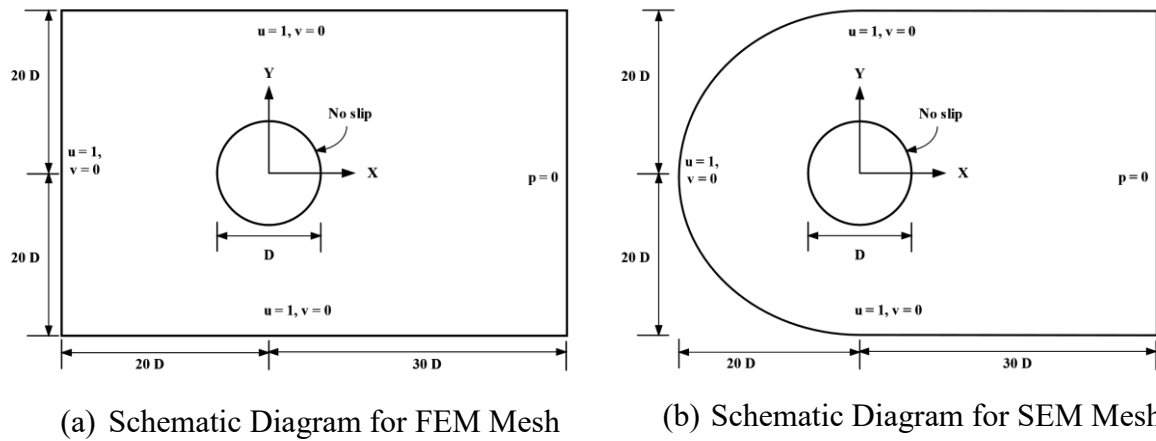


Figure 7.8 Schematic Diagram for circular cylinder

The analysis is done for Reynolds number $Re = \frac{UD}{\nu} = 100$, with free stream velocity of $U_{\infty} = 1$. All the plots for Vorticity, Velocity Contours and Lift & Drag are shown below along with mesh independence test. All the results have been simulated using PETSC CG Solver with ILU preconditioner.

FEM Mesh Results: The analysis is done for Taylor-Hood element with 2nd order interpolation for velocity involving (21826 nodes) and 1st order interpolation for pressure using (5513 nodes). Total number of quadrilateral elements in mesh are 5400. The FEM Mesh is shown below. The results are also validated with the literature.

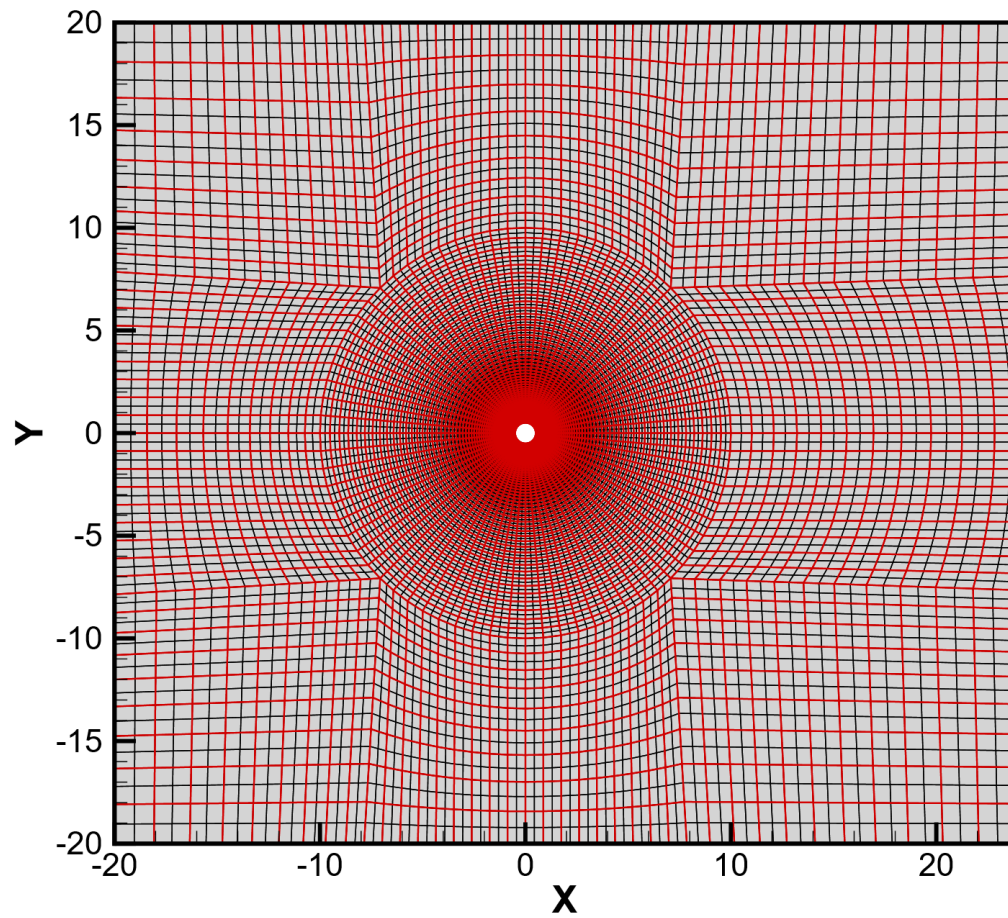
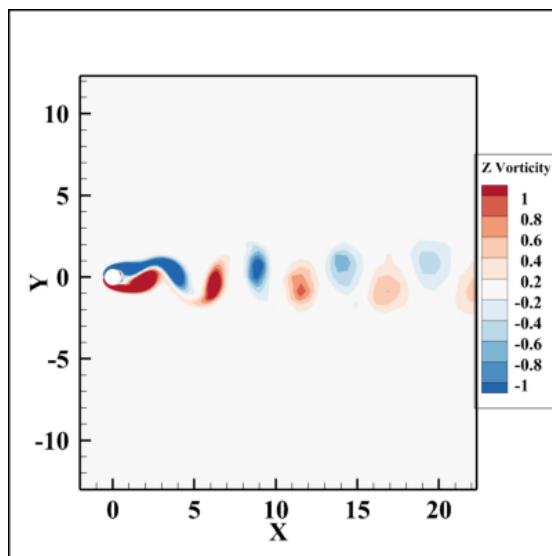
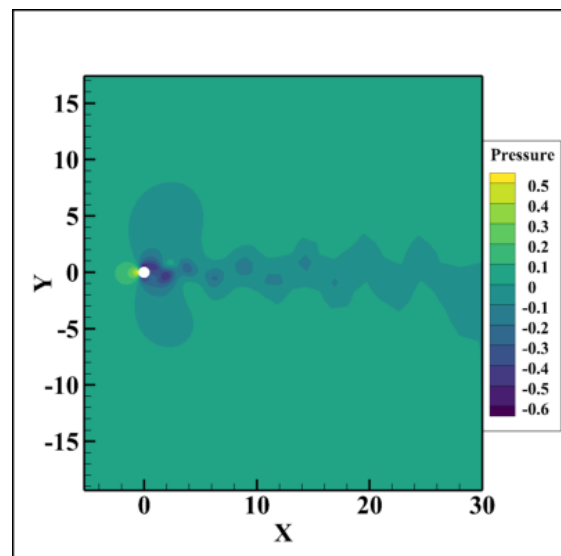


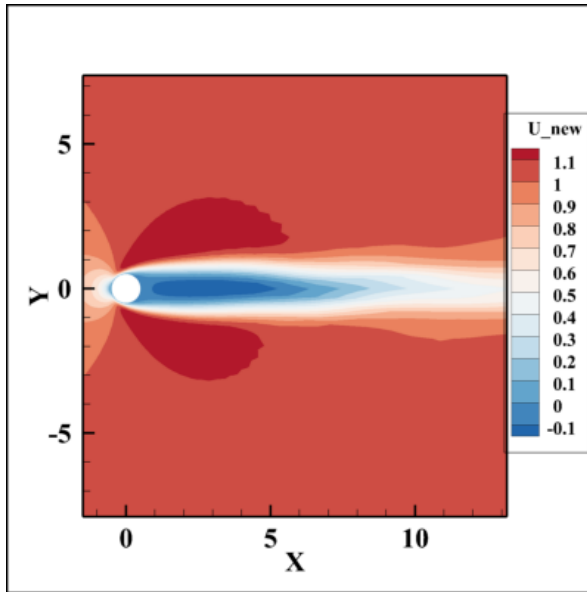
Figure 7.9 Taylor-Hood FEM Mesh for circular cylinder



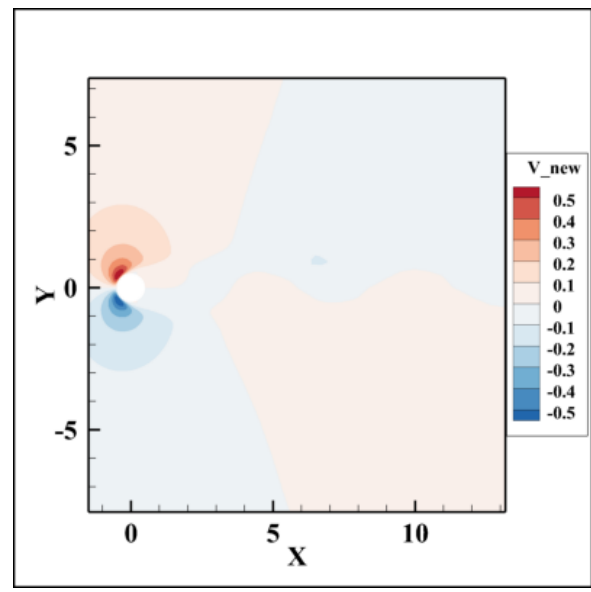
(a) Vorticity Contour



(b) Pressure Contour

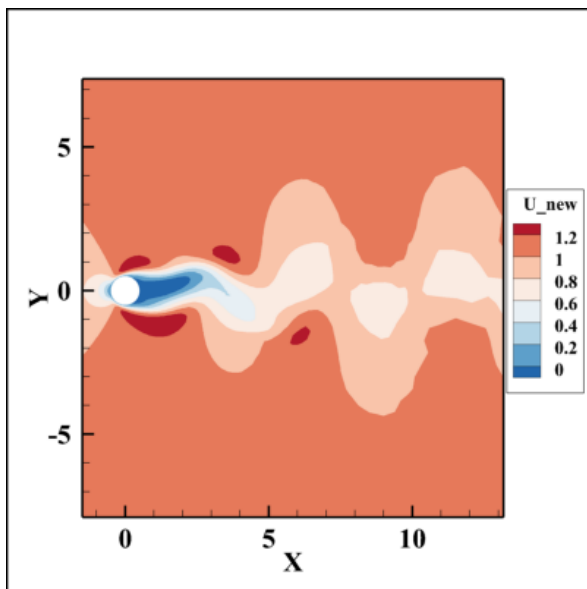


u velocity

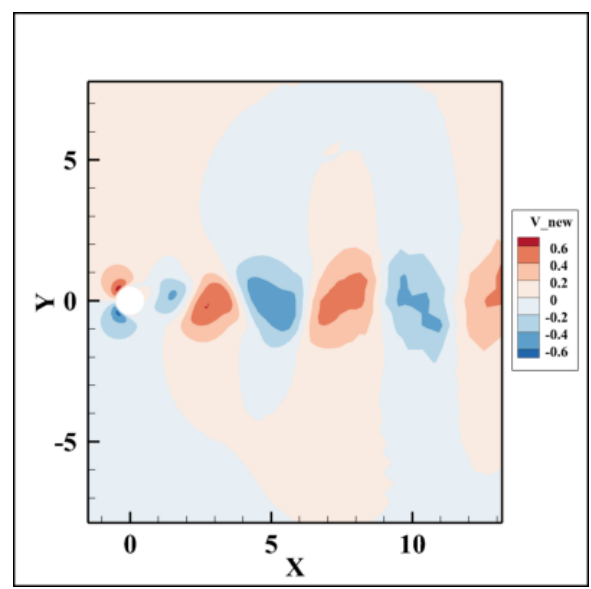


v velocity

(c) Velocity plots (Before Vortex Shedding at $t = 50$ s)

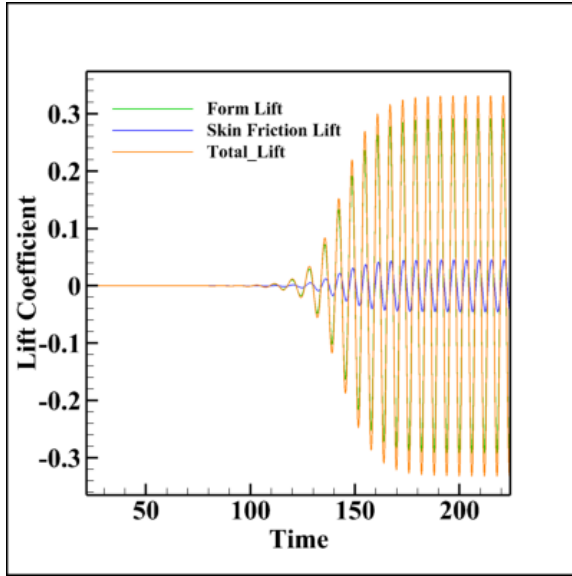


u velocity

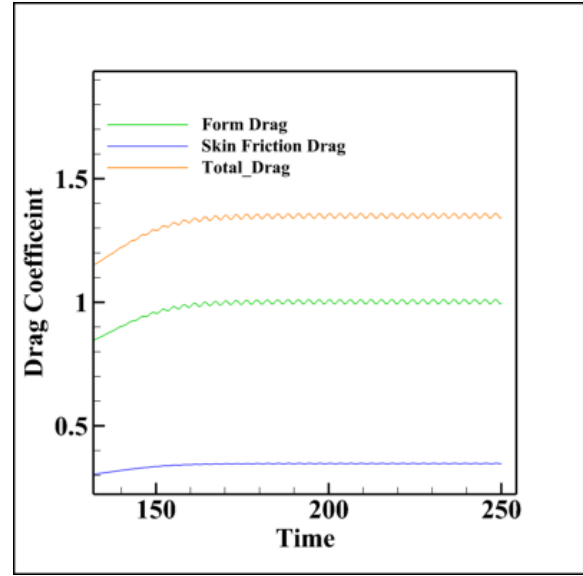


v velocity

(d) Velocity plots (After Vortex Shedding at $t = 200$ s)



(e) Lift Coefficient



(f) Drag Coefficient

Figure 7.10 Taylor-Hood FEM plots for circular cylinder

Simulation time with various preconditioners using PETSC KSP CG Solver is shown below.

Table 7.1: Results for various preconditioners for FEM mesh for circular cylinder at $Re=100$

Preconditioner	Average Iterations/timestep			Computational Time (s)
	u^*	v^*	Pressure	
No Preconditioner	181	344	312	19327.276
Jacobi Preconditioner	4	5	288	3678.5464
ILU Preconditioner	2	2	68	2819.773
GAMG Preconditioner	2	3	6	3051.1355

The ILU preconditioner takes the least amount of time. Therefore, we have used ILU preconditioner for PETSC KSP CG Solver for all further results. As the number of nodes increases to large extent, then GAMG preconditioner will give better results in terms of both: iterations and computational time.

The lift and drag values were calculated incorrectly earlier in phase 1 work. They have been corrected after modifying and rewriting the code for Lift & Drag calculation. All results are in very good range of agreement with literature. The coefficient of Lift and Drag as well as the Strouhal number are validated below. The Lift and Drag forces have major contribution from pressure, and they also have been validated with literature.

Table 7.2: Lift and Drag coefficients for FEM Mesh for circular cylinder at $Re = 100$

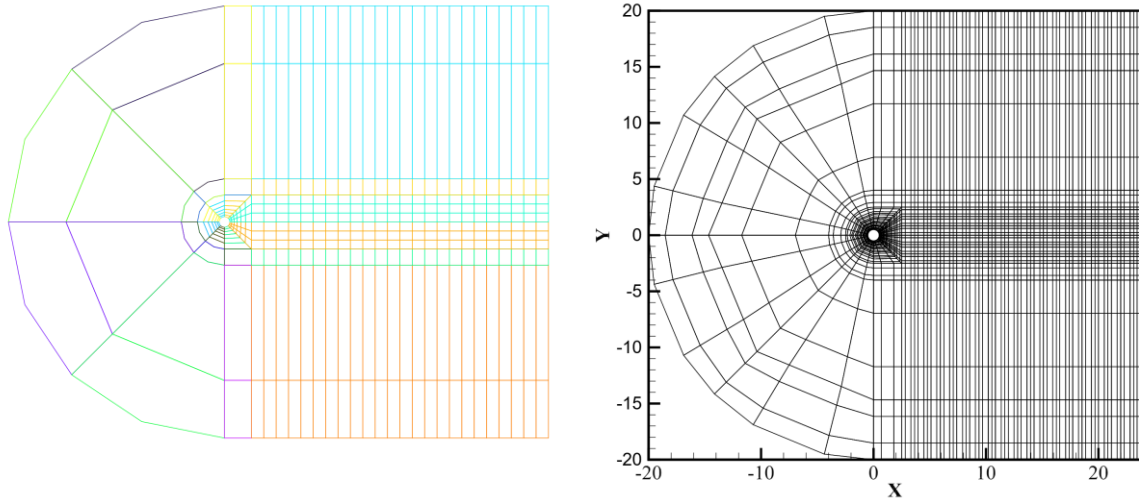
	Numerical Results		
	Mean Value	Peak Value	RMS value
Form Lift Coefficient ($C_{L,p}$)	0	0.2918	0.206122
Form Drag Coefficient ($C_{D,p}$)	1.02	-	-
Skin friction Lift Coefficient ($C_{L,v}$)	0	0.04535	0.0320673
Skin friction Drag Coefficient ($C_{D,v}$)	0.3475	-	-
Total Lift Coefficient (C_L)	0	0.332	0.23476
Total Drag Coefficient (C_D)	1.35	-	-

Table 7.3: Validation of Lift & drag coefficients and Strouhal number with literature for circular cylinder at $Re = 100$

	FEM (2,1) (Present)	Park et al[37].	Kravchenko and Moin[38]	Stalberg et al.[39]	Absolute Error (%)
$C_{D,p}$ (mean)	1.01	0.99	0.99	0.972	2.020
C_D (mean)	1.35	1.33	1.32	1.32	1.504
C_L (rms)	0.2347	0.235	0.222	0.233	2.043
Strouhal Number (S_t)	0.164	0.165	0.164	0.166	0.165

SEM Mesh Results:

With SEM or higher order mesh, the initial GMSH mesh can be coarse and we can construct a fine mesh from it using subroutine which we have discussed earlier. The number of quad elements is very less, and still, we get same results as FEM Mesh. The accuracy of the solution increases due to use of higher order polynomial for interpolation. The results for mesh independence test and Order independence test have been presented in the table below. All the results have been simulated using PETSC CG Solver with ILU preconditioner. Tolerance is set at $1.0E-6$ and total 50000 timesteps have been performed. All boundary Conditions are same as FEM case. We have chosen circular domain instead of rectangular domain for reducing the number of elements.



(a) Initial GMSH Coarse Element Mesh (b) Schematic Diagram for SEM Mesh

Figure 7.11 Initial GMSH coarse mesh and 4th order SEM mesh for circular cylinder

The GMSH mesh with 378 quadrilateral elements is plotted on left and corresponding 3rd order SEM mesh is plotted on right. All the plots of Velocity Contours, Pressure Contours, Vorticity and Lift and Drag are similar. The Lift plot between FEM and SEM results have been shown below:

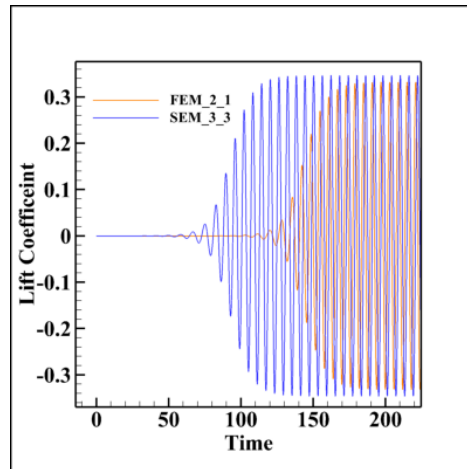


Figure 7.12 Total Lift Coefficient plots for FEM & SEM Mesh for circular cylinder

The Mesh Independence test and Order independence test have been performed and the results are tabulated below. For SEM, we use equal order interpolation for velocity and pressure. 3 meshes of 378, 454 and 568 elements have been used at 3rd, 4th and 5th order. 2nd order mesh with 378 elements shows 24% error, which indicates to use higher order mesh. The 378 elements coarse mesh cannot give results with FEM but it gives correct results with higher

order mesh (SEM). (Many of these results have been parallelly executed at a time and so they will require lower computational time if ran individually)

Key Observations

- We get results for lower number of nodes in less time with good accuracy.
 - FEM – 5513 nodes - 2819.773
 - SEM – 3519 - 2648.35242
- As the order increases, the number of nodes(unknowns) increases, which increases the computational cost
- Increasing the order increases the accuracy of the solution, but the increase in computational cost exceeds the gain of accuracy. So, we can avoid unnecessary use of very higher order SEM for negligible increase in accuracy.

So, comparing the results and ensuring optimum computational time, 4th order SEM mesh seems suitable for our current study.

Table 7.4: Results for various SEM orders for circular cylinder at Re = 100

Order of Interpolation	Quadrilateral elements	Total number of Nodes for Velocity and Pressure	Average Iterations/timestep			Computational Time (s)
			u^*	v^*	Pressure	
2 nd order	378	1590, 1590	1	1	20	617.4410
3rd order		3519, 3519	1	1	34	2648.35242
4th order		6204, 6204	1	1	46	4575.9793
5th order		9645, 9645	1	1	53	8382.1399
6th order		13842, 13842	1	1	60	10254.0523
3rd order	454	4179, 4179	1	1	40	3067.2330
4th order		7388, 7388	1	1	53	4605.7763
5th order		11505, 11505	1	1	61	11099.4297
6th order		16530, 16530	1	1	70	18112.0799
3rd order	568	5217, 5217	1	1	45	2419.5261
4th order		9228, 9228	1	1	59	3974.5102
5th order		14375, 14375	1	1	68	10064.754
6th order		20658, 20658	1	1	79	17855.978

Table 7.5: Mesh/Order Independence for circular cylinder at $Re = 100$

Quadrilateral elements = 378							
Order of Interpolation	Total Nodes for Velocity and Pressure	Total Peak Lift			Total Mean Drag		
		Value	Difference	Absolute Error (%)	Value	Difference	Absolute Error (%)
2 nd order	1590	0.25086	-0.0817	24.567	1.345	-0.01	0.7380
3 rd order	3519	0.34622	0.01366	4.10753	1.37	0.015	1.1070
4 th order	6204	0.33323	0.00067	0.20147	1.353	-0.002	0.1476
5 th order	9645	0.33217	-0.00039	0.1173	1.354	-0.004	0.0780
6 th order	13842	0.33256	0	0	1.355	0	0

7.4 Flow around square cylinder:

Similar analysis as circular cylinder is done for square cylinder. Here We make use of 1st order SEM mesh as all edges of the geometry are straight. For this geometry, the structured mesh contains each element as either square or rectangular. The SEM results are presented here. All the flow parameters and geometric boundary conditions are depicted in figure (7.3). The Cylinder hydraulic diameter is D and all other dimensions are expressed in terms of D .

Cylinder has no slip boundary condition ($u = v = 0$). The outlet has zero Neumann boundary condition ($\frac{\partial u}{\partial n} = \frac{\partial v}{\partial n} = 0$). The pressure at inlet, outlet, sidewall, and cylinder has Neumann boundary condition ($\frac{\partial p}{\partial n} = 0$). The simulation is run for 50000 timesteps with a time step of 0.005 for $Re = 100$. The Schematic diagram for this case is shown below.

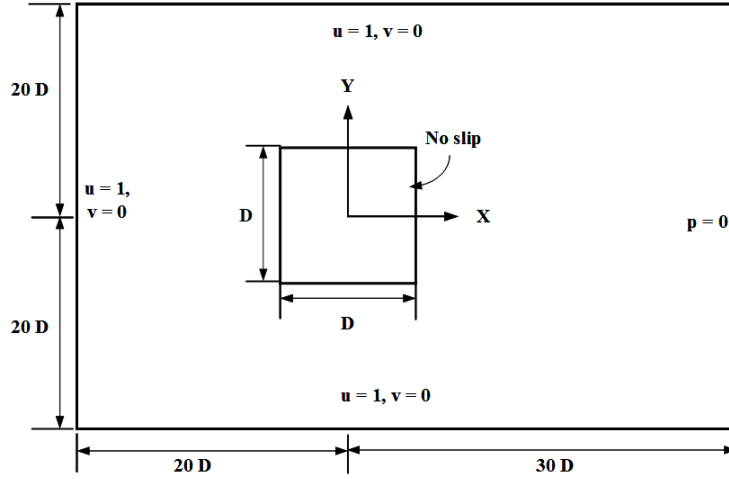
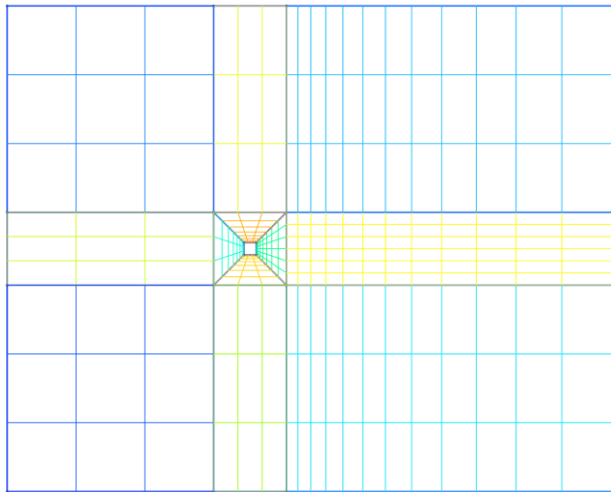


Figure 7.13 Schematic Diagram for square cylinder

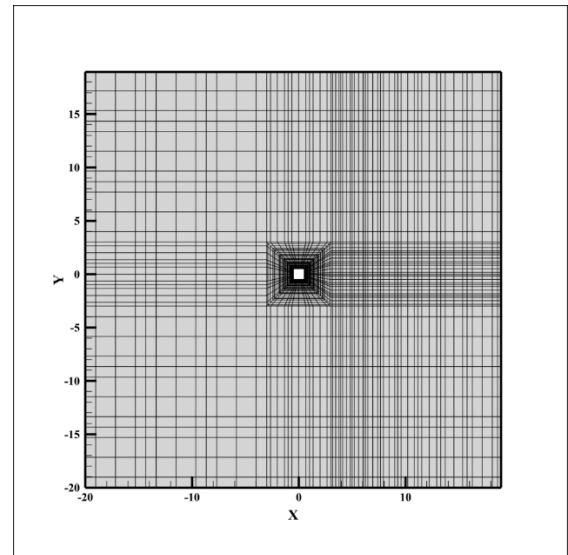
The analysis is done for Reynolds number $Re = \frac{UD}{\nu} = 100$, with free stream velocity of $U_{\infty} = 1$. All the plots for Vorticity, Velocity Contours and Lift & Drag are shown below along with mesh independence test. All the results have been simulated using PETSC CG Solver with ILU preconditioner.

SEM Mesh Results:

The GMSH mesh with 294 quadrilateral elements is plotted on left and corresponding 4th order SEM mesh is plotted on right.



(a) Initial GMSH Coarse Element Mesh



(b) Schematic Diagram for SEM Mesh

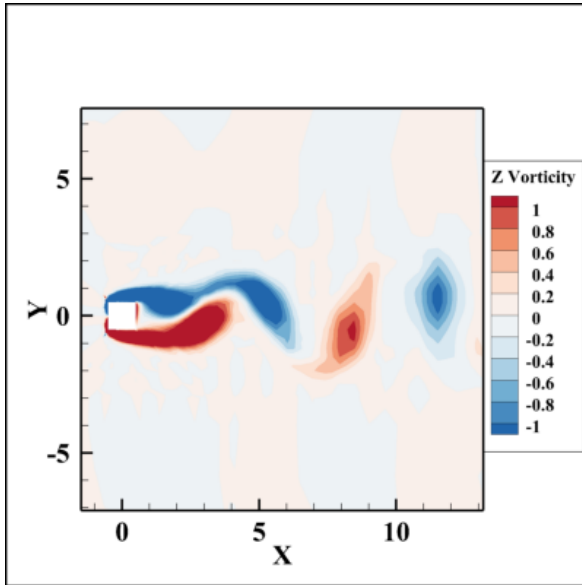
Figure 7.14 Initial GMSH coarse mesh and 4th order SEM mesh for square cylinder

The Mesh/Order independence results are tabulated below.

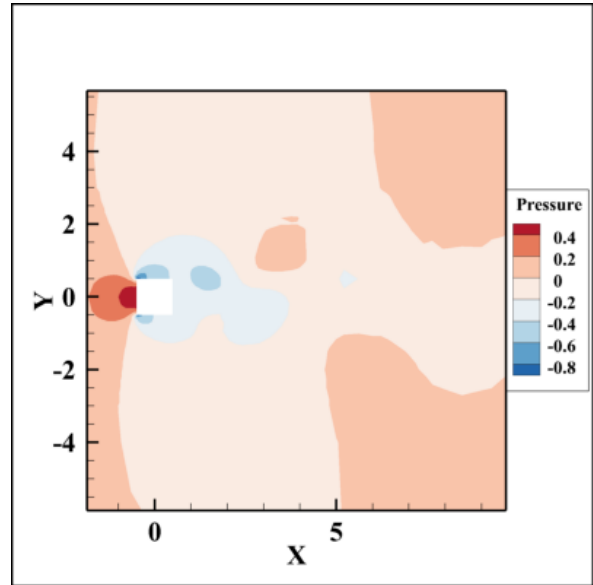
Table 7.6: Results for various SEM orders for square cylinder at $Re = 100$

Order of Interpolation	Quadrilateral elements	Total number of Nodes for Velocity and Pressure	Average Iterations/timestep			Computational Time (s)
			u^*	v^*	Pressure	
2 nd order	294	1248, 1248	1	1	21	914.2031
3rd order	294	2754, 2754	1	1	32	1645.1804
4th order	294	4848, 4848	1	1	41	2221.6833
5th order	294	7530, 7530	1	1	49	5192.4541
6th order	294	10800 ,10800	1	1	56	8948.7604

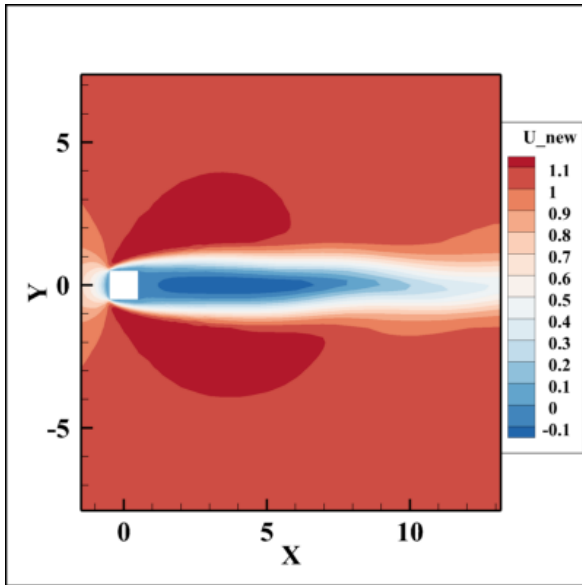
We observe very good accuracy with 4th order mesh and optimum computational cost. All the velocity, pressure contours and vorticity are plotted below along with lift and drag plots for 4th order SEM Mesh.



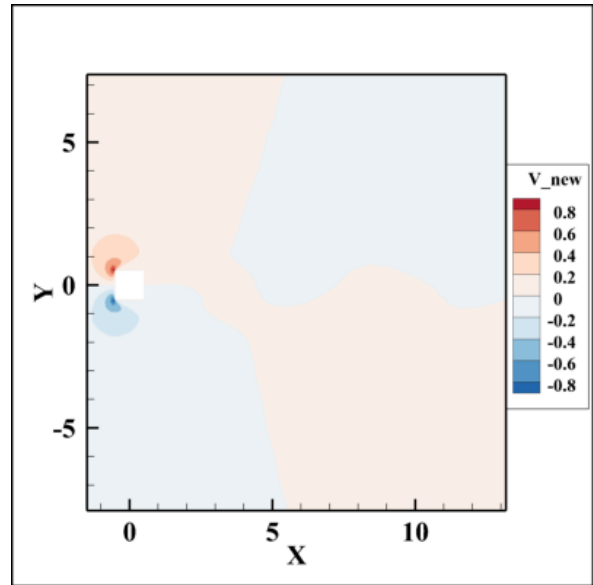
(a) Vorticity contour



(b) Pressure contour

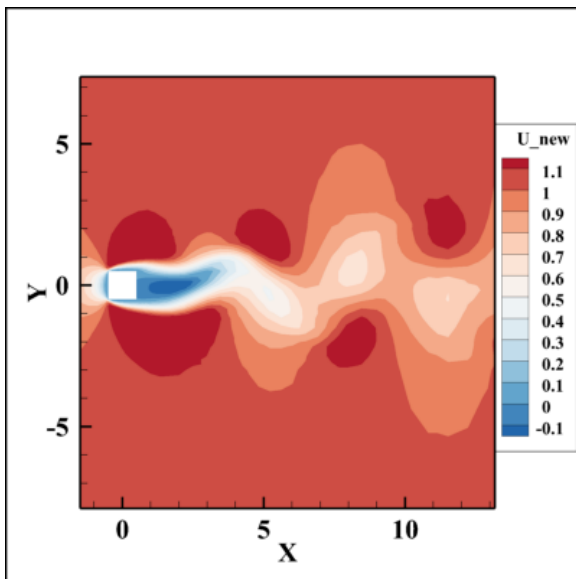


u velocity

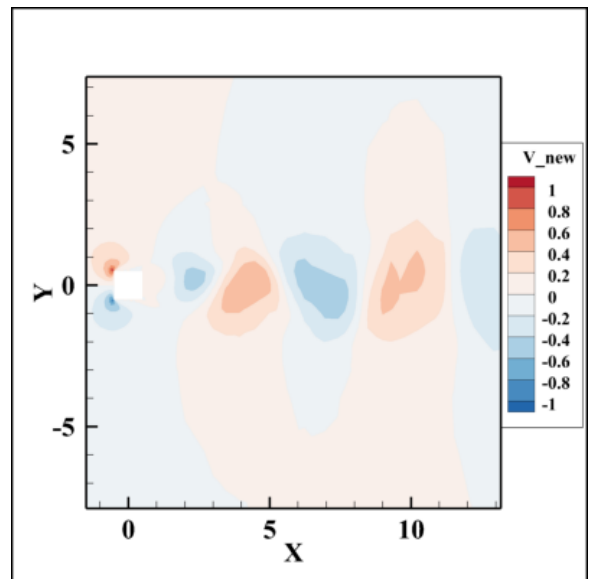


v velocity

(c) Velocity plots (Before Vortex Shedding at $t = 50$ s)

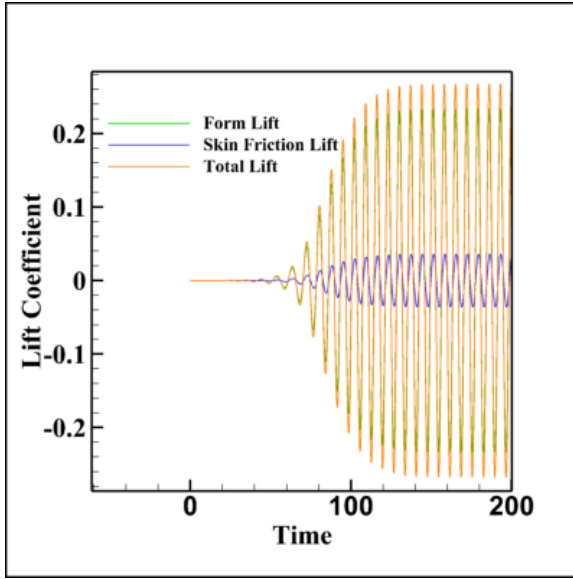


u velocity

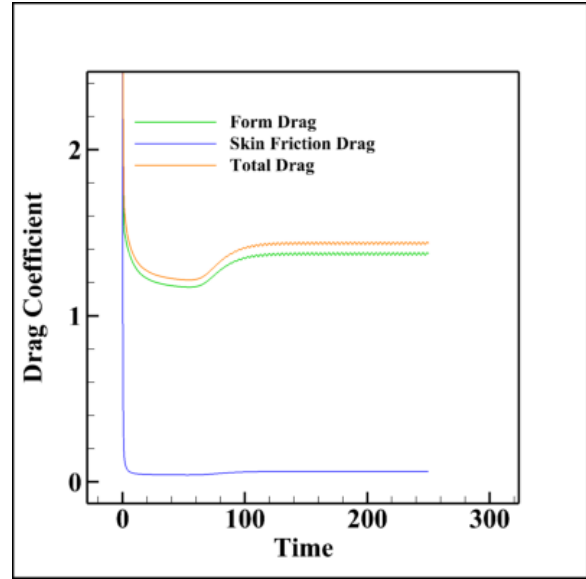


v velocity

(d) Velocity plots (After Vortex Shedding at $t = 200$ s)



(e) Lift coefficient



(f) Drag coefficient

Figure 7.15 4th order SEM plots for square cylinder

Table 7.7: Lift and Drag coefficients for 4th order SEM Mesh for circular cylinder at Re=100

4 th order SEM Mesh	Numerical Results		
	Mean Value	Peak Value	RMS value
Form Lift Coefficient ($C_{L,p}$)	0	0.23	0.163
Form Drag Coefficient ($C_{D,p}$)	1.375	-	-
Skin friction Lift Coefficient ($C_{L,v}$)	0	0.035	0.02475
Skin friction Drag Coefficient ($C_{D,v}$)	0.065	-	-
Total Lift Coefficient (C_L)	0	0.2675	0.189
Total Drag Coefficient (C_D)	1.438	-	-

Table 7.8: Validation of Lift & drag coefficients and Strouhal number with literature for square cylinder at Re = 100

4 th order SEM Mesh	SEM (4,4) (Present)	Zhao[40]	Sahu et al.[41]	Sen et al.[42]	Absolute Error (%)
C_D (mean)	1.438	1.45	1.4895	1.5287	3.263
C_L (rms)	0.189	0.188	0.188	0.1928	0.32
Strouhal Number (S_t)	0.144	0.145	0.1485	0.1452	1.53

Table 7.9: Mesh/Order Independence for square cylinder at $Re = 100$

Quadrilateral elements = 294							
Order of Interpolation	Total Nodes for Velocity and Pressure	Total Peak Lift			Total Mean Drag		
		Value	Difference	Absolute Error (%)	Value	Difference	Absolute Error (%)
2 nd order	1248	0.2997	0.04099	15.844	1.48	0.057	4.006
3rd order	2754	0.2998	0.04109	15.883	1.48	0.057	4.006
4th order	4848	0.26773	0.00902	3.486	1.438	0.015	1.054
5th order	7530	0.262	0.00329	1.272	1.428	0.005	0.3514
6th order	10800	0.25871	0	0	1.423	0	0

7.5 Flow around Elliptical cylinder:

Similar analysis as circular cylinder is done for elliptical cylinder. Here we need to make use of 2nd order SEM mesh as all edges of the bluff body are curved. The SEM results are presented here. All the flow parameters and geometric boundary conditions are depicted in figure (7.16). The Cylinder projected height on vertical plane is D and all other dimensions are expressed in terms of D . The Elliptical cylinder case also verifies the lift-drag calculations as the normal at surface here requires proper calculation and is not as straightforward as circular cylinder case. Here, Initial analysis is done for vertical and horizontal elliptical cylinder at $Re = 150$. Later, flow around inclined ellipse at $Re = 100$ has been presented and validated to assess solver's capability to simulate flow around unsymmetric obstruction.

Cylinder has no slip boundary condition ($u = v = 0$). The outlet has zero Neumann boundary condition ($\frac{\partial u}{\partial n} = \frac{\partial v}{\partial n} = 0$). The pressure at inlet, outlet, sidewall, and cylinder has Neumann boundary condition ($\frac{\partial p}{\partial n} = 0$). The simulation is run for 50000 timesteps with a time step of 0.005 for $Re = 150$. The Schematic diagram for this case is shown below.

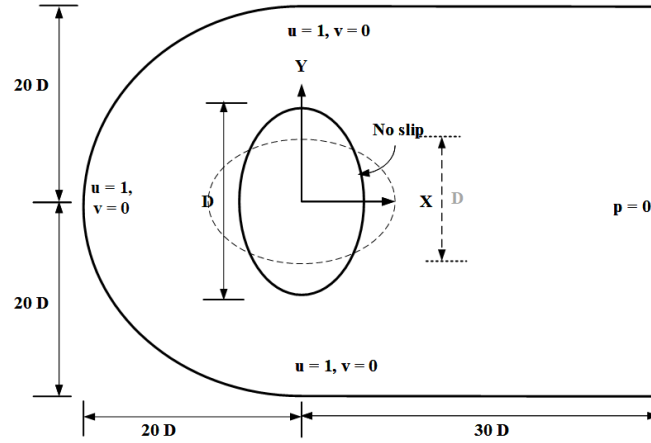
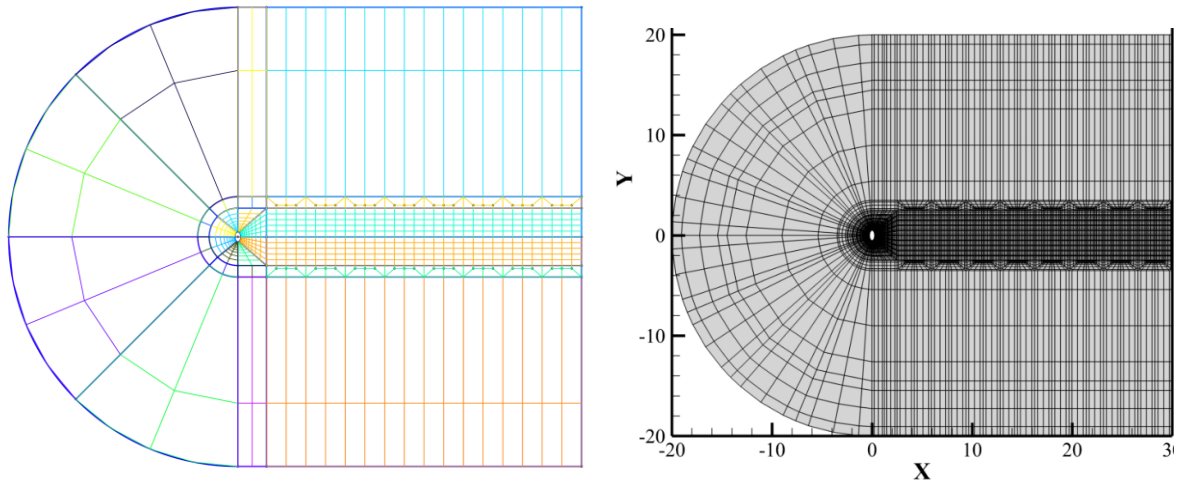


Figure 7.16 Schematic Diagram for elliptical cylinder

The analysis is done for Reynolds number $Re = \frac{UD}{\nu} = 150$, with free stream velocity of $U_{\infty} = 1$. All the plots for Vorticity, Velocity Contours and Lift & Drag are shown below along with mesh independence test. All the results have been simulated using PETSC CG Solver with ILU preconditioner.

SEM Mesh Results:

The GMSH mesh with 714 quadrilateral elements is plotted on left and corresponding 4th order SEM mesh is plotted on right.



(a) Initial GMSH Coarse Element Mesh

(b) Schematic Diagram for SEM Mesh

Figure 7.17 Initial GMSH coarse mesh and 4th order SEM mesh for vertical elliptical cylinder

The Mesh/Order independence results are tabulated below.

Table 7.10: Results for various SEM orders for vertical elliptical cylinder at $Re = 150$

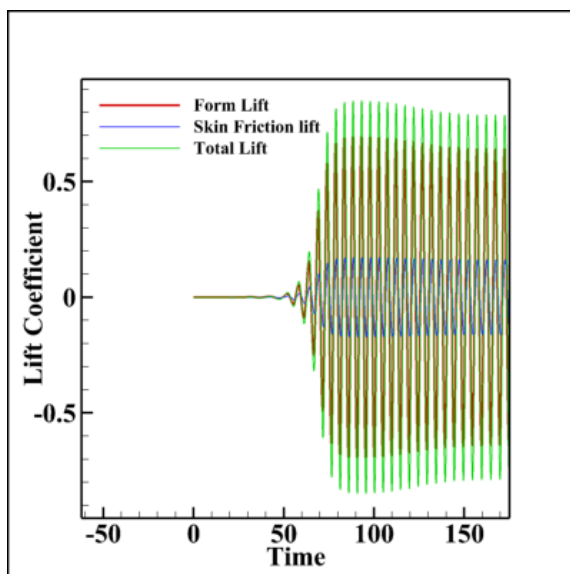
Vertical Ellipse						
Order of Interpolation	Quadrilateral elements	Total number of Nodes for Velocity and Pressure	Average Iterations/timestep			Computational Time (s)
			u^*	v^*	Pressure	
3rd order	714	6549, 6549	1	1	59	3073.4936
4th order	714	11588, 11588	1	1	76	7703.9256
5th order	714	18055, 18055	1	1	87	16187.6717

Table 7.11: Results for various SEM orders for horizontal elliptical cylinder at $Re = 150$

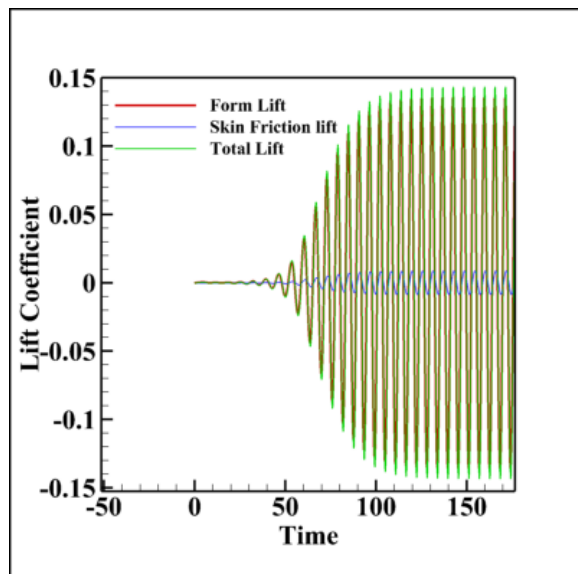
Horizontal Ellipse						
Order of Interpolation	Quadrilateral elements	Total number of Nodes for Velocity and Pressure	Average Iterations/timestep			Computational Time (s)
			u^*	v^*	Pressure	
3rd order	714	6549, 6549	1	1	52	5133.3984
4th order	714	11588, 11588	1	1	69	11544.8193
5th order	714	18055, 18055	1	1	82	23057.3479

We observe that we get good accuracy with 4th order mesh and optimum computational cost. All the velocity, pressure and vorticity contours are plotted below along with lift and drag plots for 4th order SEM mesh.

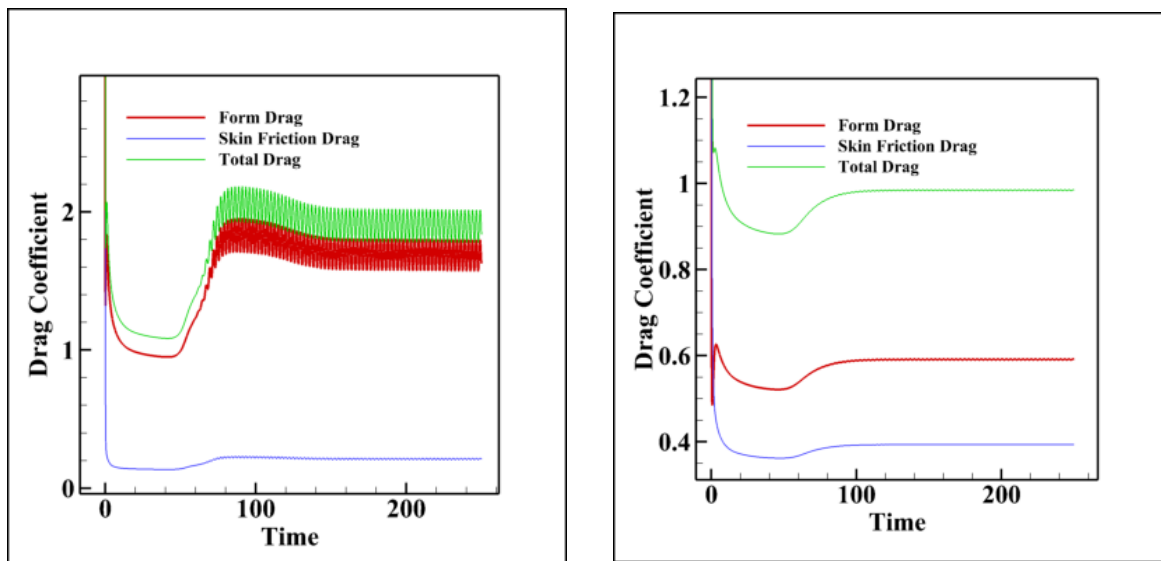
Vertical Ellipse



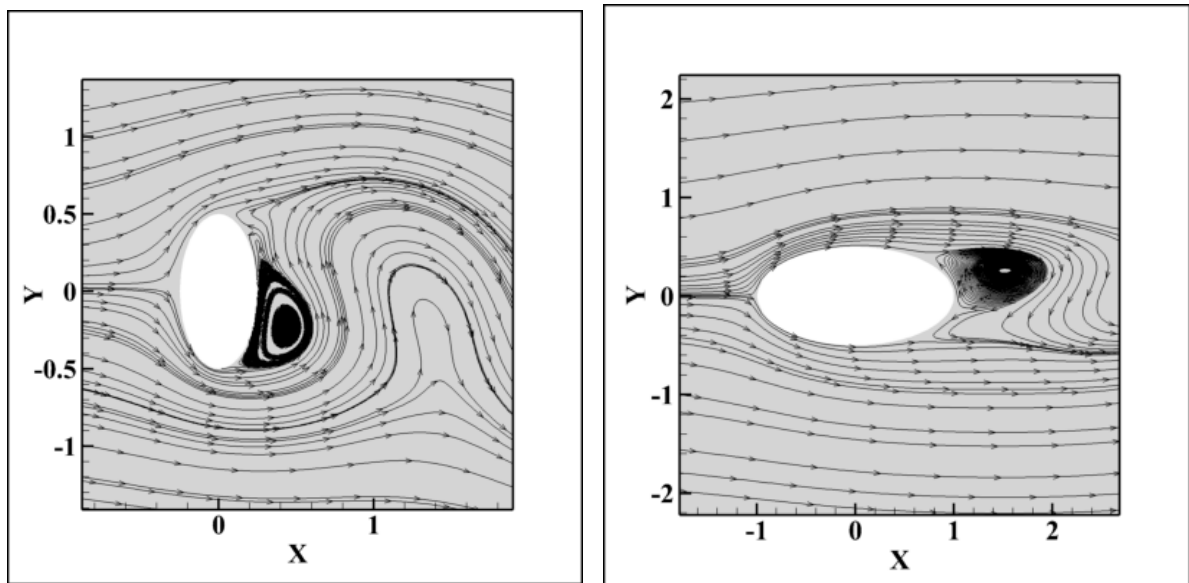
Horizontal Ellipse



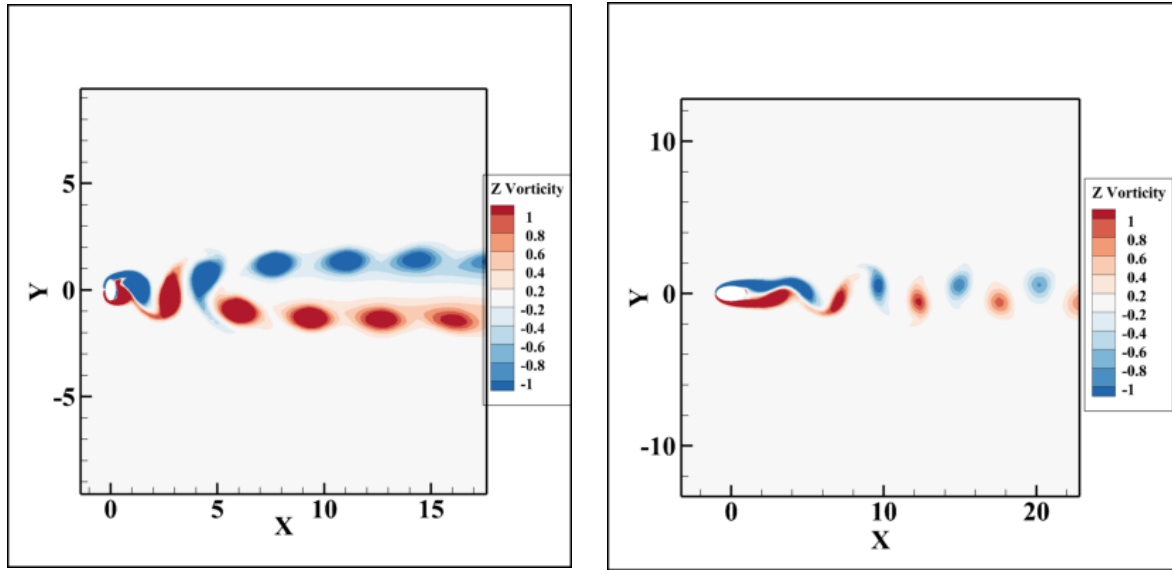
(a) Lift Coefficient



(b) Drag Coefficient



(c) Streamlines



(d) Vorticity Contours

Figure 7.18: 4th order SEM plots for elliptical cylinder

Table 7.12: Lift & Drag coefficients for 4th order SEM Mesh for elliptical cylinder at Re=150

4 th order SEM Mesh	Numerical Results					
	Vertical Ellipse			Horizontal Ellipse		
	Mean Value	Peak Value	RMS value	Mean Value	Peak Value	RMS value
Form Lift Coefficient ($C_{L,p}$)	0	0.16	0.11314	0	0.135	0.0954
Form Drag Coefficient ($C_{D,p}$)	1.68	-	-	0.59	-	-
Skin friction Lift Coefficient ($C_{L,v}$)	0	0.64	0.45255	0	0.008	0.00575
Skin friction Drag Coefficient ($C_{D,v}$)	0.21	-	-	0.395	-	-
Total Lift Coefficient (C_L)	0	0.79	0.55861	0	0.143	0.10116
Total Drag Coefficient (C_D)	1.89	-	-	0.985	-	-

The Lift and Drag values are validated with the literature below. Also Mesh/Order independence have been done for both cases. Here, too 4th order SEM analysis gives accurate results for Re = 150 case. Also, good gain in reduction in computational time is observed as compared to other commercial and opensource software. This solver can be further optimized and parallelized to decrease computational time.

Table 7.13: Validation of Lift & drag coefficients and Strouhal number with literature for elliptical cylinder at $Re = 150$

		Vertical Ellipse			Horizontal Ellipse		
4 th order SEM Mesh	SEM (4,4) (Present)	Xiaoyu Shi et al[43]	Thomson et al.[44]	Absolute Error (%)	SEM (4,4) (Present)	Xiaoyu Shi et al[43]	Absolute Error (%)
C_D (mean)	1.89	1.824	1.78	4.8	0.985	0.95	3.68
C_L (rms)	0.55861	0.53	-	5.4	0.10116	0.103	1.78
Strouhal Number (S_t)	0.2	0.191	0.189	5	0.172	0.169	1.77

Table 7.14: Mesh/Order Independence for elliptical cylinder at $Re = 150$

Quadrilateral elements = 714								
Ellipse geometr y	Order of Interpolation	Total Nodes for Velocity and Pressure	Total Peak Lift			Total Mean Drag		
			Value	Difference	Absolute Error (%)	Value	Difference	Absolute Error (%)
Vertical Ellipse	3rd order	6549	0.783	-0.005	0.63452	1.9	0.02	1.06383
	4th order	11588	0.79	0.002	0.25381	1.89	0.01	0.53192
	5th order	18055	0.788	0	0	1.88	0	0
Horizon tal Ellipse	3rd order	6549	0.149	0.006	4.19580	0.9805	-0.0095	0.9596
	4th order	11588	0.143	0	0	0.97	-0.02	2.0202
	5th order	18055	0.143	0	0	0.99	0	0

Inclined Ellipse

The Schematic Diagram of Geometry remains same as above case, only the ellipse is now inclined by 45° with the horizontal as shown in the figure. The analysis is done for 50000 timesteps at $Re = 100$. Here, $D = 1$, & the geometry is defined in terms of D . The results are simulated for 4th order SEM mesh. They are plotted below. They are in good agreement with the literature.

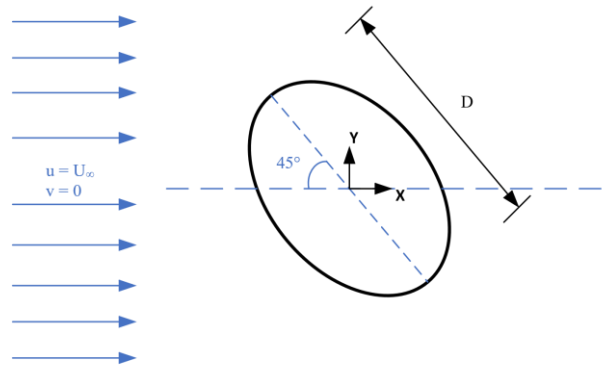
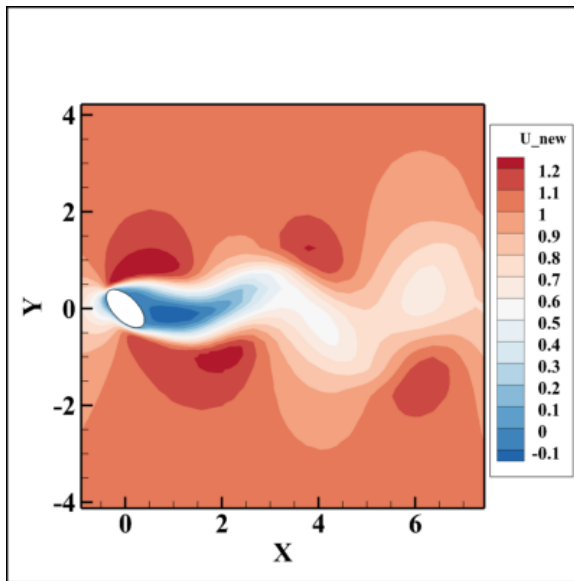
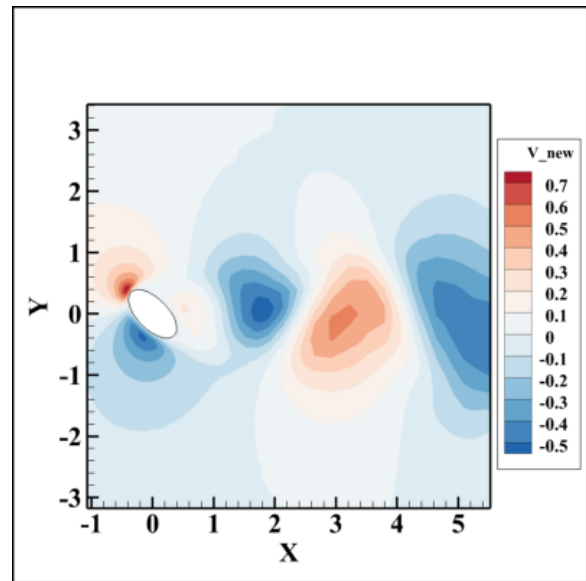


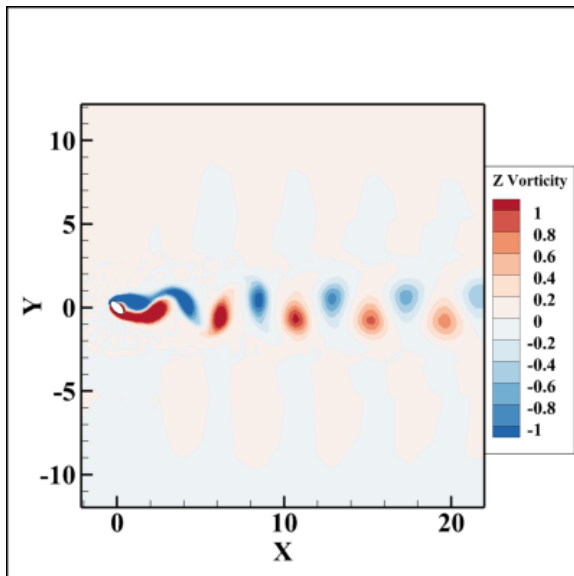
Figure 7.19: 45° backward inclined elliptical cylinder at $Re = 100$



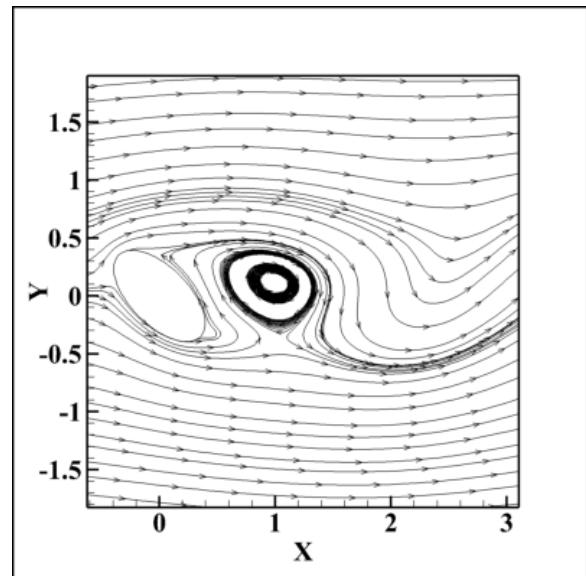
(a) u velocity contour ($t=200$ s)



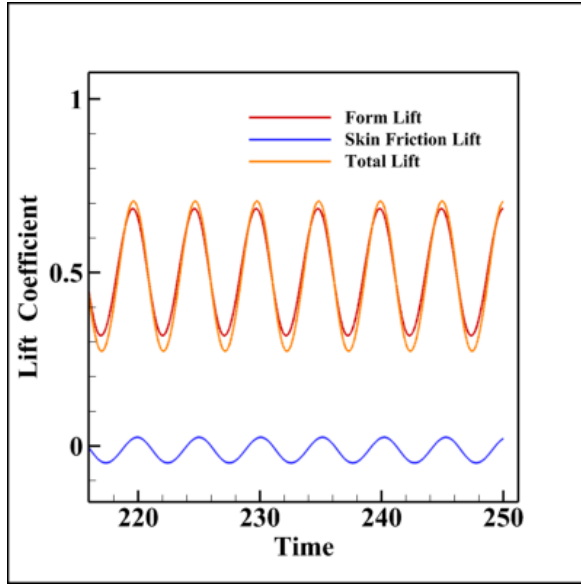
(b) v velocity contour ($t=200$ s)



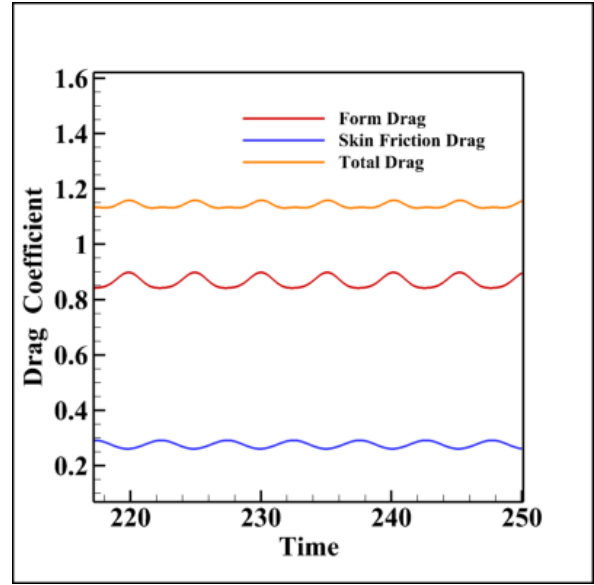
(c) Vorticity contour ($t=200$ s)



(d) Streamlines ($t=200$ s)



(e) Lift Coefficient



(f) Drag Coefficient

Figure 7.20: 4th order SEM plots for 45° backward inclined elliptical cylinder at $Re = 100$

Table 7.15: Lift-Drag calculation for 45° inclined elliptical cylinder at $Re = 100$

4 th Order SEM Mesh	Numerical Simulation		Validation Subhankar Sen et al.[45]	
	Mean Value	RMS value	Mean value	RMS value
Form Lift Coefficient ($C_{L,p}$)	0.4895	0.1295	0.5116	-
Form Drag Coefficient ($C_{D,p}$)	0.871	-	0.8768	-
Skin friction Lift Coefficient ($C_{L,v}$)	0.0121	0.02348	0.0129	-
Skin friction Drag Coefficient ($C_{D,v}$)	0.2758	-	0.2747	-
Total Lift Coefficient (C_L)	0.5012	0.15298	0.4987	0.1531
Total Drag Coefficient (C_D)	1.145	-	1.1515	-

Summary and Future Work

8.1. Summary

In this project, a lower order FEM and higher order FEM/SEM approach is used to create and validate a solver for simulating fluid flows. The initial analysis uses Taylor-hood element with 2-1 order approximation for velocity-pressure requiring fine mesh. Elementary problems like Lid driven cavity flow problems, Poiseuille's flow problem are solved by using Element-by-element Conjugate gradient method approach. The flow around obstacle problems is solved by global assembly approach and sparse linear storage solvers. The Spectral Element Method showcased its ability to generate solutions of comparable accuracy to the Finite Difference Method (FDM) while utilizing a lower degree of freedom, making it an efficient choice for computational fluid dynamics simulations. The Spectral Element Method offers the benefit of incorporating both p-type and h-type refinement. Using higher-order polynomials results in improved accuracy while reducing the required number of nodes.

PETSC's Iterative solvers are currently used with ILU preconditioners to obtain faster results. Also, currently an in-house Conjugate Gradient solver using matrices in CSR format is under development. SEM Mesh generation and node renumbering will ensure to avoid recalculating the Dirichlet boundary nodes solution using solver. It also stores the adjacent element information & all edge-element associations, which will be very fruitful for FSI problems and parallelization. Usage of curvilinear mesh captures curvature of bodies with few elements.

My MTP work, so far, included optimizing the Spectral Element Method solver and making it more robust by correcting the calculation of Lift-Drag forces. The previous version of code written in C++ was giving incorrect Lift-Drag forces for some bodies. The results have been validated for flow around square, circular and elliptical cylinder. Code restructuring in Fortran, generating SEM mesh in Fortran, along with correcting Lift-Drag calculation and development of sparse storage matrices to develop in-house CG solver mimicking existing linear solver library PETSC are some of my contributions. Significant reductions in computational expenses with efficient and correct solution of Lift-Drag forces will enable current solver to solve complex FSI problems with much more ease and accuracy.

The goal of the work, outlined at the beginning of the thesis has been substantially met, but there is still lot to be improved. Future scope will highlight the next steps for expanding the solver.

8.2. Future Work

The current solver uses PETSC linear solver library for solving system of equations in iterative manner. The in-house CG solver can further be enhanced to use multiple other preconditioners including finite element and lower order SEM itself as preconditioner. The sparse storage of global matrix enables us to also develop inhouse direct solver like Cholesky decomposition method (LL^T) using BLAS and LAPACK libraries to extract required L & L^T matrices. The node renumbering can be used to eliminate the boundary nodes completely from sparse storage format. The element-edge associations can be utilised to solve the FSI problems at element level parallelly. VIV problems can be explored using non-inertial frame of reference, overset method, stair-step method, immersed boundary method, etc.

The code can be expanded to capture flow dynamics at high Reynolds number ($Re > 200$) using techniques like Streamline Upwind/Petrov-Galerkin, utilization of bubble functions to stabilize the advective term, and by using interpolation-based filtering technique. Direct and iterative methods can be independently applied to solve the momentum and pressure-Poisson equations, enabling solver optimization and acceleration as per the specific requirements. The code can be parallelized to solve interface nodes first and internal nodes later in parallel manner using renumbered SEM mesh. The solver can be enhanced to capture unsteady flow using element level solution approach like domain decomposition. As FEM is also used to solve solid mechanics problems, the solver can be expanded to capture the interaction and deformation of solid in fluids (FSI problems). Variable order SEM in different subdomains of main domain can be implemented.

Some of the main key points which will be primarily addressed in future scope are as follows:

- Completing and enhancing the in-house CG solver and developing direct solvers
- Expanding this code to solve Vortex Induced Vibration (VIV) problems using a non-inertial reference frame and overset method
- Adapting parallel programming to reduce computational cost
- Enable the solver to simulate high Reynolds number flow
- Analysis of Fluid Structure Interaction (FSI) problems

References

- [1] A. T. Patera, "A spectral element method for fluid dynamics: Laminar flow in a channel expansion," *J Comput Phys*, vol. 54, no. 3, 1984, doi: 10.1016/0021-9991(84)90128-1.
- [2] F. X. Giraldo *et al.*, "An Introduction to Element-Based Galerkin Methods on Tensor-Product Bases Analysis, Algorithms, and Applications." [Online]. Available: <http://www.springer.com/series/5151>
- [3] M. R. Schumack, W. W. Schultz, and J. P. Boyd, "Spectral Method Solution of the Stokes Equations on Nonstaggered Grids," 1991.
- [4] "A Spectral Element Basin Model for the shallow Water Equations - 1-s2.0-S0021999183712052-main".
- [5] M. A. Shallal, A. H. Taqi, B. F. Jumaa, H. Rezazadeh, and M. Inc, "Numerical solutions to the 1-D Burgers' equation by a cubic Hermite finite element method," *Indian Journal of Physics*, vol. 96, no. 13, pp. 3831–3836, Nov. 2022, doi: 10.1007/s12648-022-02304-4.
- [6] A. K. Soti, M. C. Thompson, J. Sheridan, and R. Bhardwaj, "Harnessing electrical power from vortex-induced vibration of a circular cylinder," *J Fluids Struct*, vol. 70, pp. 360–373, Apr. 2017, doi: 10.1016/j.jfluidstructs.2017.02.009.
- [7] M. Deville and E. Mund, "Chebyshev Pseudospectral Solution of Second-Order Elliptic Equations with Finite Element Preconditioning," 1985.
- [8] G. J. Le Beau and T. E. Tezduyar, "FINITE ELEMENT COMPUTATION OF COMPRESSIBLE FLOWS WITH THE SUPG FORMULATION," 1991.
- [9] F. Brezzi, M.-O. Bristeau, L. P. Franca, M. Mallet, and G. Rog6, "A relationship between stabilized finite element methods and the Galerkin method with bubble functions," 1992.
- [10] P. F. Fischer, "An Overlapping Schwarz Method for Spectral Element Solution of the Incompressible Navier-Stokes Equations," 1997.
- [11] S. K. Tomar, "h-p spectral element method for elliptic problems on non-smooth domains using parallel computers," *Computing (Vienna/New York)*, vol. 78, no. 2, 2006, doi: 10.1007/s00607-006-0176-0.
- [12] H. Jiang and L. Cheng, "Large-eddy simulation of flow past a circular cylinder for Reynolds numbers 400 to 3900," *Physics of Fluids*, vol. 33, no. 3, Mar. 2021, doi: 10.1063/5.0041168.
- [13] A. J. Chorin, "Numerical solution of the navier-stokes equations," *Mathematics of computation*, vol. 22, no. 104, pp. 745–762, 1968.
- [14] K. Goda, "A multistep technique with implicit difference schemes for calculating two-or three-dimensional cavity flows," *J Comput Phys*, vol. 30, no. 1, pp. 76–95, 1979.
- [15] J. B. Bell, P. Colella, and L. H. Howell, "An efficient second-order projection method for viscous incompressible flow," in *10th Computational Fluid Dynamics Conference, 1991*, 1991. doi: 10.2514/6.1991-1560.

- [16] J. Kim and P. Moin, "Application of a fractional-step method to incompressible Navier-Stokes equations," *J Comput Phys*, vol. 59, no. 2, 1985, doi: 10.1016/0021-9991(85)90148-2.
- [17] J. van Kan, "A Second-Order Accurate Pressure-Correction Scheme for Viscous Incompressible Flow," *SIAM Journal on Scientific and Statistical Computing*, vol. 7, no. 3, 1986, doi: 10.1137/0907059.
- [18] D. L. Brown, R. Cortez, and M. L. Minion, "Accurate Projection Methods for the Incompressible Navier-Stokes Equations," *J Comput Phys*, vol. 168, no. 2, 2001, doi: 10.1006/jcph.2001.6715.
- [19] "A fractional-step method for the incompressible Navier–Stokes equations related to a predictor–multicorrector algorithm".
- [20] R. Codina and J. B. &c&i Tcnica, "A finite element formulation for the Stokes problem allowing equal velocity-pressure interpolation," 1997.
- [21] R. Codina, "Pressure Stability in Fractional Step Finite Element Methods for Incompressible Flows," *J Comput Phys*, vol. 170, no. 1, pp. 112–140, Jun. 2001, doi: 10.1006/jcph.2001.6725.
- [22] J. Blair Perot, "An analysis of the fractional step method," *J Comput Phys*, vol. 108, no. 1, 1993, doi: 10.1006/jcph.1993.1162.
- [23] B. J. et al G. Guennebaud, "C++ Documentation - Eigen solver," Eigen v3, <http://eigen.tuxfamily.org>, 2010.
- [24] "PETSC KSP Solver," <https://petsc.org/release/manual/ksp/>.
- [25] J. W. H. Liu, "Multifrontal method for sparse matrix solution. Theory and practice," *SIAM Review*, vol. 34, no. 1, 1992, doi: 10.1137/1034004.
- [26] Y. Saad, *Iterative Methods for Sparse Linear Systems, Second Edition*. 2003.
- [27] I. S. Duff, A. M. Erisman, and J. K. Reid, *Direct Methods for Sparse Matrices*. 2017. doi: 10.1093/acprof:oso/9780198508380.001.0001.
- [28] C. Taylor and P. Hook, "A NUMERICAL SOLUTION OF THE NAVIER-STOKES EQUATIONS USING THE FINITE ELEMENT TECHNIQUE," Pergamoo Press, 1973.
- [29] K. Vacek and P. Sváček, "On finite element approximation of fluid structure interaction by Taylor-Hood and Scott-Vogelius elements," Institute of Mathematics, Czech Academy of Sciences, Apr. 2023, pp. 259–268. doi: 10.21136/panm.2022.24.
- [30] D. R. Q. Pacheco and O. Steinbach, "SPACE-TIME TAYLOR-HOOD ELEMENTS FOR INCOMPRESSIBLE FLOWS 1".
- [31] B. S. Hosseini, M. Möller, and S. Turek, "Isogeometric Analysis of the Navier-Stokes equations with Taylor-Hood B-spline elements," *Appl Math Comput*, vol. 267, pp. 264–281, Sep. 2015, doi: 10.1016/j.amc.2015.03.104.
- [32] K. Morgan and J. Peraire, "Unstructured grid finite-element methods for fluid mechanics," *Reports on Progress in Physics*, vol. 61, no. 6, 1998, doi: 10.1088/0034-4885/61/6/001.

- [33] T. BELYTSCHKO, "The Finite Element Method: Linear Static and Dynamic Finite Element Analysis: Thomas J. R. Hughes," *Computer-Aided Civil and Infrastructure Engineering*, vol. 4, no. 3, 1989, doi: 10.1111/j.1467-8667.1989.tb00025.x.
- [34] S. V. Patankar, "A calculation procedure for two-dimensional elliptic situations," *Numerical Heat Transfer*, vol. 4, no. 4, 1981, doi: 10.1080/01495728108961801.
- [35] <https://www.youtube.com/watch?v=bDbUoRrT6Js>, "Sparse Matrix Computation," <https://www.youtube.com/watch?v=H6YGKNukGMO>.
- [36] U. Ghia, K. N. Ghia, and C. T. Shin, "High-Re Solutions for Incompressible Flow Using the Navier-Stokes Equations and a Multigrid Method*," 1982.
- [37] J. Park and H. Choi, "Numerical solutions of flow past a circular cylinder at reynolds numbers up to 160," *KSME International Journal*, vol. 12, no. 6, 1998, doi: 10.1007/BF02942594.
- [38] A. G. Kravchenko, P. Moin, and K. Shariff, "B-Spline methods and zonal grids for numerical simulations of turbulent flows," *J Comput Phys*, vol. 151, no. 2, 1999.
- [39] E. Stålberg, A. Brüger, P. Lötstedt, A. V. Johansson, and D. S. Henningson, "High order accurate solution of flow past a circular cylinder," in *Journal of Scientific Computing*, 2006. doi: 10.1007/s10915-005-9043-y.
- [40] M. Zhao, "Flow-induced vibrations of square and rectangular cylinders at low Reynolds number," *Fluid Dyn Res*, vol. 47, no. 2, 2015, doi: 10.1088/0169-5983/47/2/025502.
- [41] A. K. Sahu, R. P. Chhabra, and V. Eswaran, "Two-dimensional unsteady laminar flow of a power law fluid across a square cylinder," *J Nonnewton Fluid Mech*, vol. 160, no. 2–3, 2009, doi: 10.1016/j.jnnfm.2009.03.010.
- [42] S. Sen, S. Mittal, and G. Biswas, "Flow past a square cylinder at low Reynolds numbers," *Int J Numer Methods Fluids*, vol. 67, no. 9, 2011, doi: 10.1002/fld.2416.
- [43] X. Shi, M. Alam, and H. Bai, "Wakes of elliptical cylinders at low Reynolds number," *Int J Heat Fluid Flow*, vol. 82, 2020, doi: 10.1016/j.ijheatfluidflow.2020.108553.
- [44] M. C. Thompson, A. Radi, A. Rao, J. Sheridan, and K. Hourigan, "Low-Reynolds-number wakes of elliptical cylinders: From the circular cylinder to the normal flat plate," *J Fluid Mech*, vol. 751, 2014, doi: 10.1017/jfm.2014.314.
- [45] H. Sarkar and S. Sen, "VORTEX-INDUCED VIBRATIONS OF AN ELLIPTIC CYLINDER AT DIFFERENT ANGLES OF INCIDENCE," in *Proceedings of the Thermal and Fluids Engineering Summer Conference*, 2023. doi: 10.1615/tfec2023.fna.046072.